תיכון אלון לאמנויות ולמדעים רמת השרון

מגיש: איתן שטמלר

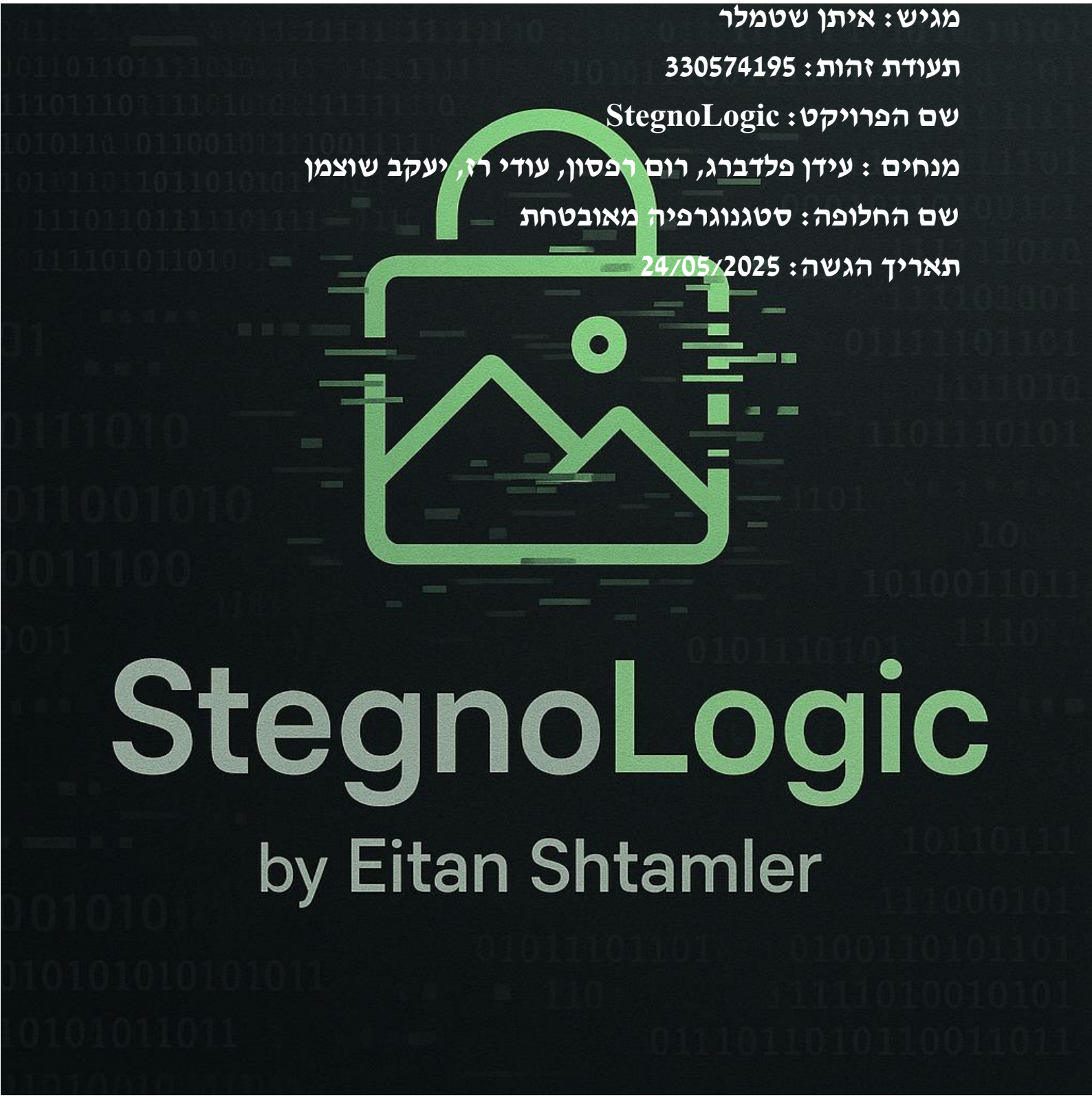
תעודת זהות: 330574195

שם הפרויקט: StegnoLogic

מנחים : עידן פלדברג, רום רפסון, עודי רז, יעקב שוצמן

שם החלופה: סטגנוגרפיה מאובטחת

תאריך הגשה: 24/05/2025



ייזום 4
תיאור מערכת ראשוני..... 4
אתגרם עיקריים..... 4
הגדרת לקוח 5
מטרות ויעדים 5
בעיות..... 6
יתרונות..... 7
סקירת פתרונות קיימים..... 8
ייחודיות..... 8
סקירת טכנולוגיה..... 9
מגבלות ואתגרים טכניים..... 10
היקף הפרויקט..... 10
תיאור המערכת 12
מנגנוני אבטחה..... 13
יכולות משתמש..... 13
מבחנים מתוכננים(קופסה שחורה) 14
ניהול זמן ופיתוח..... 15
תיאור תחום הידע 17
יכולות צד השרת..... 17
ניהול משתמשים ואימות זהות..... 17
ניהול לוגים ואבטחת שרת..... 19
סטגנוגרפיה..... 21
ניהול תקשורת מאובטחת..... 22
שליחת מיילים והודעות..... 24
יכולות צד הלקוח..... 25
ממשק גרפי מתקדם..... 25
רישום והתחברות מאובטחים..... 27
הצפנת מידע בתמונות..... 29
תקשורת מאובטחת עם השרת..... 31
ארכיטקטורת הפרויקט ותכנון המערכת 34
תיאור ארכיטקטורת המערכת..... 34
דיאגרמות RSA ו-HASH 35
טכנולוגיות רלוונטיות..... 36
תיאור זרימת הנתונים..... 37

תהליך הרישום..... 37
תהליך ההתחברות ואימות דו-שלבי..... 39
תהליך ההצפנה והפיענוח מידע מתמונה..... 40
אלגוריתמים בפרויקט..... 42
אלגוריתם LSB 42
אלגוריתם הצפנה RSA 43
אלגוריתם אחסון סיסמאות PBKDF2 44
אלגוריתם זיהוי והגנה מDDOS 45
תיאור פרוטוקול התקשרות..... 47
סוגי הודעות ומבנה..... 48
מסכי המערכת..... 52
היררכיית מסכים..... 62
UML הכולל קשרים בין המחלקות..... 63
מבני נתונים..... 64
טבלאות בסיס נתונים..... 65
יחסים בין הטבלאות ERD 66
סקירת אבטחה..... 67
יישום הפרויקט..... 72
מודולים וספריות מיובאות..... 72
מודולים ומחלקות מוגדרות על ידי המשתמש..... 73
אתגרים אלגוריתמיים..... 81
תיעוד בדיקות נוספות..... 87
רפלקציה..... 96
ביבליוגרפיה..... 99
נספחים - קוד הפרויקט שלי StegnoLogic 100
Server..... 100
client..... 110
SecurityCrypto..... 119
SteganographyHandler..... 124
EmailHandler..... 129
DatabaseHandler..... 131
Gui_app..... 137
requirements..... 177
.env..... 177

מבוא

ייזום

תיאור מערכת ראשוני

המערכת שפיתחתי, StegnoLogic, היא תוכנה לסטגנוגרפיה מאובטחת המאפשרת למשתמשים להסתיר הודעות סודיות בתוך תמונות. בניגוד לשיטות הצפנה רגילות שמסתירות את תוכן ההודעה אך משאירות גלוי את עצם קיום ההודעה, סטגנוגרפיה הולכת צעד אחד קדימה - היא מסתירה את עצם העובדה שבכלל יש תקשורת סודית.

המערכת מבוססת על ארכיטקטורת לקוח-שרת, כאשר הלקוח מאפשר למשתמשים לנהל את החשבון שלהם, להצפין הודעות בתמונות ולפענח הודעות שנשלחו אליהם. השרת מטפל בניהול המשתמשים, אחסון ההודעות והתמונות, ומבצע את תהליכי ההצפנה והפענוח.

בחרתי בפרויקט זה כי אני מאוד מתעניין בהצפנה ואבטחת מידע, במיוחד בתקופה זו של מלחמת חרבות ברזל. העולם הדיגיטלי שלנו הופך להיות יותר ויותר חשוף, וטכניקות כמו סטגנוגרפיה נותנות פתרון מעניין לשמירה על פרטיות. לדעתי, היכולת להעביר מידע בצורה שלא מושכת תשומת לב היא יכולת חשובה בעידן שבו כל התקשורת שלנו מנוטרת.

האתגרים העיקריים שצפיתי לפגוש בפיתוח המערכת היו:

1. **יישום אלגוריתם הסטגנוגרפיה** : להסתיר מידע בתמונות בלי לשנות אותן באופן גלוי לעין.
2. **פיתוח מערכת אבטחה חזקה** : ליצור מנגנונים כמו הצפנת RSA, אימות דו-שלבי, הגנת DDOS ואחסון סיסמאות מאובטח.
3. **בניית ממשק משתמש נוח** : ליצור ממשק שיהיה נוח לשימוש למרות המורכבות הטכנית של המערכת.
4. **תקשורת לקוח-שרת מאובטחת** : להבטיח שכל התקשורת בין הלקוח לשרת תהיה מוצפנת היטב.
5. **טיפול בקבצי תמונה** : לפתח מנגנון שיאפשר הסתרת הודעות בתמונות מסוגים שונים תוך שמירה על איכות התמונה.

הגדרת לקוח

המערכת מיועדת למספר קהלי יעד:

1. **אנשים פרטיים המעוניינים בפרטיות**: המערכת אידיאלית לאנשים שרוצים לתקשר באופן פרטי בלי שאף אחד ידע אפילו שהם מתקשרים. למשל, אנשים החיים במדינות עם צנזורה או עיתונאים המעבירים מידע רגיש.

2. **סטודנטים וחוקרים בתחום אבטחת המידע**: המערכת יכולה לשמש כדוגמה לימודית לטכניקות סטגנוגרפיה ואבטחת מידע.

3. **עסקים קטנים**: ארגונים המעוניינים בתקשורת פנימית מאובטחת ופרטית, במיוחד עבור מידע רגיש כמו סודות מסחריים.

4. **מערכות צבאיות**: אגף המודיעין בצה"ל ובצבאות העולם יכולים להיעזר במערכת על מנת להצפין ולפענח מסרים סודיים אשר הם לא רוצים שיתגלו לאויב.

המערכת תומכת במספר סוגי משתמשים:

- **שולחים**: משתמשים היכולים להצפין הודעות בתמונות ולשלוח אותן לאחרים
- **מקבלים**: משתמשים היכולים לפענח הודעות שנשלחו אליהם
- משתמש יכול להיות גם שולח וגם מקבל, אך ההודעות משויכות ספציפית למשתמשים מסוימים

חשוב לציין שהמערכת נבנתה לתמיכה במשתמשים מרובים, עם מערכת ניהול חשבונות מלאה, ולא רק כפתרון עבור משתמש בודד.

מטרות ויעדים

המטרות העיקריות של מערכת StegnoLogic הן:

1. **הסתרת מידע בתמונות**: לאפשר למשתמשים להסתיר הודעות טקסט בתוך תמונות בצורה שאינה ניתנת להבחנה ויזואלית.

2. **אבטחת תקשורת**: להבטיח שכל התקשורת בין המשתמשים לשרת תהיה מאובטחת באמצעות הצפנת RSA והחלפת מפתחות מאובטחת.

3. **ניהול משתמשים מאובטח**: לספק מערכת רישום והתחברות מאובטחת עם אימות דו-שלבי דרך אימייל ואחסון סיסמאות מוצפן.

4. **שליחת הודעות מאובטחת** : לאפשר למשתמשים לשלוח הודעות מוסתרות למשתמשים אחרים, כולל יכולת לשליחה למספר נמענים במקביל.

5. **הגנה מפני האזנות והתקפות** : למנוע מגורמים זרים לגלות את קיום ההודעות המוסתרות או לפענח אותן ללא הרשאה.

6. **ממשק משתמש ידידותי** : לספק חווית משתמש פשוטה ואינטואיטיבית למרות המורכבות הטכנית של המערכת.

7. **התראות והודעות אימייל** : לשלוח הודעות אימייל אוטומטיות למשתמשים בעת קבלת הודעות חדשות או כחלק מתהליך האימות.

8. **הגנה מפני התקפות DDOS** : להטמיע מנגנוני הגבלת קצב כדי למנוע התקפות מניעת שירות.

בעיות, יתרונות וחסכונות

הבעיות שהמערכת פותרת:

1. **חוסר בפרטיות בתקשורת דיגיטלית** : היום, רוב ערוצי התקשורת מנוטרים. גם אם התוכן מוצפן, עצם קיום התקשורת עדיין גלוי לעיניים מפקחות. StegnoLogic פותרת בעיה זו על ידי הסתרת עצם קיום ההודעה.

2. **מחסור בפתרונות סטגנוגרפיה ידידותיים למשתמש** : רוב כלי הסטגנוגרפיה הקיימים דורשים ידע טכני רב או מורכבים לשימוש. המערכת שלי נותנת פתרון פשוט יותר.

3. **אתגרי אבטחה בכלי סטגנוגרפיה** : הרבה מהכלים הקיימים לא משלבים שיטות אבטחה מתקדמות, כמו הצפנה חזקה ואימות דו-שלבי.

4. **מעקב אחרי מי יכול לפענח הודעות** : בכלים רבים, כל מי שיודע על קיום ההודעה יכול לנסות לפענח אותה. ב-StegnoLogic, רק הנמענים המיועדים יכולים לפענח את ההודעה.

היתרונות והתועלות של המערכת:

1. **פרטיות מוגברת** : משתמשים יכולים לתקשר ביניהם בלי להשאיר עקבות גלויים של התקשורת.

2. **שכבות אבטחה מרובות** : המערכת משלבת סטגנוגרפיה, הצפנת RSA, ואימות דו-שלבי ליצירת פתרון אבטחה רב-שכבתי.

3. **ממשק קל לשימוש** : למרות הטכנולוגיה המורכבת, הממשק נוח וידידותי, מה שמאפשר גם למשתמשים שאינם טכניים להשתמש במערכת.

4. **תמיכה במספר נמענים** : היכולת לשלוח הודעה מוצפנת למספר נמענים במקביל.

5. **התרעות אוטומטיות** : המערכת יכולה לשלוח התראות דרך אימייל כשמתקבלת הודעה חדשה.

חסכונות:

1. **חיסכון בסיכוני פרטיות** : הפחתת הסיכון לחשיפת תקשורת רגישה, מה שעשוי לחסוך בנזקים פוטנציאליים גדולים.

2. **חיסכון בזמן ומאמץ** : במקום להשתמש במספר כלים שונים (הצפנה, העברת קבצים, אימות), StegnoLogic מספקת פתרון אחד משולב.

3. **צמצום עלויות תוכנה** : מכיוון שהמערכת היא פתרון כולל, אין צורך ברכישת מספר מערכות שונות.

4. **הפחתת סיכוני אבטחה** : הגנה מפני התקפות ודליפות מידע יכולה לחסוך עלויות גבוהות של טיפול בהפרות אבטחה.

5. **חיסכון באמצעי תקשורת** : אין צורך בערוצי תקשורת מיוחדים, כי הודעות יכולות להישלח כתמונות רגילות לכל דבר ועניין.

סקירת פתרונות קיימים

ישנם מספר כלים וטכנולוגיות קיימים בתחום הסטגנוגרפיה. להלן סקירה של הפתרונות העיקריים והשוואתם ל-StegnoLogic:

1. **OpenStego**:

- **יתרונות**: פתרון קוד פתוח פופולרי, תומך במספר אלגוריתמים, ממשק גרפי בסיסי.

- **חסרונות**: חסר בשכבות אבטחה נוספות, אין מערכת ניהול משתמשים, חסר באימות דו-שלבי.

- **השוואה ל-StegnoLogic**: StegnoLogic מציעה אבטחה טובה יותר עם הצפנת RSA ואימות דו-שלבי, וכן ניהול משתמשים מובנה.

2. **Steghide**:

- **יתרונות**: כלי קוד פתוח חזק, אלגוריתם סטגנוגרפיה יעיל.

- **חסרונות**: ממשק שורת פקודה בלבד (CLI), לא ידידותי למשתמשים לא טכניים, אין ניהול משתמשים.

- **השוואה ל-StegnoLogic**: StegnoLogic מציעה ממשק גרפי נוח הרבה יותר ומערכת לקוח-שרת מלאה.

מה שהופך את StegnoLogic לייחודית:

1. **אבטחה רב-שכבתית**: שילוב של סטגנוגרפיה, הצפנת RSA, ואימות דו-שלבי במערכת אחת.

2. **ארכיטקטורת לקוח-שרת**: בניגוד לרוב הכלים שהם אפליקציות מקומיות בלבד, StegnoLogic מציעה מערכת מבוזרת עם אבטחה מובנית.

3. **ניהול משתמשים מלא**: רישום, התחברות ומערכת הרשאות מובנית לקביעה מי יכול לפענח אילו הודעות.

4. **ממשק משתמש מודרני ואינטואיטיבי**: ממשק גרפי נוח וידידותי שלא דורש ידע טכני מעמיק.

5. **תמיכה בהודעות לריבוי נמענים**: היכולת לשלוח הודעה אחת למספר נמענים אך לוודא שרק הם יכולים לפענח אותה.

6. **התראות אימייל משולבות**: שילוב מערכת התראות אימייל מובנית עבור הודעות חדשות ואימות.

סקירת טכנולוגיה

פיתוח StegnoLogic דרש שימוש במספר טכנולוגיות וספריות מתקדמות:

1. **שפת פייתון (Python):** בחרתי בפייתון כשפת התכנות העיקרית בשל הפשטות והגמישות שלה, וכן בגלל הכמות של ספריות לעיבוד תמונה והצפנה.

2. **הצפנת RSA:** השתמשתי בספריית `rsa` של פייתון ליישום הצפנה אסימטרית, שמאפשרת תקשורת מאובטחת בין הלקוח לשרת ללא צורך בשיתוף מפתח סודי מראש.

3. **סטגנוגרפיה LSB (Least Significant Bit):** פיתחתי אלגוריתם המסתיר מידע על ידי שינוי הביטים הפחות משמעותיים של ערכי פיקסלים בתמונה, כך שהשינויים לא נראים לעין אנושית.

4. **SQLite**: בחרתי במסד הנתונים SQLite לאחסון המידע על המשתמשים, ההודעות והתמונות בשל הפשטות והיעילות שלו, וכן בגלל שהוא אינו דורש שרת נפרד.

5. **CustomTkinter**: השתמשתי בספרייה זו (הרחבה של Tkinter) ליצירת ממשק גרפי מודרני ואסתטי.

6. **PIL (Python Imaging Library):** ספרייה חיונית לעיבוד תמונות בפייתון, המאפשרת פתיחה, שינוי ושמירה של קבצי תמונה בפורמטים שונים.

7. **SMTP (SimpleMailTransfer Protocol):** יישמתי תקשורת SMTP לשליחת הודעות אימייל אוטומטיות עבור אימות דו-שלבי והתראות.

8. **Sockets**: השתמשתי בממשק Socket של פייתון ליצירת תקשורת רשת בין הלקוח לשרת.

9. **Thread:** ניצלתי את מודול ה-threading לביצוע משימות במקביל, במיוחד לשליחת אימיילים למספר נמענים במקביל.

10. **Dotenv**: השתמשתי בספרייה זו לניהול בטוח של משתני סביבה רגישים כמו פרטי התחברות לאימייל.

11. **Hashlib ו-PBKDF2:** טכנולוגיות אלו משמשות לאחסון בטוח של סיסמאות עם שימוש ב"מלח" (salt) ופונקציות גיבוב חזקות.

מגבלות ואתגרים טכניים:

1. **גודל ההודעה מוגבל לגודל התמונה** : כל תמונה יכולה להכיל כמות מוגבלת של מידע, תלוי בגודל ובמימדי התמונה. הייתי צריך לפתח מנגנון לחישוב הקיבולת המקסימלית לכל תמונה.

2. **הגבלות הצפנת RSA על גודל ההודעה** : RSA מגביל את גודל הבלוק שניתן להצפין בבת אחת. פיתחתי פתרון לפיצול הודעות גדולות למקטעים קטנים יותר.

3. **ביצועים בעיבוד תמונות גדולות** : עיבוד של תמונות גדולות יכול להיות כבד על המעבד. היה צורך לאופטימיזציה של האלגוריתמים.

4. **שמירה על איכות התמונה** : היה חשוב שהתמונות המוצפנות לא יראו שונות באופן ניכר מהתמונות המקוריות, מה שדרש בדיקות מקיפות ושיפורים באלגוריתם.

5. **תאימות עם סוגי קבצים שונים** : היה צורך לוודא שהאלגוריתם יעבוד על פורמטים שונים של תמונות (PNG, JPEG, BMP).

פתרתי את המגבלות הללו על ידי תכנון מדוקדק של האלגוריתמים, בדיקות יסודיות, ושימוש בטכניקות מתקדמות לטיפול בקבצים גדולים ובפיצול הודעות.

היקף הפרויקט

תחומים שהפרויקט מכסה:

1. **אבטחת מידע:**
   - הצפנת RSA להודעות ותקשורת
   - הגנת סיסמאות עם PBKDF2 ו"מלח"
   - אימות דו-שלבי דרך אימייל
   - הגנה מפני התקפות DDOS באמצעות הגבלת קצב

2. **סטגנוגרפיה:**
   - אלגוריתם LSB להסתרת מידע בתמונות
   - בדיקות מקסימום אורך הודעה
   - שיטות לזיהוי נמענים ספציפיים בתוך ההודעה

3. **תקשורת רשת:**
   - ארכיטקטורת לקוח-שרת
   - תקשורת Socket מאובטחת
   - החלפת מפתחות בטוחה

4. **ניהול משתמשים:**
   - רישום והתחברות
   - אחסון נתוני משתמש במסד נתונים
   - ניהול הרשאות

5. **ממשק משתמש:**
   - ממשק גרפי מלא
   - אנימציות ואפקטים
   - עיצוב בסגנון Cybersecurity עם צבעים תואמים

6. **נהלי אבטחה:**
   - אכיפת מדיניות סיסמאות חזקות
   - יומני (logs) למעקב אחרי ניסיונות התחברות
   - אבטחת תקשורת בכל השכבות

7. **תקשורת אימייל:**
   - שליחת הודעות אימייל אוטומטיות
   - קודי אימות
   - התראות על הודעות חדשות

תחומים שאינם נכללים בפרויקט:

1. **סטגנוגרפיה בקבצי אודיו או וידאו** : המערכת מתמקדת בהסתרת מידע בתמונות בלבד, ולא בסוגי מדיה אחרים.

2. **ממשק ניהול מנהלים (Admin)** : המערכת לא כוללת כרגע ממשק למנהלי מערכת לניהול משתמשים ופעילויות.

3. **תקשורת קבוצתית** : אף שניתן לשלוח הודעה למספר נמענים, אין תמיכה בקבוצות או ערוצי תקשורת מובנים.

4. **הגנה מפני כל סוגי ההתקפות** : בעוד שהמערכת מטפלת בהגנות בסיסיות, היא לא מתיימרת להגן מפני כל סוגי ההתקפות האפשריים בעולם הסייבר.

תיאור המערכת (אפיון ועיצוב)

תיאור מערכת מפורט

מערכת סטגנולוגיק היא פלטפורמת תקשורת מאובטחת המבוססת על עקרונות הסטגנוגרפיה - טכניקה להסתרת מידע בתוך מדיה דיגיטלית, במקרה זה תמונות. המערכת מספקת פתרון שלם להסתרת מידע, החל מיצירת ההודעה המוסתרת ועד להעברתה ופענוחה אצל הנמענים המיועדים, הכל בסביבה מאובטחת ומוצפנת.

המערכת בנויה בארכיטקטורת שרת-לקוח:

1. **צד השרת** - מטפל בניהול משתמשים, אימות, אחסון נתונים, ותיאום בין המשתמשים השונים. השרת מיישם אמצעי אבטחה כמו הגבלת קצב חיבורים, הצפנת RSA, ותיעוד פעילות.

2. **צד הלקוח** - מספק ממשק גרפי ידידותי למשתמש, ומטפל בהצגת נתונים, הסתרת מידע בתמונות, ופענוח תמונות שהתקבלו.

תהליך עבודה עיקרי:

1. **הרשמה והתחברות** - משתמש חדש נרשם למערכת. המערכת מוודאת שהסיסמה חזקה ושומרת אותה בצורה מאובטחת. בהתחברות, המשתמש מקבל קוד אימות למייל שלו לצורך אימות דו-שלבי.

2. **הסתרת מידע** - המשתמש בוחר תמונה, מזין הודעה טקסטואלית ובוחר את הנמענים. המערכת מחשבת את גודל ההודעה המקסימלי שניתן להסתיר בתמונה הנבחרת, מסתירה את המידע בתמונה ושומרת אותה.

3. **שליחת הודעה** - התמונה המוצפנת נשלחת לשרת, שמאחסן אותה ומקשר אותה לנמענים הרלוונטיים. המשתמש יכול גם לבחור לשלוח את התמונה באימייל במקביל.

4. **פענוח הודעה** - הנמענים יכולים לפענח את התמונה שקיבלו כדי לראות את ההודעה המוסתרת. רק משתמשים שההודעה יועדה להם יכולים לפענח אותה.

טכניקת ההסתרה:

המערכת משתמשת בשיטת ה-LSB (Least Significant Bit) להסתרת המידע. בשיטה זו, כל ביט מההודעה מוסתר בביט הפחות משמעותי של ערך צבע בפיקסל. השינוי בתמונה כה מזערי שהוא בלתי ניתן להבחנה בעין אנושית. בנוסף, המידע עצמו מוצפן טרם הסתרתו, מה שמוסיף שכבת הגנה נוספת.

מנגנוני אבטחה:

1. **הצפנת RSA** - לתקשורת מאובטחת בין הלקוח לשרת
2. **אימות דו-שלבי** - באמצעות קוד חד-פעמי שנשלח למייל המשתמש
3. **סיסמאות מאובטחות** - בדיקת חוזק סיסמה והצפנת הסיסמאות במסד הנתונים
4. **מניעת DDOS** - מנגנון להגבלת כמות הבקשות לשרת
5. **זיהוי הנמען** - הטמעת מידע על הנמענים הספציפיים בתמונה המוצפנת

יכולות משתמש

* הרשמה למערכת
* התחברות עם אימות דו-שלבי
* עריכת פרטי משתמש
* בחירת תמונה להסתרת מידע
* הזנת הודעה טקסטואלית
* בחירת נמענים להודעה
* בדיקת גודל הודעה מקסימלי אפשרי
* שליחת תמונות מוצפנות
* פענוח הודעות שהתקבלו
* שליחת תמונות במייל לנמענים
* התנתקות מהמערכת

מבחנים מתוכננים (בדיקות קופסה שחורה)

מבחן 1: רישום משתמש חדש

* **בדיקה**: יכולת המערכת לרשום משתמשים חדשים ולאמת את פרטיהם.

* **אופן ביצוע**: ניסיון רישום עם נתונים תקינים וגם עם נתונים בעייתיים (סיסמה חלשה, שם משתמש קיים).

* **תוצאה מצופה**: המערכת צריכה לאפשר רישום עם נתונים תקינים, ולדחות נתונים לא תקינים עם הודעת שגיאה מתאימה.

מבחן 2: הסתרת מידע בתמונה

* **בדיקה**: יכולת המערכת להסתיר הודעה בתמונה בלי לשנות את מראה התמונה.

* **אופן ביצוע**: בחירת תמונה, הזנת הודעה, והסתרת המידע.

* **תוצאה מצופה**: התמונה המקודדת צריכה להיראות זהה לעין אנושית לתמונה המקורית, ולהכיל את המידע המוסתר.

מבחן 3: פענוח מידע מוסתר

* **בדיקה**: היכולת לחלץ הודעה מוסתרת מתמונה.

* **אופן ביצוע**: בחירת תמונה מקודדת וניסיון לפענח אותה.

* **תוצאה מצופה**: המערכת צריכה להציג בהצלחה את ההודעה המקורית שהוסתרה בתמונה.

מבחן 4: שליחת תמונה למספר נמענים

* **בדיקה**: יכולת המערכת לשלוח תמונה מקודדת למספר משתמשים.

* **אופן ביצוע**: יצירת הודעה מוסתרת ובחירת מספר נמענים.

* **תוצאה מצופה**: כל הנמענים שנבחרו צריכים לקבל את ההודעה ולהיות מסוגלים לפענח אותה.

מבחן 5: אבטחת גישה להודעות

* **בדיקה**: וידוא שרק הנמענים המיועדים יכולים לפענח את ההודעה.

* **אופן ביצוע**: ניסיון לפענח הודעה עם משתמש שלא נכלל ברשימת הנמענים.

* **תוצאה מצופה**: המערכת צריכה לסרב לפענח את ההודעה למשתמש שאינו נמען מורשה.

מבחן 6: חוסן המערכת לעומס

* **בדיקה**: יכולת המערכת לתפקד תחת עומסים.

* **אופן ביצוע**: יצירת מספר רב של בקשות חיבור במקביל.

* **תוצאה מצופה**: השרת צריך להגביל את קצב הבקשות ולהמשיך לתפקד ללא קריסה.

ניהול זמן ופיתוח

ציר זמן של אבני דרך עיקריות:

1. **תכנון ראשוני (שבועות 1-2)**
   * גיבוש רעיון הפרויקט
   * חקר טכנולוגיות סטגנוגרפיה והצפנה
   * הגדרת דרישות ויכולות המערכת

2. פיתוח תשתיות (שבועות 3-5)
   * יצירת מבנה בסיסי לשרת ולקוח
   * מימוש מנגנוני הקריפטוגרפיה הבסיסיים
   * יצירת מסד הנתונים

3. פיתוח אלגוריתמי סטגנוגרפיה (שבועות 6-8)
   * מימוש אלגוריתם LSB להסתרת מידע בתמונות
   * פיתוח מנגנון לחישוב קיבולת המידע של תמונה
   * בדיקות ביצועים ואיכות תמונה

4. פיתוח ממשק משתמש וגרסה סופית (שבועות 9-12)
   * עיצוב ובניית ממשק גרפי
   * יצירת אנימציות ואלמנטים ויזואליים
   * שילוב מסכי התחברות והרשמה
   * חיבור כל רכיבי המערכת
   * בדיקות מקיפות של כל התהליכים
   * תיקון באגים והטמעת שיפורים
   * שיפור ביצועים
   * כתיבת תיעוד
   * הכנת גרסה סופית

תכנית מקורית מול לוח זמנים בפועל:

לפי התכנית המקורית, הפרויקט היה אמור להסתיים תוך 12 שבועות. בפועל, נתקלתי במספר אתגרים שגרמו לעיכובים:

1. קשיים בפיתוח ממשק המשתמש (עיכוב של שבועיים):

בחרתי להשתמש בספריית CustomTkinter שלא הכרתי לפני כן, בגלל היכולות העיצוביות המתקדמות שלה. ההחלפה של Tkinter בCustomTkinter לקחה לי זמן והייתי צריך ללמוד את הספרייה החדשה.

2. שיפור ביצועים של הצפנת RSA (התווסף במהלך הפרויקט):

במהלך הפיתוח זיהיתי שהביצועים של מנגנון ההצפנה כשמדובר בהודעות ארוכות היו איטיים. הוספתי מנגנון לחלוקת הודעות ארוכות לחלקים והצפנתם בנפרד, מה שדרש זמן נוסף לא מתוכנן.

בסופו של דבר, הפרויקט הסתיים כשלושה שבועות מאוחר יותר מהתכנון המקורי, אך התוצאה הסופית עלתה על הציפיות מבחינת איכות ופונקציונליות.

ניהול סיכונים

סיכון 1: אובדן אבטחת מידע

* **תיאור הסיכון**: חשיפה של המידע המוסתר או מפתחות ההצפנה לגורמים לא מורשים.

* **פתרונות מתוכננים**: שימוש בהצפנת RSA לכל התקשורת, אימות דו-שלבי, הצפנת סיסמאות במסד הנתונים.

* **בפועל**: המערכת אכן נשארה מאובטחת.

סיכון 2: ביצועים ירודים בעומס

* **תיאור הסיכון**: קריסה או האטה משמעותית של השרת בעומס גבוה, או בניסיון התקפת DDOS.

* **פתרונות מתוכננים**: מנגנון להגבלת קצב בקשות, שימוש ב-multithreading לטיפול במספר לקוחות במקביל.

* **בפועל**: בבדיקות עומס המערכת הצליחה לעמוד במספר רב של חיבורים, אך בעומסים גבוהים במיוחד זיהיתי האטה בטיפול בקבצי תמונות גדולים. פתרתי זאת על ידי הוספת מנגנון לבקרת גודל תמונה והגבלת גודל ההודעות.

סיכון 3: מגבלת גודל מידע בסטגנוגרפיה

* **תיאור הסיכון**: חוסר יכולת להסתיר מידע גדול בתמונות קטנות, או פגיעה באיכות התמונה.

* **פתרונות מתוכננים**: פיתוח מנגנון לחישוב הגודל המקסימלי של המידע שניתן להסתיר, ומתן משוב למשתמש.

* **בפועל**: המנגנון עבד היטב, אבל עדיין היו משתמשים שניסו להסתיר מידע גדול מדי. הוספתי חיווי ויזואלי לעזור למשתמשים להבין את המגבלות, כולל מד גודל הודעה שמשתנה צבעו כשמתקרבים למגבלה.

סיכון 4: תלות בשירותי אימייל

* **תיאור הסיכון**: בעיות בשליחת קודי אימות או תמונות במייל בגלל מגבלות ספקי שירות.

* **פתרונות מתוכננים**: שימוש ב-SMTP עם תמיכה ב-SSL/TLS, ואפשרות להגדרת פרטי שרת מייל מותאם.

* **בפועל**: נתקלתי בבעיות עם חשבונות Gmail שהגבילו שליחת מיילים מ"אפליקציות פחות מאובטחות". פתרתי את הבעיה באמצעות מתן הוראות למשתמשים כיצד ליצור "סיסמה לאפליקציה" בהגדרות Google, והוספתי את האפשרות לשימוש בשרתי מייל אחרים.

תיאור תחום הידע - סקירה נרטיבית

מערכת סטגנולוגיק - יכולות מרכזיות

פרויקט הסטגנולוגיק שבניתי מהווה מערכת שרת-לקוח מבוזרת, המאפשרת הסתרת מידע בתמונות באמצעות טכניקות סטגנוגרפיה מתקדמות. המערכת מאפשרת למשתמשים להחליף מידע רגיש באופן מאובטח וחשאי, כאשר המידע עצמו מוסתר בתוך תמונות שנראות תמימות לחלוטין. בניגוד לשיטות הצפנה רגילות, שבהן ניתן לראות שקיימת תקשורת מוצפנת (גם אם לא ניתן לפענח אותה), הסטגנוגרפיה מסתירה את עצם קיום התקשורת הסודית. זוהי יתרון משמעותי במצבים שבהם גם עצם קיום התקשורת המוצפנת עלול להיות חשוד.

המערכת מחולקת לשני חלקים עיקריים: צד השרת, שמנהל את המשתמשים, מאחסן נתונים ומאפשר תקשורת בין המשתמשים השונים, וצד הלקוח, שמספק ממשק משתמש גרפי ומאפשר בחירת תמונות, הסתרת מידע ופענוחו. להלן תיאור מפורט ומורחב של היכולות המרכזיות שמימשתי בפרויקט, מחולקות לפי צד השרת וצד הלקוח.

יכולות צד השרת

1. ניהול משתמשים ואימות זהות

מטרה:

לנהל באופן מקיף את מסד הנתונים של משתמשי המערכת, לאפשר רישום משתמשים חדשים, ולאמת את זהות המשתמשים בעת ההתחברות. ניהול משתמשים הוא חלק קריטי במערכת שלי, שכן אני צריך לוודא שרק משתמשים מורשים יכולים לגשת למידע המוסתר שנשלח אליהם, ולכן נדרשת מערכת אימות חזקה במיוחד.

פעולות ורכיבים:

הרשמת משתמשים חדשים - כשמשתמש חדש מעוניין להצטרף למערכת, עליו לספק שם משתמש ייחודי, שם מלא, כתובת אימייל תקפה וסיסמה. המערכת מבצעת סדרה של בדיקות על המידע הזה: היא בודקת שהשם משתמש אינו קיים כבר במסד הנתונים (למניעת התנגשויות), שכתובת המייל מתאימה לתבנית תקפה (באמצעות ביטויים רגולריים), ושהסיסמה חזקה מספיק לפי קריטריונים מוגדרים (אורך, מורכבות וכו'). רק לאחר שכל הבדיקות האלה עוברות בהצלחה, המשתמש החדש נרשם במסד הנתונים.

אימות פרטי התחברות - בעת ניסיון התחברות, המשתמש מספק שם משתמש וסיסמה. המערכת מחפשת את שם המשתמש במסד הנתונים, ואם הוא קיים, היא משווה את הסיסמה שהוזנה עם הגיבוב השמור של סיסמת המשתמש במסד הנתונים. אימות הסיסמה מבוצע באמצעות פונקציית verify_password שלא משווה את הסיסמאות ישירות, אלא מחשבת גיבוב של הסיסמה המוזנת (עם ה"מלח" השמור) ומשווה את התוצאה לגיבוב השמור במסד הנתונים. זו גישה הרבה יותר בטוחה מהשוואת סיסמאות גלויות.

אימות דו-שלבי - כדי לחזק את האבטחה, הוספתי שכבת אבטחה נוספת בצורה של אימות דו-שלבי. לאחר שהמשתמש מזין שם משתמש וסיסמה נכונים, המערכת מייצרת קוד אימות אקראי בן שש ספרות (בין 100,000 ל-999,999) ושולחת אותו לכתובת המייל הרשומה של המשתמש. המשתמש נדרש להזין את הקוד הזה כדי להשלים את תהליך ההתחברות. הקוד תקף לחמש דקות בלבד, כאשר שעון עצר מציג למשתמש כמה זמן נותר. אם הקוד פג תוקף, המשתמש יצטרך להתחיל את תהליך ההתחברות מחדש. זה מונע התקפות מסוג "brute force" על הקוד, שכן מספר הניסיונות האפשרי מוגבל בזמן.

בדיקת חוזק סיסמה - הטמעתי אלגוריתם מתקדם לבדיקת חוזק סיסמאות, שמוודא שכל סיסמה חדשה או מעודכנת עומדת בדרישות אבטחה מחמירות. הסיסמה חייבת לכלול לפחות 8 תווים, אות גדולה אחת לפחות, אות קטנה אחת לפחות, ספרה אחת לפחות, ותו מיוחד אחד לפחות (כגון !@#$). הבדיקות האלה מיושמות באמצעות ביטויים רגולריים (regex) שבודקים את נוכחות כל אחד מהרכיבים הנדרשים.
בממשק המשתמש, סרגל דינמי מראה למשתמש את רמת החוזק של הסיסמה שלו בזמן אמת, עם שינוי צבע שמשקף את רמת האבטחה (אדום לחלש, כתום לבינוני, ירוק לחזק), ומונע ממנו לבחור סיסמאות חלשות מדי.

הצפנת סיסמאות - נושא קריטי באבטחת מערכות הוא שמירת סיסמאות בצורה מאובטחת. בפרויקט שלי, סיסמאות המשתמשים אף פעם לא נשמרות כטקסט גלוי במסד הנתונים, אלא רק בצורה מוצפנת באמצעות טכניקת "מלח וגיבוב" (salt and hash). כשמשתמש נרשם, המערכת מייצרת מחרוזת אקראית ייחודית באורך של 32 בייט (המכונה "מלח"), משלבת אותה עם הסיסמה, ומחשבת גיבוב PBKDF2 עם אלגוריתם SHA-256, עם 100,000 איטרציות. ה"מלח" הייחודי מבטיח שגם אם שני משתמשים בוחרים את אותה סיסמה, הגיבוב השמור במסד הנתונים יהיה שונה לחלוטין. 100,000 האיטרציות גורמות לפעולת הגיבוב להיות איטית במכוון (אך עדיין מהירה מספיק למשתמש לגיטימי), מה שמקשה מאוד על התקפות "brute force" או "dictionary attack" במקרה של פריצה למסד הנתונים.

רכיבים נדרשים:

מודול DatabaseHandler - מודול מקיף שאחראי על כל האינטראקציות עם מסד הנתונים. הוא כולל פונקציות לשמירה ושליפה של נתוני משתמשים, כמו גם פונקציות לניהול הודעות, תמונות ולוגים. המודול מטפל בכל פעולות ה-SQL, כולל יצירת טבלאות (אם אינן קיימות), הוספת רשומות, עדכון נתונים קיימים, מחיקה וחיפוש. השתמשתי בשאילתות מוכנות (prepared statements) SQL כדי למנוע התקפות SQL injection, והקפדתי על טיפול נכון בשגיאות ומצבי קצה. המודול עובד עם SQLite, שבחרתי בגלל הפשטות והניידות שלו.

מודול SecurityCrypto - מודול אבטחה רב-תכליתי שאחראי על כל הפעולות הקריפטוגרפיות במערכת. הוא מטפל בהצפנה ופענוח של מידע, יצירת וניהול זוגות מפתחות RSA, בדיקת חוזק סיסמאות, הצפנת סיסמאות באמצעות מלח וגיבוב, ואימות סיסמאות. המודול הזה מהווה את ליבת האבטחה של המערכת, ולכן הקדשתי תשומת לב מיוחדת לבדיקות ולטיפול בשגיאות. למעשה, שילבתי את המודול הזה גם עם מודול האבטחה הכללי, SecurityProvider, כדי לפשט את הארכיטקטורה ולמנוע שכפול קוד.

מודול EmailHandler - מודול ייעודי לשליחת אימיילים, שמשמש בעיקר לשליחת קודי אימות דו-שלבי, אך גם לשליחת הודעות והתראות למשתמשים. המודול מתממשק עם שרתי SMTP (כמו Gmail), תומך ב-SSL/TLS לאבטחת התקשורת, ויכול לשלוח הן הודעות טקסט פשוטות והן הודעות עם קבצים מצורפים (כמו התמונות המקודדות). המודול נבנה בצורה גמישה שמאפשרת תצורה קלה באמצעות קובץ env., כך שמנהל המערכת יכול בקלות להחליף את החשבון שממנו נשלחים המיילים.

מסד נתונים SQLite עם טבלת משתמשים - מסד הנתונים של המערכת נבנה בטכנולוגיית SQLite, שמאפשרת אחסון נתונים בקובץ יחיד, ללא צורך בשרת מסד נתונים נפרד. טבלת המשתמשים כוללת עמודות לשם משתמש (מפתח ראשי), שם מלא, סיסמה מוצפנת וכתובת אימייל. בנוסף, יש טבלאות נוספות להודעות, תמונות, ולוגים, שמקושרות לטבלת המשתמשים באמצעות מפתחות זרים (foreign keys). המבנה הזה מבטיח שלמות נתונים ומאפשר שאילתות יעילות.

הפונקציה check_password_strength שפיתחתי מהווה דוגמה טובה לאופן שבו טיפלתי באבטחה במערכת. בניתי אותה כך שתבצע סדרה של בדיקות על כל סיסמה מוצעת, ותחזיר הן תוצאה בוליאנית (האם הסיסמה עומדת בכל הדרישות) והן הודעת שגיאה ספציפית במקרה שאחת הבדיקות נכשלה. הדבר מאפשר למערכת להציג למשתמש משוב ברור על מה בדיוק הוא צריך לשפר בסיסמה שלו.

לדוגמה, אם המשתמש מנסה להירשם עם הסיסמה "password123", הפונקציה תחזיר False יחד עם ההודעה "Password must contain at least one uppercase letter", מכיוון שלמרות שהסיסמה מכילה אותיות קטנות וספרות, היא חסרה אות גדולה. ממשק המשתמש יציג הודעה זו ויסמן את שדה הסיסמה באדום, מה שמנחה את המשתמש לתקן את הבעיה.

מנגנון אחסון הסיסמאות שפיתחתי הוא מתקדם ומאובטח במיוחד. כשמשתמש נרשם, המערכת מייצרת "מלח" אקראי באורך 32 בייט, שהוא מספיק גדול כדי למנוע התקפות "rainbow tables". לאחר מכן, המערכת משתמשת בפונקציית PBKDF2 (Password-Based Key Derivation Function) עם אלגוריתם SHA-256, עם 100,000 איטרציות. מספר גבוה זה של איטרציות גורם לפעולה להיות איטית במכוון, מה שמקשה מאוד על התקפות brute force.

2. ניהול לוגים ואבטחת שרת

מטרה:

לתעד באופן מקיף כל פעילות במערכת, למנוע התקפות סייבר שונות, ולאפשר ניתוח מעמיק של אירועי אבטחה. ניהול לוגים הוא חלק קריטי באבטחת המערכת, מכיוון שהוא מאפשר לאתר דפוסי פעילות חשודים, לחקור אירועי אבטחה, ולהבין כיצד המערכת משמשת בפועל. בנוסף, שילבתי מנגנונים אקטיביים למניעת התקפות שכיחות, כמו DDOS (מניעת שירות מבוזרת).

פעולות ורכיבים:

תיעוד התחברויות למערכת - כל ניסיון התחברות למערכת, בין אם הצליח או נכשל, נרשם במסד הנתונים עם פרטים מלאים. הלוג כולל את שם המשתמש שנעשה בו שימוש, תאריך ושעה מדויקים של הניסיון, הסטטוס (הצלחה או כישלון), וכתובת ה-IP של המקור. בניגוד למערכות בסיסיות שמתעדות רק כישלונות, בחרתי לתעד גם התחברויות מוצלחות, כי גם הן יכולות להצביע על פעילות חשודה במקרים מסוימים (למשל, התחברות בשעה חריגה או ממיקום גיאוגרפי לא צפוי). הדבר מספק למנהל המערכת תמונה מלאה של דפוסי ההתחברות, ומאפשר זיהוי של פריצות אפשריות.

תיעוד פעולות משתמשים - מעבר לתיעוד התחברויות, המערכת מתעדת גם פעולות משמעותיות שמשתמשים מבצעים בתוכה. זה כולל שליחת הודעות מוסתרות, פענוח תמונות, שינוי פרטי משתמש, וכדומה. כל פעולה נרשמת עם פרטים רלוונטיים, כולל המשתמש שביצע אותה, תאריך ושעה, ואם רלוונטי - הנמענים או המשאבים שעליהם הפעולה בוצעה. לדוגמה, כששולחים הודעה מוסתרת, הלוג יכלול את שם השולח, הנמענים, וזמן השליחה, אך לא את תוכן ההודעה עצמה (שנשאר מאובטח). תיעוד זה חיוני לשחזור פעולות במקרה של תקלה, ולזיהוי פעילות חשודה.

הגנה מפני DDOS (Distributed Denial of Service) - התקפות DDOS מנסות להפיל שרת על ידי הצפתו בבקשות רבות בזמן קצר. כדי להתגונן, פיתחתי מנגנון "הגבלת קצב" (rate limiting) שמנטר את כמות הבקשות מכל כתובת IP ברמה של כל דקה. המנגנון שומר היסטוריה של זמני הבקשות, ומסנן בקשות חדשות אם כמותן חורגת מהסף המוגדר (בדרך כלל 180 בקשות לדקה). חשוב לציין שהסף הזה מאזן בין אבטחה לבין שימושיות; הוא גבוה מספיק כדי לא להפריע למשתמשים לגיטימיים, אך נמוך מספיק כדי למנוע התקפות משמעותיות. כשבקשה נדחית בגלל חריגה מהסף, הדבר מתועד בלוג האבטחה, ומשוב מתאים נשלח ללקוח.

ניתור חיבורים פעילים והתנתקויות - השרת שומר מעקב אחר כל החיבורים הפעילים אליו בכל רגע נתון. הדבר כולל שמירת אובייקט ה-socket של כל חיבור, מיפוי בין חיבורים למשתמשים, וניהול מצב החיבור. כשמשתמש מתנתק באופן מסודר (למשל על ידי לחיצה על כפתור "Log out"), החיבור נסגר בצורה מסודרת. אך החשיבות האמיתית של המנגנון הזה היא בזיהוי התנתקויות לא מסודרות - למשל, כשחיבור נקטע באופן פתאומי בגלל בעיית רשת. במקרים כאלה, השרת מזהה את הניתוק, מנקה את המשאבים המשויכים לחיבור, ומתעד את האירוע. בלי הניקוי הזה, היו עלולות להיווצר "דליפות משאבים" שהיו פוגעות בביצועי השרת לאורך זמן.

רכיבים נדרשים:

מודול SecurityCrypto - לניטור וניהול חיבורים, זהו מודול רב-תכליתי שכולל, בין השאר, את אלגוריתם הגבלת הקצב למניעת DDOS. המודול מספק פונקציות כמו limit_connection_rate, שבודקת אם יש לאפשר בקשה חדשה לפי ההיסטוריה של בקשות קודמות. פונקציה זו מיושמת באופן חכם - היא לא פשוט סופרת את מספר הבקשות, אלא גם "שוכחת" בקשות ישנות (מלפני יותר מדקה), כך שהחישוב מדויק ולא צובר מידע מיותר. בנוסף, המודול אחראי על אבטחת התקשורת ומתן גישה למשאבים רק למשתמשים מורשים.

מודול DatabaseHandler - לתיעוד הלוגים במסד הנתונים, מודול זה כולל פונקציות ייעודיות לשמירת רשומות לוג, כמו log_login_attempt שמתעדת ניסיונות התחברות, ופונקציות נוספות לתיעוד פעולות אחרות במערכת. המודול דואג לפורמט אחיד של רשומות הלוג, כולל תיעוד אוטומטי של מועד הפעולה (בפורמט ISO), ונתונים רלוונטיים נוספים. כל רשומת לוג כוללת מזהה ייחודי מספרי (ID) שמאפשר מיון וחיפוש יעילים.

טבלת logs במסד הנתונים - טבלה ייעודית לשמירת רשומות לוג, עם מבנה מותאם לסוגי הלוגים השונים. הטבלה כוללת עמודות עבור מזהה הלוג, שם המשתמש הרלוונטי, חותמת זמן, סטטוס הפעולה, וכתובת ה-IP של הלקוח. טבלת הלוגים מקושרת לטבלת המשתמשים באמצעות מפתח זר, כך שניתן בקלות לחפש את כל הפעולות של משתמש מסוים. בנוסף, תכננתי את הטבלה כך שתהיה יעילה הן בפעולות כתיבה (שמתבצעות בתדירות גבוהה) והן בפעולות חיפוש.

הלוגים נשמרים במסד הנתונים עם מבנה קבוע שמאפשר ניתוח קל. המבנה הזה מאפשר למנהל המערכת לבצע שאילתות מורכבות, כמו "כמה ניסיונות התחברות כושלים היו בשעה האחרונה?" או "האם יש דפוס של ניסיונות חוזרים ונשנים מכתובת IP ספציפית?", מה שמסייע בזיהוי התקפות ובמניעתן.

3. סטגנוגרפיה - הטמעת וחילוץ מידע

מטרה:

לאפשר הסתרת מידע בתמונות והוצאתו בצורה שלא ניתנת לזיהוי בעין רגילה. זהו הליבה הטכנולוגית של הפרויקט - היכולת להסתיר מידע בתמונות כך שאדם שמסתכל על התמונה לא יוכל לדעת שהיא מכילה מידע מוסתר. סטגנוגרפיה היא אמנות עתיקה של הסתרת מידע, אך במערכת שלי היא מיושמת באופן דיגיטלי מתקדם שמאפשר שיתוף מידע סודי באופן חשאי לחלוטין.

פעולות ורכיבים:

חישוב קיבולת המידע - לפני שמסתירים מידע בתמונה, חשוב לדעת כמה מידע ניתן להסתיר בה מבלי לפגוע באיכותה. פיתחתי אלגוריתם מתוחכם שמחשב את הקיבולת המקסימלית של תמונה בהתבסס על מימדיה (רוחב וגובה) ומספר הערוצים שלה (RGB). הנוסחה הבסיסית היא: מספר הפיקסלים כפול מספר הערוצים (בדרך כלל 3), מחולק ב-8 כדי להמיר מביטים לבתים, והכל מוכפל ב-0.9 כפקטור בטיחות. למשל, תמונה בגודל 1000X1000 פיקסלים יכולה להכיל תיאורטית עד כ-337.5KB של מידע. אלגוריתם זה מחזיר למשתמש את הגודל המקסימלי בתווים, כך שהוא יודע מראש אם ההודעה שלו תתאים לתמונה הנבחרת.

הטמעת מידע בתמונות - אחרי שנבחרה תמונה והמשתמש הזין הודעה, המערכת מסתירה את המידע בתמונה באמצעות טכניקת LSB (Least Significant Bit). בשיטה זו, כל ביט מההודעה מוסתר בביט הפחות משמעותי של ערך צבע בפיקסל. לדוגמה, אם יש לנו פיקסל עם ערכי RGB של (128, 65, 201), והביטים שאנחנו רוצים להסתיר הם "101", נשנה את הביט האחרון בכל ערך צבע כך שנקבל משהו כמו (129, 65, 200). השינוי כה מזערי (שינוי של 1 מתוך 256 אפשרויות בכל ערך) שהעין האנושית לא יכולה להבחין בהבדל. המערכת מסתירה תחילה את המטא-דאטה (כמו רשימת הנמענים המורשים), ואז את ההודעה עצמה, ולבסוף מוסיפה סימן סיום מיוחד ("1111111111111110") שמאפשר לזהות את סוף ההודעה בעת הפענוח. לאחר הסתרת המידע, התמונה נשמרת בפורמט PNG, שאינו מאבד מידע בשל דחיסה, כדי לשמור על המידע המוסתר. למרות שתמונה כזו נראית זהה לחלוטין לתמונה המקורית, היא מכילה את ההודעה הסודית שרק מי שיודע איך להסתכל יכול למצוא.

זיהוי נמענים - הייחודיות של המערכת שלי היא שהיא מאפשרת לשלוח הודעה למספר משתמשים, כאשר כל אחד מהם יכול לפענח את ההודעה, אך רק אם הוא ברשימת הנמענים המורשים. כדי לממש זאת, המערכת מוסיפה לפני ההודעה המוסתרת רשימה של שמות המשתמשים המורשים, מופרדים בתו מיוחד ("|"). לדוגמה, אם ההודעה מיועדת למשתמשים "alice" ו-"bob", המערכת מסתירה "alice|bob:ההודעה האמיתית". בעת הפענוח, המערכת בודקת אם המשתמש הנוכחי מופיע ברשימה לפני נקודותיים, ורק אז מציגה את ההודעה. הדבר מבטיח שגם אם משתמש לא מורשה ינסה לפענח את התמונה, הוא לא יוכל לגשת למידע.

חילוץ מידע מתמונות - תהליך החילוץ הוא למעשה היפוך של תהליך ההטמעה. המערכת קוראת את הביט האחרון מכל ערך צבע בכל פיקסל, מחברת אותם לביטים שלמים, וממירה לתווים. אחרי שהיא מזהה את רשימת הנמענים (עד לתו נקודותיים), היא בודקת אם המשתמש הנוכחי ברשימה. אם כן, היא ממשיכה בפענוח עד שהיא מגיעה לסימן הסיום, ואז מציגה את ההודעה למשתמש. אם לא, היא מציגה הודעת שגיאה מתאימה. תהליך זה מבטיח שרק הנמענים המיועדים יכולים לגשת למידע, אפילו אם התמונה המקודדת מגיעה לידיים לא נכונות.

רכיבים נדרשים:

מודול SteganographyHandler - זהו המודול המרכזי שמטפל בכל פעולות הסטגנוגרפיה. הוא מכיל פונקציות לחישוב קיבולת תמונה, הטמעת מידע, וחילוץ מידע מוסתר. המודול מיישם את אלגוריתם ה-LSB בצורה יעילה, כולל טיפול במקרי קצה כמו תמונות קטנות מדי או הודעות גדולות מדי. הוא גם מטפל בפורמטים שונים של תמונות (PNG, JPEG, וכו'), מחליט מתי נדרשת המרה לפני פעולת ההסתרה, ומבטיח שהתמונה המקודדת נשמרת בפורמט שלא מאבד מידע (PNG). בנוסף, הוא כולל מנגנוני הגנה מפני טעויות, למשל בדיקה שהתמונה גדולה מספיק להכיל את ההודעה, וטיפול בשגיאות בזמן הקידוד או הפענוח.

ספריית PIL (Python Imaging Library) - ספרייה חיצונית שמאפשרת טיפול בתמונות בפייתון. השתמשתי בה לטעינת תמונות, שינוי גודל במידת הצורך, גישה לנתוני הפיקסלים, ושמירת התמונות המקודדות. הספרייה מאפשרת גישה נוחה לכל פיקסל ולערכי ה-RGB שלו, מה שחיוני לאלגוריתם ה-LSB. בנוסף, היא תומכת במגוון רחב של פורמטים, מה שהופך את המערכת ליותר גמישה. אחד היתרונות העיקריים של PIL הוא היכולת שלה לעבוד עם תמונות גדולות באופן יעיל, ללא צריכת זיכרון מופרזת.

האלגוריתם שפיתחתי לחישוב קיבולת תמונה זה שיפור משמעותי על גישות קודמות. הוא לא רק מחשב את המספר הגולמי של ביטים שאפשר להסתיר, אלא גם לוקח בחשבון את המטא-דאטה (כמו רשימת הנמענים) ואת סמן הסיום. הנוסחה כוללת גם פקטור בטיחות של 90%, שמשאיר מרווח למקרה שחלקים מהתמונה אינם מתאימים להסתרת מידע (למשל אזורים אחידים לחלוטין).

4. ניהול תקשורת מאובטחת

מטרה:

להבטיח תקשורת מוצפנת ומאובטחת בין השרת ללקוחות, כך שגורמים עוינים לא יוכלו ליירט, לשנות או לזייף מידע העובר ביניהם. אבטחת התקשורת היא שכבה קריטית בהגנה על המערכת, במיוחד כאשר מדובר במידע רגיש כמו פרטי משתמשים והודעות פרטיות. חשוב להבטיח שגם אם התעבורה בין הלקוח לשרת נלכדת על ידי גורם זר, הוא לא יוכל להבין את תוכנה או לשנות אותה.

פעולות ורכיבים:

החלפת מפתחות RSA - כאשר לקוח מתחבר לשרת, הדבר הראשון שקורה הוא תהליך החלפת מפתחות. השרת מייצר זוג מפתחות RSA (ציבורי ופרטי) ייחודי לכל חיבור, ושולח את המפתח הציבורי ללקוח. במקביל, הלקוח מייצר זוג מפתחות משלו, ושולח את המפתח הציבורי לשרת. כעת, כל צד יכול להצפין מידע עם המפתח הציבורי של הצד השני, והצד השני יכול לפענח אותו עם המפתח הפרטי שלו. זהו תהליך החלפת מפתחות סטנדרטי בהצפנה אסימטרית, שמאפשר הקמת ערוץ תקשורת מאובטח מבלי לשתף סודות מראש.

הצפנת מסרים - לאחר שהוקם ערוץ מאובטח, כל התקשורת מוצפנת. לפני שליחת מידע, הצד השולח מצפין אותו עם המפתח הציבורי של הצד המקבל. רק הצד המקבל, שמחזיק במפתח הפרטי המתאים, יכול לפענח את המידע. הדבר מבטיח סודיות מלאה - גם אם מישהו מצותת לתקשורת, הוא רואה רק נתונים מוצפנים שאין לו דרך לפענח. בנוסף, ההצפנה גם מאפשרת אימות מקור - משום שרק בעל המפתח הפרטי יכול היה ליצור הודעה שניתן לפענח עם המפתח הציבורי המתאים.

טיפול בהודעות גדולות - אלגוריתם RSA יש מגבלה טכנית על גודל המידע שניתן להצפין בבת אחת - בדרך כלל לא יותר מ-245 בייט בקידוד בינארי ישיר (תלוי בגודל המפתח). כדי להתגבר על כך, פיתחתי מנגנון שמחלק הודעות גדולות למקטעים, מצפין כל מקטע בנפרד, ומחבר אותם שוב בצד המקבל. המנגנון משתמש בתווים מיוחדים ("CHUNK") להפריד בין המקטעים המוצפנים, כך שהצד המקבל יודע היכן כל מקטע מתחיל ומסתיים. בדרך זו, המערכת יכולה להצפין הודעות בכל גודל, מבלי להיתקל במגבלות האלגוריתם.

ניהול חיבורים - השרת מנהל מעקב אחר כל החיבורים הפעילים, כולל המפתחות הציבוריים של כל לקוח, כתובות ה-IP שלהם, ומצב ההצפנה. כשחיבור נסגר (אם באופן יזום או עקב בעיית רשת), השרת מנקה את המשאבים המשויכים לחיבור זה, כולל פינוי זיכרון והסרה ממבני הנתונים. ניהול החיבורים כולל גם זיהוי של התנתקויות לא מסודרות, וטיפול בהן באופן שלא פוגע ביציבות השרת. המערכת גם יודעת לטפל במצבים של "סשן ישן", כאשר משתמש מתחבר מחדש אחרי שחיבור קודם שלו נקטע בצורה לא צפויה.

רכיבים נדרשים:

מודול SecurityCrypto - המודול המרכזי למימוש הצפנה והחלפת מפתחות. הוא כולל פונקציות ליצירת זוגות מפתחות RSA, הצפנה ופענוח של מידע, וטיפול בהודעות גדולות. המודול מטפל בכל ההיבטים הקריפטוגרפיים של המערכת, ומספק ממשק פשוט ואחיד לשאר חלקי הקוד. הוא מתוכנן להיות מודולרי ונפרד מהלוגיקה העסקית, כך שניתן יהיה בעתיד להחליף את אלגוריתמי ההצפנה בלי לשנות את שאר הקוד.

ספריית rsa - ספריית פייתון חיצונית שמיישמת את אלגוריתם ההצפנה האסימטרי RSA. השתמשתי בה ליצירת זוגות מפתחות, הצפנה ופענוח של מידע, וטעינה/שמירה של מפתחות בפורמט PEM. בחרתי בספרייה זו בגלל היותה מאובטחת, מתוחזקת היטב, ופשוטה לשימוש. אלגוריתם RSA נבחר בגלל שהוא מוכח, מאובטח, ונתמך היטב.

ספריית socket - ספריית פייתון סטנדרטית לתקשורת רשת בסיסית. השתמשתי בה ליצירת חיבורי TCP בין הלקוח לשרת, שליחה וקבלה של נתונים, וטיפול בשגיאות רשת. הספרייה מספקת את התשתית הבסיסית לתקשורת, שמעליה בניתי את שכבת האבטחה.

מנגנון threading - ספריית פייתון סטנדרטית למימוש תכנות מקבילי. השתמשתי בה כדי לאפשר לשרת לטפל במספר לקוחות במקביל. כל חיבור לקוח חדש מטופל בתהליכון (thread) נפרד, כך שהשרת יכול להמשיך לקבל חיבורים חדשים ולתת שירות ללקוחות קיימים בו-זמנית. הדבר חיוני למערכת שנועדה לשרת מספר משתמשים במקביל.

תהליך החלפת המפתחות והקמת ערוץ מוצפן בנוי בצורה עמידה וחסינה לבעיות. כשלקוח חדש מתחבר, השרת שולח לו את המפתח הציבורי שלו בפורמט PEM (Privacy Enhanced Mail), שהוא פורמט סטנדרטי לאחסון והעברה של מפתחות קריפטוגרפיים. הלקוח שולח את המפתח הציבורי שלו חזרה לשרת באותו פורמט. כל צד שומר את המפתח הציבורי של הצד השני, ומשתמש בו להצפנת הודעות יוצאות.

5. שליחת מיילים והודעות

מטרה:

לאפשר אימות משתמשים ולשלוח הודעות והתראות למשתמשים באמצעות אימייל. מערכת האימייל נועדה להרחיב את יכולות התקשורת של המערכת מעבר לתקשורת ישירה בין הלקוח לשרת, ולספק ערוץ תקשורת נוסף למשימות כמו אימות דו-שלבי, שליחת התראות על הודעות חדשות, והעברת תמונות מקודדות לנמענים. אימייל הוא אמצעי קריטי במערכת, שמשלים את מנגנון האבטחה ומשפר את חוויית המשתמש.

פעולות ורכיבים:

שליחת קודי אימות - המערכת יוצרת ושולחת קודי אימות חד-פעמיים למשתמשים בעת התחברות, כחלק ממנגנון האימות הדו-שלבי. כשמשתמש מזין שם משתמש וסיסמה נכונים, המערכת מייצרת קוד אקראי בן 6 ספרות (למשל, "273591") ושולחת אותו לכתובת המייל הרשומה של המשתמש. הקוד תקף לזמן מוגבל בלבד (5 דקות), ומוצג למשתמש שעון עצר שמראה כמה זמן נותר. תהליך זה מוסיף שכבת אבטחה משמעותית, שכן גם אם מישהו השיג את פרטי ההתחברות של משתמש, הוא עדיין זקוק לגישה לחשבון המייל כדי להשלים את ההתחברות. בנוסף, הקוד החד-פעמי מקשה על התקפות brute force, מכיוון שהזמן המוגבל מגביל את מספר הניסיונות האפשרי.

שליחת הודעות לנמענים - בנוסף לאחסון הודעות מוסתרות בשרת, המערכת מאפשרת לשלוח התראות למשתמשים על הודעות חדשות שהוסתרו בתמונות. כשמשתמש שולח הודעה מוסתרת למספר נמענים, המערכת יכולה לשלוח מייל אוטומטי לכל אחד מהם, שמיידע אותם שיש להם הודעה חדשה. המייל לא כולל את ההודעה עצמה (שנשארת מוסתרת בתמונה), אלא רק התראה שמעודדת את הנמען להתחבר למערכת כדי לקרוא את ההודעה. זה משפר את חוויית המשתמש על ידי מתן התראות בזמן-אמת, אך בלי לסכן את הסודיות של ההודעה המוסתרת.

משלוח תמונות - פונקציונליות מתקדמת יותר של המערכת היא היכולת לשלוח את התמונות המקודדות ישירות במייל לנמענים. המשתמש יכול לבחור אם לשלוח רק התראה, או לכלול גם את התמונה המקודדת כקובץ מצורף. במקרה השני, הנמען יכול לשמור את התמונה מהמייל ולפענח אותה באמצעות המערכת, בלי לעבור דרך השרת. זה מספק נתיב תקשורת חלופי, שיכול להיות שימושי אם יש בעיות בחיבור ישיר לשרת. התמונה המצורפת נשלחת בפורמט PNG כדי להבטיח שכל המידע המוסתר נשמר ללא דחיסה מאבדת מידע.

רכיבים נדרשים:

מודול EmailHandler - מודול מקיף לניהול כל היבטי שליחת המיילים. הוא כולל פונקציות לשליחת קודי אימות, התראות על הודעות חדשות, ותמונות מקודדות. המודול מתוכנן להיות מודולרי וגמיש, עם תמיכה בתבניות דואר אלקטרוני שונות לסוגים שונים של מיילים. כל פונקציה בודקת קודם שהפרמטרים תקינים (למשל, שכתובת המייל חוקית), ומטפלת בשגיאות באופן מסודר (למשל, בעיות בחיבור לשרת SMTP). המודול גם תומך בשליחה מקבילה של מיילים למספר נמענים, באמצעות multi-threading, כדי לא לגרום לעיכוב בשרת.

ספריות מייל בפייתון - השתמשתי במספר ספריות סטנדרטיות של פייתון לטיפול בדואר אלקטרוני. הספרייה smtplib משמשת לחיבור לשרתי SMTP (כמו Gmail) ולשליחת המיילים עצמם. ספריית email.mime משמשת ליצירת הודעות מייל מורכבות, כולל טקסט רגיל, תמונות מצורפות, וכותרות מתאימות. ספריית ssl מספקת את שכבת האבטחה לתקשורת עם שרת המייל, ומבטיחה שהמיילים עצמם נשלחים בצורה מאובטחת.

הגדרות שרת SMTP בקובץ env. - כדי לשפר את הגמישות והאבטחה, הנתונים הרגישים של שרת המייל (כתובת אימייל, סיסמה) לא נשמרים בקוד עצמו, אלא בקובץ סביבה נפרד (env.). הקובץ נטען על ידי ספריית dotenv בזמן ריצה, והפרטים משמשים ליצירת החיבור לשרת SMTP. זה מאפשר למנהל המערכת לשנות את פרטי החיבור בלי לשנות את הקוד, ומבטיח שפרטים רגישים לא יופיעו בקוד המקור. הגדרות נוספות, כמו IP השרת וPORT, גם הן ניתנות לשינוי באמצעות הקובץ.

המודול EmailHandler כולל פונקציות מיוחדות לטיפול במקרים מורכבים, כמו שליחת מיילים למספר נמענים במקביל. הפונקציה send_emails_to_multiple_recipients יוצרת תהליכון (thread) נפרד עבור כל נמען, כך שהמיילים נשלחים במקביל ולא ברצף. זה חשוב במיוחד כשיש מספר גדול של נמענים, כי שליחת מייל יכולה לקחת זמן לא מבוטל, ושליחה סדרתית הייתה עלולה לגרום לעיכוב משמעותי.

יכולות צד הלקוח

1. ממשק משתמש גרפי מתקדם

מטרה:

לספק למשתמשים ממשק אינטואיטיבי, אסתטי ופשוט לשימוש, שמסתיר את המורכבות הטכנית של פעולות סטגנוגרפיה והצפנה. ממשק משתמש טוב הוא קריטי להצלחת כל מערכת, במיוחד כזו שעוסקת בטכנולוגיות מורכבות כמו הצפנה וסטגנוגרפיה. המטרה שלי הייתה לבנות ממשק שהוא נעים לעין, קל להבנה, ומספק משוב ברור למשתמש בכל שלב.

פעולות ורכיבים:

מסך פתיחה ואנימציית כניסה - המערכת פותחת באנימציה מרשימה בסגנון סייבר-אבטחה, שמציגה אלמנטים גרפיים כמו קוד בינארי זורם, מנעול מתהווה, והודעות על אתחול מערכות אבטחה. זה לא רק מוסיף לחוויה האסתטית, אלא גם מעביר למשתמש מסר חזק על מהות המערכת - אבטחה, פרטיות וחשאיות. האנימציה נבנתה עם תשומת לב לפרטים, כולל תזמון מדויק, שינויי צבע חלקים, ומעברים נוזליים בין שלבים שונים. היא גם ממלאת תפקיד פרקטי - היא מתבצעת בזמן שהמערכת מאתחלת את מודולי האבטחה ומתחברת לשרת.

מסך אימות ורישום - אחרי האנימציה, המשתמש מגיע למסך כניסה מאובטח עם אפשרויות להתחברות או הרשמה. הממשק כולל טפסים ידידותיים עם בדיקות תקינות בזמן אמת. למשל, כאשר משתמש מזין סיסמה בטופס הרישום, סרגל דינמי מראה את רמת החוזק של הסיסמה, עם שינוי צבע (אדום לחלש, כתום לבינוני, ירוק לחזק) והודעות הסבר על הדרישות. הממשק גם תומך באימות דו-שלבי, עם מסך ייעודי להזנת קוד האימות שנשלח למייל, הכולל שעון עצר ויזואלי שמראה כמה זמן נותר לפני שהקוד פג תוקף. כל המסכים האלה עוצבו בסגנון סייבריסטי, עם פלטת צבעים תואמת לנושא האבטחה (כחול כהה, ירוק צבאי, ואפור-כסוף).

מערכת לשוניות (Tabs) - לאחר התחברות מוצלחת, המשתמש מגיע למסך הראשי, שמאורגן במערכת לשוניות אינטואיטיבית. הלשוניות העיקריות הן "Encode" (להסתרת מידע בתמונות) ו-"Decode" (לפענוח מידע מוסתר). כל לשונית מכילה את כל הפקדים והממשק הרלוונטי למשימה הספציפית. מערכת הלשוניות מפשטת את חווית המשתמש על ידי הסתרת פונקציונליות לא רלוונטית, והצגה רק של הפקדים שהמשתמש צריך באותו רגע. זה מקטין את העומס הקוגניטיבי ומקל על ההתמצאות.

ממשק הסתרת מידע (Encode) - לשונית ה-"Encode" כוללת כל מה שצריך כדי להסתיר מידע בתמונה: לחצן לבחירת תמונה (עם תמיכה בגרירה והשלכה), שדה טקסט להזנת ההודעה הסודית, מונה תווים שמתעדכן בזמן אמת ומראה כמה מהקיבולת המקסימלית נוצלה, ממשק לבחירת נמענים (עם אפשרות חיפוש ואימות מול השרת), ולחצן להסתרת המידע ושליחתו. הממשק מספק משוב ויזואלי ברור בכל שלב - למשל, הוא מראה את התמונה המקורית, מציג את הקיבולת המקסימלית בתווים, ומעדכן את רשימת הנמענים בזמן אמת.

ממשק פענוח מידע (Decode) - לשונית ה-"Decode" פשוטה יותר, וכוללת לחצן לבחירת תמונה, אזור להצגת ההודעה המפוענחת, ולחצן לפענוח. כאשר המשתמש מפענח הודעה בהצלחה, ההודעה מוצגת באופן ברור, עם מידע על השולח והזמן. אם הפענוח נכשל (למשל, אם המשתמש אינו הנמען המיועד), מוצגת הודעת שגיאה ברורה.

חלונות מידע מודאליים - לאחר פעולות משמעותיות, המערכת מציגה חלונות מידע מודאליים שמספקים משוב מפורט. למשל, אחרי הסתרת מידע בתמונה, מוצג חלון שמראה את התמונה המקורית לצד התמונה עם המידע המוסתר (שנראות זהות לעין), את פרטי ההודעה, ואת רשימת הנמענים. חלונות אלו מעוצבים בקפידה, עם כותרות ברורות, שטח תצוגה נדיב, ולחצני פעולה בולטים.

רכיבים דרושים:

ספריית CustomTkinter - בסיס הממשק בנוי על ספרייה מתקדמת זו, שהיא הרחבה של Tkinter הסטנדרטי. CustomTkinter מספקת רכיבי ממשק מודרניים, עם תמיכה בעיצוב מותאם אישית, אנימציות חלקות, ותמיכה טובה יותר בתצוגות במסכים בעלי רזולוציה גבוהה. בחרתי בספרייה זו בגלל השילוב של מראה מודרני, קלות שימוש, ותאימות עם פלטפורמות שונות. השתמשתי בתכונות מתקדמות של הספרייה, כמו אפקטים של hover, מעברי שקיפות, ועיגול פינות של רכיבים.

מחלקת TechButton - יצרתי מחלקה מותאמת אישית שמרחיבה את הכפתור הבסיסי של CustomTkinter, עם סגנון ויזואלי אחיד בכל המערכת. הכפתורים האלה כוללים אפקטים מתקדמים, כמו שינוי צבע כשהעכבר עובר מעליהם, צללית קלה, ואנימציית לחיצה. זה לא רק יוצר מראה מקצועי ואחיד, אלא גם מספק משוב ויזואלי טוב למשתמש, שעוזר לו להבין טוב יותר את אפשרויות האינטראקציה.

מחלקת CyberStyleAnimation - זוהי מחלקה מורכבת שאחראית על אנימציית הפתיחה של המערכת. היא משתמשת בקנבס Tkinter ליצירת אנימציה מתקדמת עם אלמנטים כמו קוד בינארי זורם, מנעול דיגיטלי שמתגבש, וסמל המערכת שמופיע בהדרגה. האנימציה משלבת אפקטים של תנועה, שינויי שקיפות, ושינויי גודל, לקבלת מראה חלק ודינמי.

פלטת צבעים מותאמת - המערכת משתמשת בפלטת צבעים ייחודית שנבחרה בקפידה כדי לשדר תחושה של אבטחה, טכנולוגיה, ומקצועיות. הצבעים העיקריים הם כחול-אפור כהה לרקע (#1E293B), אפור כחול בהיר יותר לאלמנטים משניים (#334155), ירוק זרחני לדגשים (#4ADE80), וגוונים שונים של אפור לטקסט ולאלמנטים משניים. פלטה זו לא רק אסתטית, אלא גם פונקציונלית - היא מאפשרת ניגודיות טובה לקריאות נוחה, היררכיה ויזואלית ברורה, ותומכת במשוב ויזואלי (למשל, אדום לאזהרות, ירוק להצלחה).

הממשק הגרפי שפיתחתי מייצג איזון עדין בין אסתטיקה, שימושיות ואבטחה. למרות שהמערכת מבצעת פעולות מורכבות מאוד מאחורי הקלעים (הצפנה, סטגנוגרפיה, תקשורת מאובטחת), הממשק מציג למשתמש תמונה פשוטה וברורה. כל אלמנט בממשק תוכנן בקפידה, עם תשומת לב לפרטים כמו מרווחים, גדלים, צבעים, ומשוב ויזואלי. התוצאה היא ממשק שנראה מקצועי ומתקדם, אך עדיין נוח וידידותי למשתמש.

לדוגמה, כשמשתמש מנסה להסתיר הודעה בתמונה, הממשק מציג את המידע הרלוונטי (גודל התמונה, קיבולת מקסימלית להסתרה, מספר התווים שכבר הוזנו) בצורה ברורה וישירה. אם ההודעה ארוכה מדי, המערכת מתריעה על כך בזמן-אמת עם שינוי צבע והודעה מתאימה. כשהמשתמש מוסיף נמען לרשימה, המערכת בודקת מיד מול השרת אם המשתמש קיים, ומעדכנת את הרשימה בהתאם, עם משוב ויזואלי. כל אלה הם דוגמאות לתכנון ממשק שלא רק נראה טוב, אלא גם מנחה את המשתמש ומונע טעויות.

2. רישום והתחברות מאובטחים

מטרה:

לספק למשתמשים אפשרות להירשם למערכת ולהתחבר אליה באופן מאובטח ונוח, עם דגש על אבטחה רב-שכבתית. מערכת הזדהות מאובטחת היא היסוד לכל מערכת שעוסקת במידע רגיש, ולכן השקעתי מאמץ רב בפיתוח תהליכי רישום והתחברות שהם גם בטוחים וגם ידידותיים למשתמש.

פעולות ורכיבים:

תהליך רישום מאובטח - המערכת מציגה טופס רישום מקיף שדורש שם מלא, שם משתמש ייחודי, סיסמה חזקה, וכתובת אימייל תקפה. כל שדה עובר בדיקות תקינות בצד הלקוח, בזמן-אמת, עם משוב ויזואלי מיידי. למשל, המערכת בודקת אם שם המשתמש כבר קיים במסד הנתונים (על ידי שאילתה לשרת) ומציגה הודעה מתאימה. לסיסמאות, המערכת מציגה מד חוזק דינמי שמשנה צבע (אדום, כתום, ירוק) לפי רמת האבטחה, ומסביר בדיוק מה חסר (למשל, "נדרשת אות גדולה"). הנתונים נשלחים לשרת רק אחרי שכל הבדיקות עוברות בהצלחה, והשרת עצמו מבצע בדיקות נוספות לפני שמירת המשתמש החדש.

תהליך התחברות מאובטח - תהליך ההתחברות כולל שלושה שלבים: הזנת פרטי כניסה (שם משתמש וסיסמה), אימות דו-שלבי דרך אימייל, ואימות אמינות השרת. בשלב הראשון, המשתמש מזין את שם המשתמש והסיסמה, והמערכת שולחת אותם לשרת באופן מוצפן. אם הפרטים נכונים, המערכת עוברת לשלב השני - אימות דו-שלבי. קוד אימות אקראי נשלח לאימייל של המשתמש, והוא צריך להזין אותו תוך 5 דקות. שעון עצר ויזואלי מראה כמה זמן נותר. בשלב השלישי, המערכת בודקת את אמינות השרת על ידי וידוא שהתשובות שהתקבלו הן אכן מוצפנות עם המפתח הנכון. רק אחרי ששלושת השלבים עוברים בהצלחה, המשתמש מקבל גישה למערכת.

טיפול בשגיאות התחברות - המערכת מטפלת במגוון רחב של מקרי קצה ושגיאות אפשריות בתהליך ההתחברות, תוך אספקת משוב ברור למשתמש. למשל, אם המשתמש מזין שם משתמש או סיסמה שגויים, מוצגת הודעה מדויקת ("שם משתמש או סיסמה שגויים") בלי לחשוף מידע מיותר (המערכת לא מגלה אם השגיאה היא בשם המשתמש או בסיסמה, כדי לא לעזור לתוקפים). אם המשתמש מזין קוד אימות שגוי, או שהקוד פג תוקף, מוצגת הודעה מתאימה. אם יש בעיית תקשורת עם השרת, המערכת מציגה הודעת שגיאה ברורה עם אפשרויות פעולה. בכל המקרים, המערכת מנסה לספק לא רק הסבר על הבעיה, אלא גם הנחיות לפתרון.

ניהול סשן - אחרי התחברות מוצלחת, המערכת מנהלת סשן מאובטח למשתמש. זה כולל שמירת פרטי המשתמש המחובר (בלי הסיסמה, כמובן), ניהול פרטי החיבור לשרת, וטיפול בהתנתקות (יזומה או עקב אי-פעילות). המערכת תומכת בהתנתקות מסודרת, עם ניקוי נכון של כל הנתונים הרגישים מהזיכרון. בנוסף, יש תמיכה בהתאוששות מבעיות חיבור זמניות, עם יכולת לחדש את החיבור בלי לאלץ את המשתמש להתחבר מחדש.

רכיבים נדרשים:

טפסי רישום והתחברות - אלה הם רכיבים ויזואליים מורכבים, שבנויים מ-Frame של CustomTkinter, המכיל שדות קלט, תוויות, וכפתורים. הטפסים מעוצבים בסגנון נקי ומודרני, עם מרווחים נכונים, התאמה לגדלים שונים של מסך, ותמיכה בניווט באמצעות המקלדת (TAB ו-ENTER). הם גם כוללים בדיקות קלט בצד הלקוח, עם הצגת הודעות שגיאה ליד השדות הרלוונטיים.

מנגנון הערכת חוזק סיסמה - רכיב ויזואלי שמורכב משלושה חלקים: מד גרפי (בר צבעוני שמתמלא לפי חוזק הסיסמה), תווית טקסט (שמתארת את רמת החוזק), ורשימת דרישות (שמתעדכנת בזמן-אמת להראות אילו קריטריונים עדיין לא מולאו). הרכיב מחובר לפונקציה שמנתחת את הסיסמה על בסיס קריטריונים מרובים: אורך, נוכחות אותיות גדולות וקטנות, ספרות, ותווים מיוחדים. הניתוח מתבצע בזמן-אמת בכל הקשה, כך שהמשתמש רואה מיד את ההשפעה של השינויים שהוא מבצע.

מסך אימות קוד - מסך שלם שמוקדש להזנת קוד האימות הדו-שלבי. המסך כולל שדה להזנת הקוד, הסבר על התהליך, ושעון עצר ויזואלי שמראה כמה זמן נותר עד לפקיעת תוקף הקוד. שעון העצר מתעדכן בזמן-אמת, עם שינוי צבע כשמתקרבים לסוף הזמן המוקצב. המסך גם כולל אפשרות לחזור לשלב הקודם, למקרה שהמשתמש רוצה להזין שם משתמש או סיסמה אחרים.

חיבור למודול Client - כל פעולות ההתחברות והרישום דורשות תקשורת עם השרת, שמתבצעת דרך מודול ה-Client. הממשק משתמש בפונקציות כמו login_user, register_user, ו-log_login_attempt כדי לתקשר עם השרת, ומטפל בתשובות ובשגיאות שמתקבלות.

המערכת ניצלת את הטכנולוגיות המתקדמות ביותר לאבטחת תהליכי הרישום וההתחברות. כל התקשורת עם השרת מוצפנת ברמת ה-socket באמצעות RSA (הצפנה אסימטרית), הסיסמאות מוצפנות בשרת עם אלגוריתמים חזקים (PBKDF2 עם הרבה איטרציות ומלח ייחודי), ויש אימות דו-שלבי שדורש גישה לחשבון האימייל של המשתמש. אך למרות רמת האבטחה הגבוהה, הממשק נשאר פשוט וידידותי למשתמש, עם הסברים ברורים והתאמה אינטואיטיבית.

3. הצפנת מידע בתמונות

מטרה:

לאפשר למשתמשים להסתיר מידע בתמונות בצורה פשוטה ובטוחה, ולשלוח אותו לנמענים ספציפיים. זוהי אחת הפונקציות המרכזיות של המערכת - היכולת לקחת תמונה רגילה, להסתיר בה מידע בצורה בלתי נראית לעין, ולשלוח אותה ליעדה, כך שרק הנמענים המיועדים יוכלו לחלץ את המידע.

פעולות ורכיבים:

בחירת תמונה ומידע - הממשק מאפשר למשתמש לבחור תמונה מהמחשב שלו (באמצעות חלון בחירת קבצים סטנדרטי) ולהזין את המידע שהוא רוצה להסתיר. המערכת תומכת במגוון פורמטים של תמונות (PNG, JPEG, BMP, GIF), ומספקת משוב מיידי על הבחירה - היא מציגה את התמונה שנבחרה, את שמה ואת גודלה, ומחשבת את כמות המידע המקסימלית שניתן להסתיר בה. חישוב זה מתעדכן בזמן אמת ומוצג למשתמש, כך שהוא יודע כמה תווים הוא יכול להכניס. שדה הטקסט להזנת המידע כולל מונה תווים דינמי, שמתעדכן בכל הקשה ומראה כמה מהקיבולת המקסימלית נוצלה.

בחירת נמענים - המערכת מאפשרת למשתמש לבחור למי המידע מיועד. זה יכול להיות נמען יחיד או מספר נמענים. הממשק כולל שדה להזנת שם משתמש, כפתור הוספה, ורשימה שמציגה את כל הנמענים שכבר נוספו. כשמשתמש מנסה להוסיף נמען, המערכת בודקת מול השרת אם שם המשתמש קיים, ומוסיפה אותו לרשימה רק אם כן. יש גם אפשרות להסיר נמענים מהרשימה, או לנקות את כל הרשימה ולהתחיל מחדש. הממשק מספק משוב ויזואלי ברור בכל שלב.

הסתרת המידע בתמונה - אחרי שהמשתמש בחר תמונה, הזין מידע, ובחר נמענים, הוא יכול להסתיר את המידע באמצעות לחיצה על כפתור "Encode and Send". המערכת מבצעת סדרה של בדיקות לפני ההסתרה, למשל שהתמונה גדולה מספיק להכיל את המידע, שיש לפחות נמען אחד, ושהמידע אינו ריק. אם כל הבדיקות עוברות, המערכת שולחת את התמונה והמידע לשרת, שמסתיר את המידע בתמונה ומחזיר את התמונה המקודדת. המערכת מציגה למשתמש את התמונה המקורית והמקודדת זו לצד זו, להדגמה שאין הבדל נראה לעין.

רכיבים נדרשים:

ממשק ניווט לבחירת תמונות - רכיב שמשלב את חלון הניווט הסטנדרטי של מערכת ההפעלה עם לוגיקה מותאמת אישית לסינון סוגי קבצים ולטיפול בשגיאות. הרכיב מאפשר למשתמש לנווט בתיקיות שלו ולבחור תמונה, עם סינון מובנה שמציג רק קבצי תמונה תומכים. הוא גם כולל מנגנון שמירה של תיקיות אחרונות שהמשתמש ביקר בהן, כדי להקל על ניווט עתידי.

אזור עריכת טקסט מותאם - שדה טקסט מורחב שבנוי על רכיב ה-CTkTextbox של CustomTkinter, עם שיפורים להתאמה לצרכים הספציפיים של המערכת. זה כולל מונה תווים מובנה שמתעדכן בזמן אמת, עיצוב מותאם (צבעים, גופנים, מסגרת), ואפשרויות עריכה מורחבות (כמו ביטול ושחזור). הרכיב גם כולל טיפול חכם במקרים שבהם המשתמש מנסה להזין יותר מדי טקסט - הוא משנה את צבע המונה לאדום ומציג הודעה מתאימה, אך עדיין מאפשר הזנה (למקרה שהמשתמש מתכוון למחוק חלק מהטקסט אחר כך).

מערכת ניהול נמענים - מערכת מורכבת שכוללת שדה להזנת שמות משתמשים, לוגיקה לבדיקת תקינות השמות מול השרת, ואזור תצוגה שמראה את הרשימה המעודכנת של נמענים. כשמשתמש מנסה להוסיף נמען, המערכת שולחת שאילתה לשרת שבודקת אם שם המשתמש קיים, ומציגה משוב ויזואלי מתאים. הרשימה עצמה מוצגת באזור תצוגה עם אפשרות גלילה, ויש כפתורים להסרת נמענים בודדים או לניקוי כל הרשימה.

חלון תוצאות מתקדם - חלון מודאלי שמציג את תוצאות תהליך ההסתרה. החלון מחולק לשלושה אזורים: אזור תמונות, שמציג את התמונה המקורית והמקודדת זו לצד זו; אזור מידע, שמפרט את רשימת הנמענים ואת פרטי ההודעה; ואזור פעולות, עם כפתורים לשמירת התמונה, שליחתה בדוא"ל, או סגירת החלון. החלון מעוצב בסגנון אחיד עם שאר המערכת, והוא נפתח במרכז המסך עם אפקט כניסה חלק.

תצוגת התקדמות שליחת אימיילים - רכיב שמראה את התקדמות תהליך שליחת האימיילים, עם מחוון גרפי שמתעדכן בזמן אמת. הרכיב מציג גם מידע טקסטואלי על מספר האימיילים שנשלחו, אלו שנכשלו, ואלו שעדיין בתור. בסיום התהליך, הוא מציג סיכום מפורט, עם אפשרות לנסות שוב לשלוח לנמענים שהשליחה אליהם נכשלה.

יכולת הסתרת המידע שלי עובדת על כל סוגי התמונות הנפוצים, אך יש הבדלים בתמיכה וביעילות:

- PNG: התמיכה הטובה ביותר, מאחר שזהו פורמט ללא איבוד נתונים.

- JPEG: נתמך, אך בעל סיכון קטן יותר לאיבוד הנתונים המוסתרים עקב דחיסה.

- BMP: נתמך מצוין בשל היעדר דחיסה, אך גודל הקובץ גדול יחסית.

- GIF: תמיכה בסיסית, מוגבל בעיקר לתמונות GIF סטטיות.

הניסיון שלנו הראה שעדיף להשתמש בפורמט PNG לתוצאה הסופית בכל מקרה, גם אם התמונה המקורית היא בפורמט אחר, ולכן המערכת תמיד שומרת את התמונה המקודדת בפורמט זה.

תהליך הפענוח עצמו מתבצע ברובו בצד השרת, אך הממשק בצד הלקוח אחראי על חווית המשתמש ועל הצגת התוצאות. המערכת מוודאת שהמשתמש יקבל משוב ברור וישיר על תוצאות הפענוח, בין אם הוא הצליח או נכשל. בכל מקרה, היא מנחה את המשתמש מה לעשות הלאה, כחלק מהגישה הכללית של המערכת - להיות פשוטה להבנה ולשימוש, גם עבור משתמשים שאינם טכניים.

4. תקשורת מאובטחת עם השרת

מטרה:

להבטיח שכל התקשורת בין אפליקציית הלקוח לבין השרת מאובטחת, ושהמידע שעובר ביניהם מוגן מפני האזנה, שינוי, או זיוף. זהו היסוד הטכנולוגי שעליו בנויות כל יכולות המערכת, והוא חיוני להבטחת פרטיות ואבטחת המידע של המשתמשים.

פעולות ורכיבים:

יצירת חיבור מאובטח - כאשר אפליקציית הלקוח מופעלת, הדבר הראשון שהיא עושה הוא יצירת חיבור ל-Socket עם השרת. מיד לאחר יצירת החיבור, מתבצע תהליך החלפת מפתחות, שבו כל צד מייצר זוג מפתחות RSA (ציבורי ופרטי) ושולח את המפתח הציבורי לצד השני. הלקוח שומר את המפתח הציבורי של השרת, והשרת שומר את המפתח הציבורי של הלקוח. כך, כל צד יכול להצפין מידע עם המפתח הציבורי של הצד השני, והצד השני יכול לפענח אותו עם המפתח הפרטי שלו. התהליך הזה מבטיח שאפילו אם מישהו מאזין לתקשורת, הוא לא יוכל להבין את תוכנה.

הצפנת בקשות לשרת - לפני שליחת כל בקשה לשרת, הלקוח מצפין אותה עם המפתח הציבורי של השרת. הבקשה עצמה היא בדרך כלל אובייקט JSON, שכולל את סוג הבקשה ואת הנתונים הרלוונטיים. למשל, בקשת התחברות תכלול את שם המשתמש והסיסמה, ובקשת הסתרת מידע תכלול את התמונה, ההודעה, ורשימת הנמענים. האובייקט JSON מומר למחרוזת, מוצפן עם המפתח הציבורי של השרת, ונשלח דרך ה-Socket. ההצפנה מבטיחה שגם אם מישהו יצליח ליירט את התקשורת, הוא לא יוכל לקרוא את תוכן הבקשה או לזייף בקשות חדשות. זה מגן על מידע רגיש כמו סיסמאות, נתוני משתמשים, והודעות פרטיות.

פענוח תשובות מהשרת - כשהשרת מקבל בקשה מהלקוח, הוא מעבד אותה ושולח תשובה. התשובה מוצפנת עם המפתח הציבורי של הלקוח, כך שרק הלקוח הספציפי שביצע את הבקשה יכול לפענח אותה. כשהלקוח מקבל את התשובה המוצפנת, הוא מפענח אותה עם המפתח הפרטי שלו, ממיר אותה חזרה לאובייקט JSON, ומעבד אותה בהתאם. תהליך זה מבטיח שרק הלקוח המיועד יכול לקרוא את התשובה, וגם מאמת שהתשובה אכן הגיעה מהשרת (שכן רק השרת יכול היה ליצור תשובה שניתן לפענח עם המפתח הפרטי של הלקוח).

טיפול בשגיאות תקשורת - מערכת התקשורת כוללת טיפול מקיף בשגיאות אפשריות, כמו בעיות בהתחברות לשרת, ניתוקים לא צפויים, או זמן תגובה ארוך מדי. כשמתרחשת שגיאה, המערכת מציגה למשתמש הודעה מתאימה, עם הסבר על הבעיה והמלצות לפתרון. לדוגמה, אם הלקוח לא מצליח להתחבר לשרת, הוא יציג הודעה כמו "לא ניתן להתחבר לשרת. אנא ודא שהשרת פעיל ושיש לך חיבור אינטרנט תקין." המערכת גם מנסה להתאושש באופן אוטומטי ממצבי שגיאה, במידת האפשר, כמו ניסיון מחודש של התחברות אחרי זמן מסוים.

רכיבים נדרשים:

מודול Client - מודול מרכזי שמטפל בכל התקשורת עם השרת. הוא מספק ממשק פשוט וברור לשאר חלקי המערכת, עם פונקציות כמו connect (להתחברות לשרת), send_request (לשליחת בקשה ספציפית), close (לסגירת החיבור), וכו׳. המודול מטפל בכל הפרטים הטכניים של התקשורת, כולל יצירת החיבור, החלפת מפתחות, הצפנה ופענוח, וטיפול בשגיאות. הוא גם מטפל בפירוק והרכבה של הודעות גדולות, וניהול סטטוס החיבור.

שימוש במודול SecurityCrypto - לצורך כל פעולות ההצפנה והפענוח, הלקוח משתמש באותו מודול SecurityCrypto שמשמש גם את השרת. המודול מספק פונקציות כמו encrypt (להצפנת מידע עם המפתח הציבורי של השרת), decrypt (לפענוח תשובות עם המפתח הפרטי של הלקוח), generate_keys (ליצירת זוג מפתחות חדש), וכו׳. השימוש באותו מודול בשני הצדדים מבטיח תאימות מלאה ומפשט את הקוד.

אובייקט Socket לתקשורת רשת - החיבור עצמו מתבצע באמצעות אובייקט Socket, שמספק ערוץ תקשורת דו-כיווני עם השרת. השימוש ב-Socket מאפשר תקשורת יעילה ואמינה, עם יכולת לשלוח ולקבל מידע בכל גודל. האובייקט מוגדר עם timeout מתאים, כדי למנוע תקיעות במקרה של בעיות תקשורת, והוא מנוהל בצורה בטוחה, עם סגירה מסודרת בסיום השימוש.

מערכת התקשורת המאובטחת מהווה את התשתית הטכנולוגית שעליה בנויות כל יכולות המערכת. היא מבטיחה שכל המידע שעובר בין הלקוח לשרת - פרטי משתמשים, סיסמאות, הודעות, ותמונות - מוגן מפני האזנה, שינוי, או זיוף. זה חיוני במיוחד במערכת כמו זו, שעוסקת במידע רגיש וסודי.

המערכת משתמשת בהצפנה אסימטרית (RSA), שנחשבת לאחת השיטות המאובטחות ביותר לתקשורת. הצפנה אסימטרית מאפשרת לשני צדדים לתקשר בצורה מאובטחת גם אם הם לא חולקים סוד משותף מראש. זה פותר את ״בעיית התרנגולת והביצה״ של הצפנה סימטרית - איך להעביר את המפתח בצורה מאובטחת. עם הצפנה אסימטרית, כל צד יכול ליצור זוג מפתחות, לשמור את המפתח הפרטי לעצמו, ולשתף את המפתח הציבורי עם העולם. כך, כל אחד יכול לשלוח לו מידע מוצפן, אך רק הוא יכול לפענח אותו.

נקודה חשובה נוספת היא שהמערכת מצפינה לא רק את תוכן ההודעות, אלא גם את המטא-דאטה - למי ההודעה מיועדת, מי שלח אותה, וכו׳. גם המטא-דאטה הזה יכול להיות רגיש, ולכן חשוב להגן עליו. זה מתאים לפילוסופיה הכללית של המערכת - להגן לא רק על תוכן התקשורת, אלא גם על עצם קיומה.

סיכום יכולות המערכת

מערכת StegnoLogic שפיתחתי מהווה פתרון מקיף לסטגנוגרפיה מאובטחת, עם דגש על קלות שימוש, אבטחה רב-שכבתית, ופונקציונליות עשירה. המערכת מחולקת לשני חלקים עיקריים - צד השרת וצד הלקוח - שעובדים יחד בהרמוניה כדי לספק חוויית משתמש חלקה ומאובטחת.

בצד **השרת**, המערכת כוללת יכולות מתקדמות כמו:

- **ניהול משתמשים ואימות זהות** - רישום משתמשים, אימות התחברות, אבטחת סיסמאות, ואימות דו-שלבי.

- **ניהול לוגים ואבטחת שרת** - תיעוד פעילות, הגנה מפני התקפות DDOS, וניטור חיבורים.

- **סטגנוגרפיה - הטמעת וחילוץ מידע** - הסתרת מידע בתמונות, בדיקת הרשאות הנמען, וחילוץ מידע מוסתר.

- **ניהול תקשורת מאובטחת** - הצפנת כל התקשורת עם RSA, החלפת מפתחות, וטיפול בהודעות גדולות.

- **שליחת מיילים והודעות** - תמיכה באימות דו-שלבי, התראות על הודעות חדשות, ושליחת תמונות מקודדות.

בצד **הלקוח**, המערכת מציעה ממשק משתמש מודרני ואינטואיטיבי, עם יכולות כמו:

- **ממשק משתמש גרפי מתקדם** - עיצוב בסגנון סייבר-אבטחה, אנימציות חלקות, ומערכת לשוניות אינטואיטיבית.

- **רישום והתחברות מאובטחים** - טפסים קלים לשימוש, בדיקת חוזק סיסמה בזמן אמת, ואימות דו-שלבי.

- **הצפנת מידע בתמונות** - ממשק לבחירת תמונה, הזנת מידע, בחירת נמענים, והסתרת המידע.

- **פענוח מידע מתמונות** - בחירת תמונה, פענוח המידע המוסתר, והצגתו בצורה נוחה.

- **ניהול נמענים ושליחת הודעות** - הוספה וניהול של נמענים, בדיקת תקינות, ושליחת תמונות מקודדות בדוא"ל.

- **תקשורת מאובטחת עם השרת** - הצפנת כל הבקשות והתשובות, טיפול בשגיאות, וניהול סשן.

היתרון הגדול של המערכת הוא השילוב בין אבטחה ברמה גבוהה לבין נוחות שימוש. המערכת מיישמת את כל שיטות האבטחה המקובלות בתעשייה - הצפנת RSA, אימות דו-שלבי, הצפנת סיסמאות, הגנה מפני DDOS - אך מסתירה את המורכבות הטכנית מאחורי ממשק פשוט ואינטואיטיבי. זה מאפשר גם למשתמשים שאינם טכניים ליהנות מיתרונות הסטגנוגרפיה והאבטחה המתקדמת.

גישת האבטחה העמוקה של המערכת מתבטאת גם בפרטים הקטנים - למשל, בדיקות תקינות קלט בכל מקום אפשרי, שימוש בפרוטוקולים מאובטחים (כמו SSL/TLS לאימיילים) ועוד..

ארכיטקטורת הפרויקט ותכנון המערכת

1. תיאור ארכיטקטורת המערכת

מערכת StegnoLogic שבניתי מבוססת על ארכיטקטורת לקוח-שרת מובהקת. בחרתי בארכיטקטורה זו כדי לאפשר ניהול מרכזי של משתמשים והודעות, ובמקביל לספק ממשק משתמש עשיר וקל לשימוש בצד הלקוח. המערכת יכולה לרוץ במגוון תצורות, כולל שרת מרכזי ולקוחות מרובים, או אפילו שרת ולקוח על אותה מכונה לשימוש מקומי.

מבחינת הפריסה הלוגית, המערכת מחולקת לכמה רכיבים עיקריים:

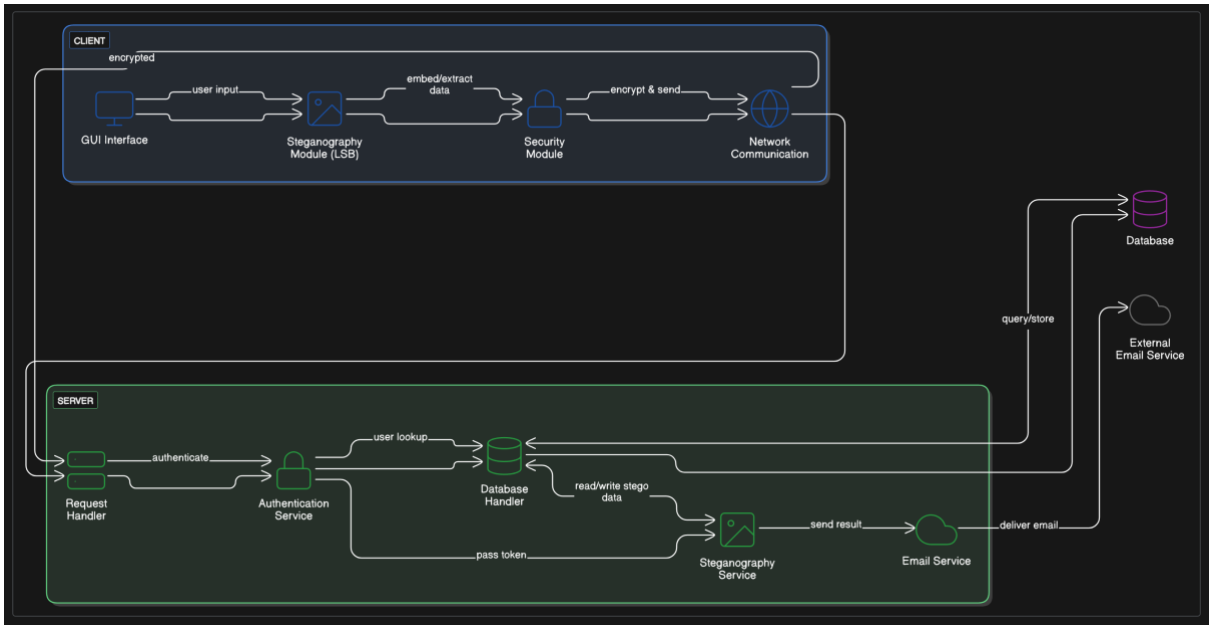
- **שרת מרכזי** - מנהל את מסד הנתונים, משתמשים, תקשורת, ואת מנוע הסטגנוגרפיה עצמו. השרת אחראי גם על אבטחת המידע והתקשורת.

- **אפליקציית לקוח** - מספקת ממשק גרפי למשתמש, ומאפשרת הסתרת מידע, פענוח מידע, ותקשורת עם השרת.

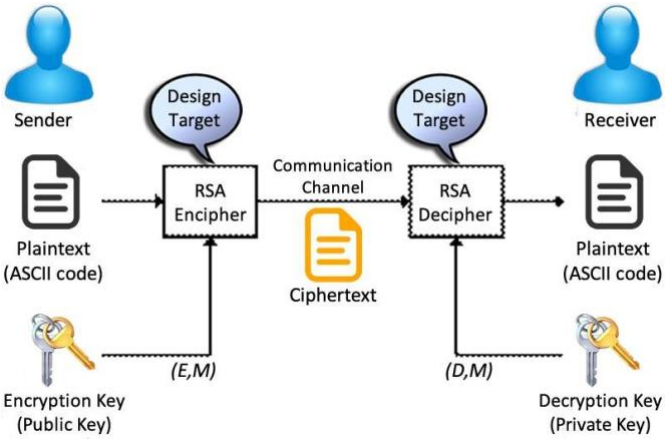
- **מסד נתונים** - רץ בצד השרת ומאחסן נתוני משתמשים, הודעות, ולוגים.

- **מערכת דואר אלקטרוני** - משולבת בשרת לצורך שליחת קודי אימות והודעות.

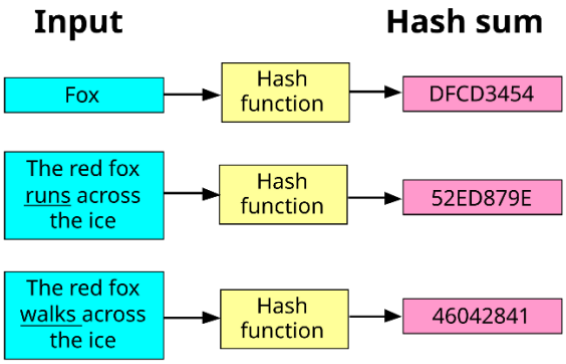
התקשורת בין הלקוח לשרת מתבצעת דרך חיבור TCP/IP מאובטח, כאשר כל ההודעות מוצפנות באמצעות RSA. כל משתמש במערכת יכול להתקשר עם משתמשים אחרים דרך השרת, על ידי הסתרת הודעות בתמונות ושליחתן למשתמשים אחרים.



דיאגרמה המסבירה איך עובד RSA



תרשים המסביר איך עובד HASH



בפרויקט הנוכחי, המערכת מתוכננת לרוץ בסביבה מקומית או ארגונית, ולא כשירות ענן ציבורית, בשל רגישות המידע שעובר בה. עם זאת, התכנון המודולרי שלה מאפשר בקלות יחסית להתאים אותה לסביבת ענן בעתיד, אם יידרש.

2. טכנולוגיות רלוונטיות שנעשה בהן שימוש

בחרתי במגוון טכנולוגיות לפיתוח המערכת, כל אחת מסיבות ספציפיות:

שפות תכנות:

- **Python** - בחרתי בפייתון כשפה העיקרית לפרויקט בגלל הפשטות, הגמישות והספריות הרבות שהיא מציעה לעיבוד תמונה, הצפנה ופיתוח ממשקי משתמש. פייתון גם נחשבת לשפה קלה יחסית ללמידה וקריאה, מה שיקל על תחזוקה עתידית של הקוד.

מערכת הפעלה:

- **קרוס-פלטפורמה** – בעוד שאני השתמשתי בWindows , המערכת תוכננה לרוץ על Windows, Linux ו-macOS, מכיוון שבחרתי בטכנולוגיות שעובדות על כל הפלטפורמות הללו. זה מאפשר גמישות מקסימלית למשתמשים.

פרוטוקולי תקשורת:

- **TCP/IP-ו-Sockets** - בחרתי ב-TCP כי הוא מבטיח העברת נתונים אמינה ומסודרת, חיוני למערכת שמעבירה מידע רגיש. השתמשתי בספריית Socket של פייתון ליישום התקשורת בין הלקוח לשרת.

- **SMTP** - לתקשורת דואר אלקטרוני, המשמשת לאימות דו-שלבי ולשליחת התראות על הודעות חדשות.

תחומי מיקוד:

- **הצפנה ואבטחת מידע** - השתמשתי בספריית RSA לצורך הצפנה אסימטרית, שמאפשרת תקשורת מאובטחת בין הלקוח לשרת.

- **סטגנוגרפיה** - פיתחתי אלגוריתם (Least Significant Bit) LSB מותאם אישית להסתרת מידע בתמונות, עם תוספות ייחודיות כמו שילוב מזהי נמענים בהודעה המוסתרת.

- **אבטחת משתמשים** - יישמתי מערכת אימות דו-שלבי ואחסון סיסמאות מאובטח באמצעות PBKDF2 עם "מלח" אקראי.

- **ממשק משתמש גרפי** - השתמשתי בספריית CustomTkinter לפיתוח ממשק גרפי מודרני ואינטואיטיבי.

ספריות ומסגרות עבודה עיקריות:

- **PIL (Python Imaging Library)** - לעיבוד תמונות ולעבודה עם פיקסלים, חיוני לסטגנוגרפיה.

- **CustomTkinter** - הרחבה של Tkinter הסטנדרטי, שמספקת רכיבי ממשק גרפי מודרניים ואסתטיים יותר.

- **SQLite** - מסד נתונים קל משקל שמתאים לפרויקט זה, מכיוון שהוא לא דורש שרת מסד נתונים נפרד.

- **smtplib** - לשליחת אימיילים מתוך האפליקציה.

- **hashlib ו-PBKDF2** - להצפנת סיסמאות ולאחסון מאובטח שלהן.

בחרתי בפייתון כשפה העיקרית בגלל השילוב המנצח של קלות פיתוח וזמינות של ספריות. לדוגמה, ספריית PIL מספקת גישה נוחה לפיקסלים בתמונה, מה שחיוני לאלגוריתם ה-LSB שמשתמש בביטים הפחות משמעותיים של ערכי צבע להסתרת מידע. CustomTkinter נבחר כי הוא מאפשר יצירת ממשק גרפי אטרקטיבי וידידותי למשתמש, שהוא גם קרוס-פלטפורמה, מה שמאפשר למערכת לרוץ על מגוון מערכות הפעלה ללא שינויים.

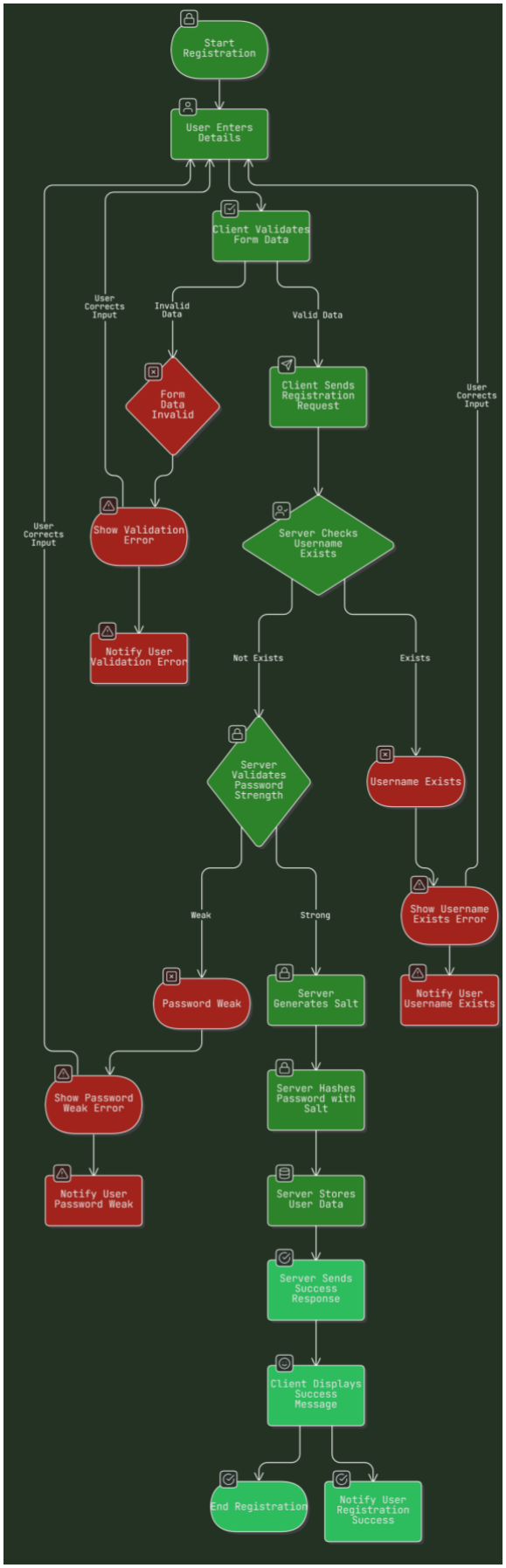
השתמשתי ב-SQLite כמסד נתונים בגלל הפשטות והניידות שלו. למרות היותו קל-משקל, SQLite מספק את כל היכולות הנדרשות למערכת שלנו - אחסון נתוני משתמשים, הודעות, לוגים ותמונות.

3. תיאור זרימת הנתונים

להלן תיאור של זרימת הנתונים עבור היכולות העיקריות של המערכת:

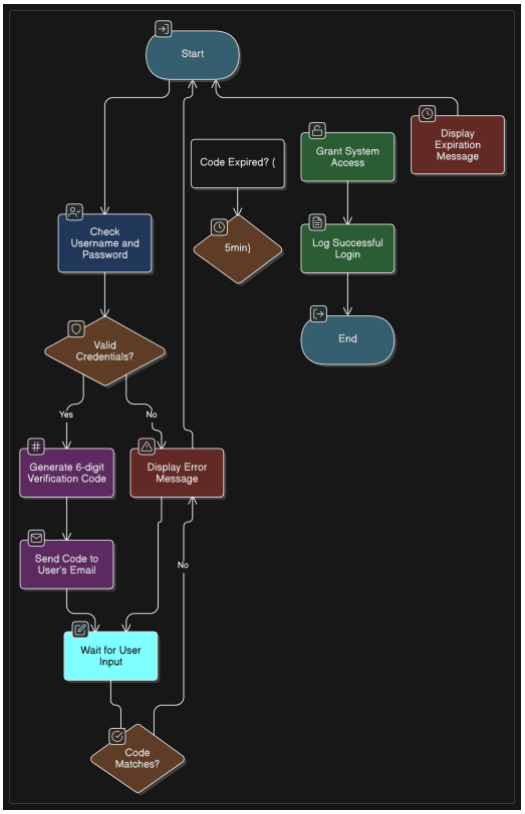
תהליך הרישום:

1. המשתמש מזין פרטים בטופס הרישום (שם מלא, שם משתמש, סיסמה, אימייל).
2. הלקוח מבצע בדיקות תקינות ראשוניות של הקלט.
3. הלקוח שולח בקשת רישום מוצפנת לשרת, הכוללת את כל הפרטים.
4. השרת מפענח את הבקשה ומבצע בדיקות תקינות נוספות:
   - בודק אם שם המשתמש כבר קיים במסד הנתונים
   - בודק אם האימייל תקף
   - בודק אם הסיסמה עומדת בדרישות האבטחה
5. אם הבדיקות עוברות בהצלחה, השרת:
   - מייצר "מלח" אקראי
   - מחשב גיבוב של הסיסמה+המלח באמצעות PBKDF2
   - שומר את שם המשתמש, השם המלא, גיבוב הסיסמה+המלח, וכתובת האימייל במסד הנתונים
6. השרת שולח תשובה מוצפנת ללקוח, שמציינת הצלחה או כישלון ופרטי השגיאה במקרה של כישלון.
7. הלקוח מפענח את התשובה ומציג למשתמש הודעה מתאימה.


Start Registration
User Enters Details
Client Validates Form Data
User Corrects Input
Invalid Data
Valid Data
Form Data Invalid
Client Sends Registration Request
User Corrects Input
User Corrects Input
Show Validation Error
Server Checks Username Exists
Notify User Validation Error
Not Exists
Exists
Server Validates Password Strength
Username Exists
Show Username Exists Error
Weak
Strong
Password Weak
Server Generates Salt
Notify User Username Exists
Show Password Weak Error
Server Hashes Password with Salt
Notify User Password Weak
Server Stores User Data
Server Sends Success Response
Client Displays Success Message
End Registration
Notify User Registration Success

תהליך ההתחברות ואימות דו-שלבי:

1. המשתמש מזין שם משתמש וסיסמה בטופס ההתחברות.
2. הלקוח שולח בקשת התחברות מוצפנת לשרת, הכוללת את שם המשתמש והסיסמה.
3. השרת מפענח את הבקשה ומבצע אימות:
   - מחפש את שם המשתמש במסד הנתונים
   - אם נמצא, מחשב גיבוב של הסיסמה שהוזנה עם ה"מלח" השמור
   - משווה את הגיבוב שחושב עם הגיבוב השמור במסד הנתונים
4. אם האימות הצליח, השרת:
   - מייצר קוד אימות אקראי בן 6 ספרות
   - שולח את הקוד באימייל לכתובת האימייל הרשומה של המשתמש
   - מתעד את ניסיון ההתחברות בטבלת הלוגים
5. השרת שולח תשובה מוצפנת ללקוח, הכוללת את תוצאת האימות ואת קוד האימות במקרה של הצלחה.
6. הלקוח מפענח את התשובה ומציג למשתמש מסך להזנת קוד האימות, עם שעון עצר שמציג את הזמן הנותר (5 דקות).
7. המשתמש מזין את הקוד שקיבל באימייל.
8. הלקוח בודק אם הקוד תואם לקוד שהתקבל מהשרת, ואם כן, מעביר את המשתמש למסך הראשי של האפליקציה.



תהליך הסתרת מידע בתמונה:

1. המשתמש בוחר תמונה, מזין הודעה, ובוחר נמענים.
2. הלקוח שולח בקשה לשרת לחישוב קיבולת מקסימלית של התמונה, הכוללת את נתוני התמונה.
3. השרת מחשב את הקיבולת ומחזיר אותה ללקוח.
4. הלקוח מוודא שאורך ההודעה אינו חורג מהקיבולת המקסימלית.
5. המשתמש לוחץ על "Encode and Send".
6. הלקוח שולח בקשה מוצפנת לשרת, הכוללת את התמונה, ההודעה, ורשימת הנמענים.
7. השרת מבצע את הסתרת המידע:
   - מכין מחרוזת שכוללת את רשימת הנמענים, מופרדים בתו "|", ואחריה נקודותיים והמידע עצמו
   - ממיר את המחרוזת לרצף ביטים
   - טוען את התמונה וניגש לפיקסלים שלה
   - מסתיר את הביטים בביטים הפחות משמעותיים של ערכי הצבע (RGB) של כל פיקסל
   - מוסיף סמן סיום להודעה
   - שומר את התמונה המקודדת בפורמט PNG
8. השרת שומר את המידע במסד הנתונים:
   - שומר את התמונה המקודדת בטבלת התמונות
   - מוסיף רשומה לטבלת ההודעות עבור כל נמען
9. השרת שולח תשובה מוצפנת ללקוח, הכוללת את התמונה המקודדת ומזהה התמונה במסד הנתונים.
10. הלקוח מפענח את התשובה, שומר את התמונה המקודדת במחשב המקומי, ומציג תצוגה מקדימה של התמונה המקורית והמקודדת.
11. אם המשתמש בחר לשלוח אימיילים, הלקוח:
    - מבקש מהשרת את כתובות האימייל של הנמענים
    - שולח בקשה לשרת לשלוח את התמונה המקודדת בדוא"ל לנמענים
    - מציג למשתמש עדכון על מצב השליחה

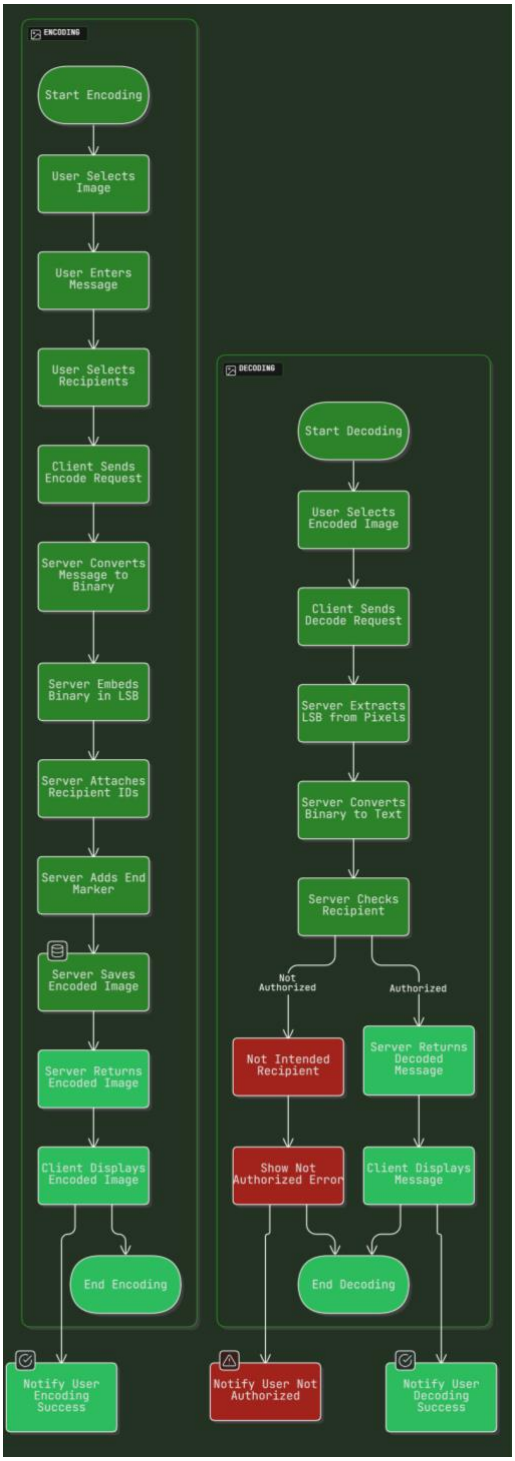
תהליך פענוח מידע מתמונה:

1. המשתמש בוחר תמונה מקודדת ולוחץ על "Decode".
2. הלקוח שולח בקשה מוצפנת לשרת, הכוללת את התמונה ואת שם המשתמש הנוכחי.
3. השרת מבצע את פענוח המידע:
   - טוען את התמונה וניגש לפיקסלים שלה
   - מחלץ את הביטים הפחות משמעותיים מכל ערכי הצבע
   - ממיר את רצף הביטים למחרוזת טקסט
   - מזהה את חלק הנמענים (לפני הנקודותיים) ואת ההודעה עצמה (אחרי הנקודותיים)
   - בודק אם המשתמש הנוכחי נמצא ברשימת הנמענים

4. אם המשתמש נמצא ברשימת הנמענים, השרת מחזיר את ההודעה המפוענחת. אחרת, הוא מחזיר הודעת שגיאה.

5. השרת שולח תשובה מוצפנת ללקוח, הכוללת את תוצאת הפענוח.

6. הלקוח מפענח את התשובה ומציג למשתמש את ההודעה המפוענחת או את הודעת השגיאה.



4. אלגוריתמים שבשימוש בפרויקט

אלגוריתם הסטגנוגרפיה (LSB - Least Significant Bit)

הבעיה שהאלגוריתם פותר:

הסתרת מידע בתמונות דיגיטליות בצורה שאינה מורגשת לעין אנושית, כך שאפילו מי שיש לו גישה לתמונה לא ידע שיש בה מידע מוסתר.

סקירת אלגוריתמים קיימים בתחום:

אלגוריתם LSB הוא אחד האלגוריתמים הנפוצים ביותר בתחום הסטגנוגרפיה בתמונות. הוא משתמש בעובדה שהביט הפחות משמעותי בכל ערך RGB של פיקסל יכול להשתנות מבלי שהשינוי יהיה נראה לעין. אלגוריתמים אחרים כוללים:

- PVD (Pixel Value Differencing) - שיטה שמחביאה יותר מידע באזורי קצה של התמונה

- Wavelet Transform - מסתיר מידע בתחום התדרים באמצעות התמרות גל

לאחר מחקר מעמיק בתחום, בחרתי באלגוריתם LSB הבסיסי והוספתי לו שיפורים משלי.

למה בחרתי באלגוריתם LSB:

1. **פשטות ויעילות** - ה-LSB קל ליישום אך אפקטיבי מאוד, מה שהתאים לזמן והמשאבים שהיו לי לפרויקט.
2. **השפעה מינימלית על התמונה** - השינויים בביט בודד לכל ערך צבע הם כמעט בלתי נראים לעין.
3. **קיבולת סבירה** - ניתן להסתיר עד 3 ביטים בכל פיקסל (אחד בכל ערוץ RGB), מה שמספיק לרוב ההודעות.
4. **תאימות לפורמטים ללא דחיסה** - האלגוריתם עובד מצוין על פורמטים כמו PNG ו-BMP.

השיפורים שהוספתי לאלגוריתם:

1. **שילוב מזהי נמענים** - הוספתי מנגנון ייחודי שמאפשר להסתיר הודעה שמיועדת למספר נמענים ספציפיים, כך שרק הם יוכלו לפענח אותה. זה מתבצע על ידי הוספת רשימת שמות משתמשים בתחילת ההודעה המוסתרת.
2. **סמן סיום** - הוספתי סמן סיום ייחודי ("1111111111111110") כדי לדעת בדיוק היכן ההודעה מסתיימת בזמן הפענוח.
3. **פקטור בטיחות** - בחישוב הקיבולת המקסימלית, אני משתמש רק ב-90% מהקיבולת התיאורטית, כדי להשאיר מרווח בטיחות למטא-דאטה ולסמן הסיום.
4. **תמיכה בגופן עברית** - וידאתי שהאלגוריתם תומך כראוי בתווי Unicode, כולל עברית, על ידי המרה נכונה של כל תו ל-8 ביטים בקידוד UTF-8.

אלגוריתם הצפנה (RSA) לתקשורת מאובטחת

הבעיה שהאלגוריתם פותר:

אבטחת התקשורת בין הלקוח לשרת, כך שגם אם מישהו מיירט את התקשורת, הוא לא יוכל לקרוא את תוכנה או לזייף בקשות.

סקירת אלגוריתמים קיימים בתחום:

RSA הוא אלגוריתם הצפנה אסימטרי שפותח ב-1977, והוא אחד האלגוריתמים הנפוצים ביותר להצפנה א-סימטרית. אלגוריתמים אחרים כוללים:

- ECC (Elliptic Curve Cryptography) - משתמש בעקומים אליפטיים, מספק אבטחה דומה עם מפתחות קצרים יותר

- DSA (Digital Signature Algorithm) - מיועד בעיקר לחתימות דיגיטליות

- AES (Advanced Encryption Standard) - הצפנה סימטרית, מהירה יותר אך דורשת מפתח משותף

- TLS/SSL - פרוטוקולים שמשלבים הצפנה אסימטרית וסימטרית

למה בחרתי באלגוריתם RSA:

1. **אבטחה מוכחת** - RSA נחשב לאלגוריתם מאובטח ומהימן כשמשתמשים בו נכון עם מפתחות באורך מתאים.

2. **זמינות** - קיימות ספריות RSA זמינות ומתוחזקות היטב בפייתון, מה שאפשר לי להתמקד בפתרון הבעיה ולא ביישום האלגוריתם.

3. **פשטות יחסית** - על אף מורכבותו המתמטית, הממשק לשימוש ב-RSA פשוט יחסית.

4. **אימות מקור** - RSA מאפשר גם חתימות דיגיטליות, שמוודאות את מקור ההודעה.

השיפורים והתאמות שביצעתי:

1. **טיפול בהודעות גדולות** - פיתחתי מנגנון לפיצול הודעות גדולות למקטעים שלא עולים על 245 בייט (המגבלה של RSA עם מפתחות 2048 סיביות), הצפנת כל מקטע בנפרד, ואיחוד המקטעים המוצפנים עם סימן מיוחד.

2. **ניהול מפתחות דינמי** - בנוסף, פיתחתי מנגנון שמייצר זוג מפתחות חדש לכל סשן, במקום לעבוד עם מפתחות קבועים.

3. **קידוד Base64** - כדי להקל על העברת הנתונים המוצפנים כטקסט, הוספתי קידוד Base64 אוטומטי לפני שליחה ופענוח Base64 בעת קבלה.

אלגוריתם אחסון סיסמאות (PBKDF2)

הבעיה שהאלגוריתם פותר:

אחסון בטוח של סיסמאות משתמשים, כך שגם אם מסד הנתונים נחשף, התוקף לא יוכל לדעת את הסיסמאות המקוריות.

סקירת אלגוריתמים קיימים בתחום:

קיימים מספר אלגוריתמים לגיבוב סיסמאות:

- MD5 ו-SHA1 - אלגוריתמים מהירים מאוד, אך נחשבים כיום לא בטוחים מספיק לאחסון סיסמאות.

- SHA-512 - חזק יותר, אך עדיין מהיר מדי ופגיע להתקפות ברוטאליות עם חומרה מודרנית.

- bcrypt - תוכנן במיוחד לגיבוב סיסמאות, איטי במכוון ומשתמש במלח מובנה.

- PBKDF2 (Password-Based Key Derivation Function 2) - אלגוריתם מבוסס על פונקציות גיבוב חוזרות, עם מספר גבוה של איטרציות ומלח.

למה בחרתי באלגוריתם PBKDF2:

1. **תקן מוכר ומאומת** - PBKDF2 עבר אימות וסקירה נרחבים.

2. **גמישות** - ניתן לשלוט במספר האיטרציות כדי להקשות על התקפות, ולהשתמש בפונקציות גיבוב שונות כבסיס.

3. **תמיכה מובנית בפייתון** - PBKDF2 נתמך בספריית hashlib הסטנדרטית של פייתון, מה שהקל על היישום.

4. **איטיות מכוונת** - האלגוריתם נועד להיות איטי כדי להקשות על התקפות כוח גס, אך עדיין מספיק מהיר לאימות חוקי.

היישום הספציפי שלי:

1. **"מלח" גדול וייחודי** - אני מייצר "מלח" אקראי בגודל 32 בייט עבור כל משתמש, מה שמונע התקפות rainbow table וטבלאות מוכנות מראש.

2. **100,000 איטרציות** - בחרתי במספר גבוה של איטרציות (100,000) כדי להאט את תהליך הגיבוב, מה שמקשה מאוד על התקפות כוח גס.

3. **אלגוריתם SHA-256** - משתמש ב-SHA-256 כפונקציית הגיבוב הבסיסית, שנחשבת בטוחה עם המספר הגבוה של האיטרציות.

4. **אחסון משולב של מלח וגיבוב** - אני שומר את המלח ביחד עם הגיבוב במסד הנתונים, מקודדים ב-Base64, מה שמפשט את תהליך האימות.

אלגוריתם זיהוי והגנה מפני DDOS (Rate Limiting)

הבעיה שהאלגוריתם פותר:

הגנה על השרת מפני התקפות מניעת שירות מבוזרות (DDOS) או התקפות כוח גס על מנגנון ההתחברות.

סקירת אלגוריתמים קיימים בתחום:

- Token Bucket - אלגוריתם פשוט עם "דלי" של אסימונים שמתמלא בקצב קבוע

- Leaky Bucket - דומה ל-Token Bucket, אך בדגש על קצב יציאה קבוע

למה בחרתי באלגוריתם Sliding Window Logs:

1. **דיוק** - האלגוריתם שומר מעקב מדויק אחרי הזמן של כל בקשה, מה שמאפשר מדידה מדויקת של קצב הבקשות.
2. **הוגנות** - משתמשים לגיטימיים לא "ייענשו" על פעילות קודמת אם הם עומדים במגבלת הקצב הנוכחית.
3. **חיסכון במשאבים** - המנגנון מסנן החוצה בקשות ישנות, כך שלא נוצר צריכת זיכרון מוגזמת לאורך זמן.
4. **פשטות היישום** - ניתן ליישם את האלגוריתם בצורה פשוטה יחסית, עם רשימה של חותמות זמן.

היישום הספציפי שלי:

פיתחתי פונקציה limit_connection_rate שמקבלת רשימה של חותמות זמן (ההיסטוריה) ומספר מקסימלי של בקשות לדקה. הפונקציה:

1. מחשבת את הזמן המינימלי לבקשות רלוונטיות (דקה אחת אחורה מהזמן הנוכחי)
2. מסננת החוצה בקשות ישנות (לפני יותר מדקה)
3. בודקת אם מספר הבקשות בדקה האחרונה חורג מהמקסימום המותר
4. אם לא, היא מוסיפה את הבקשה הנוכחית להיסטוריה ומאשרת אותה
5. אם כן, היא דוחה את הבקשה

5. סביבת הפיתוח

לפיתוח מערכת StegnoLogic השתמשתי בסביבת פיתוח מקיפה ומגוונת, שנבחרה על מנת לספק את כל הכלים הדרושים לכתיבת קוד איכותי וסביבת בדיקות יעילה. להלן פירוט הכלים העיקריים:

כלי פיתוח קוד:

- **PyCharm** - בחרתי בסביבת הפיתוח הזו בגלל התמיכה המעולה שלה בפייתון, היכולות המתקדמות לניתוח קוד, והאינטגרציה עם Git. השתמשתי במיוחד ביכולות הדיבוג המתקדמות שלה, שאפשרו לי לעקוב אחרי זרימת התוכנית ולזהות באגים בקלות.

- SQLite - כלי זה שימש אותי לניהול וחקירה של מסד הנתונים SQLite. הוא אפשר לי לצפות בטבלאות, לערוך את המבנה, ולבדוק שאילתות SQL באופן ויזואלי, מה שהיה חיוני לבדיקת פעולות מסד הנתונים.

כלי בדיקות:

- **Wireshark** - כלי ניתוח רשת שעזר לי לבדוק את התקשורת המוצפנת ולוודא שאכן כל המידע שעובר ברשת מוצפן כראוי.

- **בדיקות ידניות** - מעבר לבדיקות האוטומטיות, ביצעתי בדיקות ידניות מקיפות של כל פונקציונליות, כולל ניסיונות "לשבור" את המערכת על ידי קלט לא צפוי או שימוש בלתי רגיל.

6. תיאור פרוטוקול התקשורת

מערכת StegnoLogic משתמשת בפרוטוקול תקשורת מבוסס TCP, עם מבנה הודעות ייחודי שפיתחתי עבור המערכת. להלן תיאור מפורט של הפרוטוקול:

כללי:

- **פרוטוקול בסיסי**: TCP, שנבחר בשל האמינות שלו והבטחת העברת נתונים בסדר הנכון.
- **פורט ברירת מחדל**: 5555, ניתן לשינוי דרך קובץ התצורה.
- **הצפנה**: כל התקשורת מוצפנת עם RSA, כאשר כל צד מצפין עם המפתח הציבורי של הצד השני.

מבנה ההודעות:

כל ההודעות בפרוטוקול מבוססות על **JSON מוצפן**.

לאחר ההצפנה, ההודעה היא מחרוזת Base64 שמייצגת את הנתונים המוצפנים. אם ההודעה גדולה מדי להצפנה בבת אחת (מעל 245 בייט), היא מפוצלת למקטעים, כל מקטע מוצפן בנפרד, והתוצאות מחוברות עם המפריד "||CHUNK||".

תהליך החלפת מפתחות:

בעת יצירת חיבור חדש, מתבצע תהליך החלפת מפתחות כדלקמן:

1. השרת שולח את המפתח הציבורי שלו ללקוח בפורמט PEM (לא מוצפן, כמובן).
2. הלקוח מקבל את המפתח הציבורי של השרת, שומר אותו, ושולח את המפתח הציבורי שלו לשרת (גם כן בפורמט PEM).
3. בשלב זה, כל צד יכול להצפין הודעות עם המפתח הציבורי של הצד השני, ולפענח הודעות נכנסות עם המפתח הפרטי שלו.

סוגי הודעות ומבנה:

1. בקשת התחברות (login):

שולח: לקוח

מקבל: שרת

מבנה:

```
{
    "type": "login",
    "data": {
        "username": "<שם משתמש>",
        "password": "<סיסמה>"
    }
}
```

תשובה מהשרת:

```
{
    "status": "success" | "error",
    "user": [<פרטי משתמש>] | null,
    "verification_code": "<קוד אימות>" | null,
    "message": "<הודעת שגיאה>" | null
}
```

2. בקשת רישום (register):

שולח: לקוח

מקבל: שרת

מבנה:

```
{
    "type": "register",
    "data": {
        "username": "<שם משתמש>",
        "name": "<שם מלא>",
        "password": "<סיסמה>",
        "email": "<כתובת אימייל>"
    }
}
```

תשובה מהשרת:

```
{
    "status": "success" | "error",
    "message": "<הודעת הצלחה או שגיאה>"
}
```

3. בקשת הסתרת מידע (encode_image_multiple):

שולח: לקוח

מקבל: שרת

מבנה:

```
{
    "type": "encode_image_multiple",
    "data": {
        "image_data": "<נתוני התמונה בקידוד Base64>",
        "message": "<ההודעה להסתרה>",
        "receivers": ["<שם משתמש 2>" ,"<שם משתמש 1>", ...],
        "sender": "<שם משתמש השולח>"
    }
}
```

תשובה מהשרת:

```
{
    "status": "success" | "error",
    "encoded_image": "<התמונה המקודדת בקידוד Base64>" | null,
    "image_id": <מזהה התמונה במסד הנתונים> | null,
    "message": "<הודעת שגיאה>" | null
}
```

4. בקשת פענוח (decode_image):

שולח: לקוח

מקבל: שרת

מבנה:

```
{
    "type": "decode_image",
    "data": {
        "image_data": "<נתוני התמונה בקידוד Base64>",
        "username": "<שם המשתמש הנוכחי>"
    }
}
```

תשובה מהשרת:

```
{
    "status": "success" | "error",
    "message": "<הודעת שגיאה>" | "<ההודעה המפוענחת>"
}
```

5. בקשת שליחת אימייל (send_emails_multiple):

שולח: לקוח

מקבל: שרת

מבנה:

```
{
    "type": "send_emails_multiple",
    "data": {
        "recipients": {"<אימייל 2>" :"<שם משתמש 2>" ,"<אימייל 1>" :"<שם משתמש 1>", ...},
        "subject": "<נושא האימייל>",
        "body": "<גוף האימייל>",
        "attachment_data": "<נתוני הקובץ המצורף בקידוד Base64>" | null,
        "attachment_name": "<שם הקובץ המצורף>" | null
    }
}
```

תשובה מהשרת:

```
{
    "status": "success" | "error",
    "results": {"<שם משתמש 1>": true|false, "<שם משתמש 2>": true|false, ...} | null,
    "message": "<הודעת שגיאה>" | null
}
```

שיקולי אבטחה בפרוטוקול:

- **כל התעבורה מוצפנת**: אפילו אם התוקף מיירט את התקשורת, הוא יראה רק נתונים מוצפנים.

- **מפתחות חד-פעמיים**: זוג המפתחות נוצר בכל התחברות מחדש, מה שמקטין את החלון להתקפות.

- **אימות הדדי**: השרת יכול לאמת את הלקוח (דרך שם משתמש וסיסמה), והלקוח יכול לאמת את השרת (על ידי בדיקה שהתשובות ניתנות לפענוח עם המפתח הפרטי שלו).- *מניעת replay attacks*: מכיוון שהמפתחות מתחלפים בכל סשן, התקפות "שידור חוזר" של הודעות ישנות לא יעבדו.

הפרוטוקול שפיתחתי מאזן בין פשטות לבין אבטחה, ומספק תשתית חזקה לתקשורת מאובטחת בין הלקוח לשרת. העובדה שהפרוטוקול משתמש ב-JSON כפורמט הבסיסי מקלה על הבנת המבנה ועל היישום, בעוד ששימוש ב-RSA להצפנה מספק שכבת אבטחה חזקה.

פרוטוקול SMTP בפרויקט

SMTP (Simple Mail Transfer Protocol) הוא פרוטוקול תקשורת סטנדרטי המשמש להעברת דואר אלקטרוני ברשת האינטרנט.

השימושים העיקריים:

1. **אימות דו-שלבי (FA2)** - כאשר משתמש מתחבר למערכת, המערכת שולחת קוד אימות לכתובת המייל שלו באמצעות SMTP. זה מוסיף שכבת אבטחה נוספת לתהליך ההתחברות.

2. **שליחת תמונות מוצפנות** - לאחר שהמערכת מצפינה הודעה סודית בתוך תמונה, היא שולחת את התמונה לנמענים שנבחרו דרך המייל באמצעות SMTP.

המימוש הטכני:

- השתמשתי בספריית `smtplib` של Python לתקשורת עם שרת SMTP

- המערכת מתחברת לשרת Gmail (smtp.gmail.com) בפורט 587

- השימוש ב-TLS מבטיח שהתקשורת מוצפנת ומאובטחת

היתרונות:

- פרוטוקול מוכר ואמין

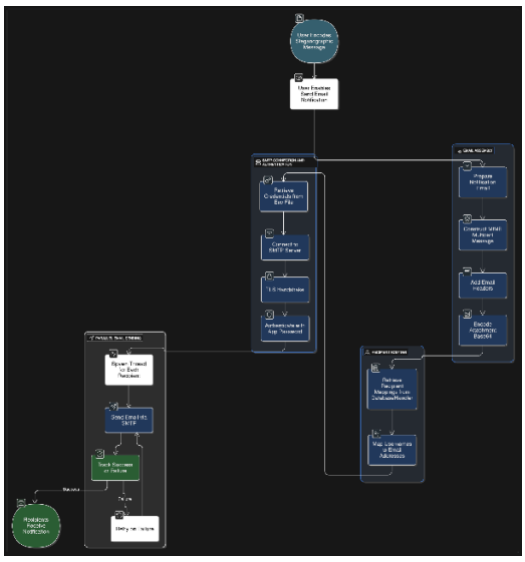
- תמיכה רחבה בכל תוכנות המייל

- אפשרות לשלוח קבצים מצורפים (התמונות המוצפנות)

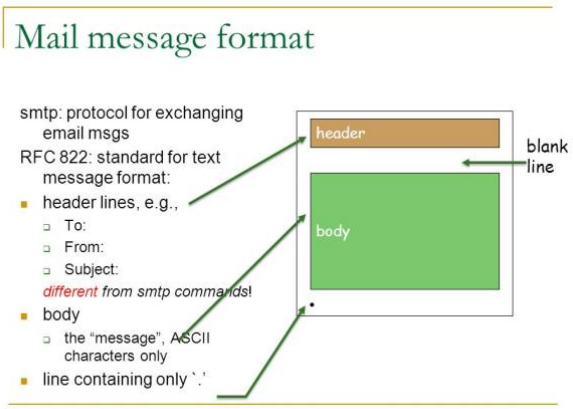
- חיבור מאובטח באמצעות TLS

מבנה הודעה בפרוטוקול SMTP: הודעת SMTP מורכבת משלושה חלקים עיקריים:
1. **Envelope (מעטפת):** כולל את פקודות השליחה: `EHLO` / `HELO` – זיהוי השרת השולח. `MAIL FROM`: – כתובת השולח. `RCPT TO`: – כתובת הנמען. `DATA` – התחלת גוף ההודעה.
2. **Header (כותרת):** מידע על ההודעה: `From`: – כתובת השולח. `To`: – כתובת הנמען. `Subject`: – נושא ההודעה. `Date`: – תאריך השליחה.
3. **Body (גוף ההודעה)**: תוכן ההודעה (הטקסט שנשלח). בסיום ההודעה נשלחת נקודה (`.`) בשורה נפרדת לציון סוף ההודעה.

תרשים זרימה של שליחת הודעה



מבנה הודעה בSMTP



מסכי המערכת (ממשק המשתמש)

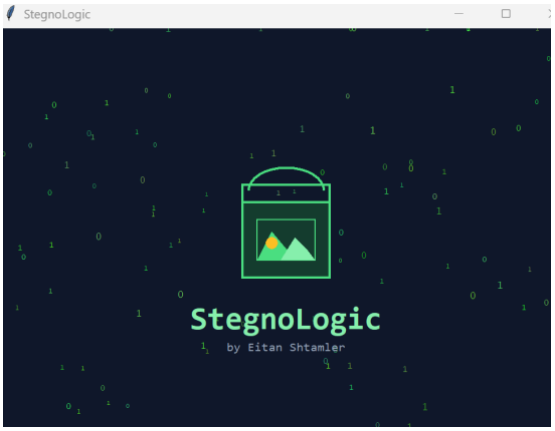
מערכת StegnoLogic כוללת מספר מסכים שתוכננו לספק חוויית משתמש נוחה ואינטואיטיבית, תוך שמירה על עקרונות האבטחה. להלן פירוט המסכים העיקריים במערכת:

1. מסך אנימציית פתיחה

מטרה: להציג אנימציה אסתטית בעת הפעלת האפליקציה, ובמקביל לאתחל את חיבור השרת והמודולים הנדרשים ברקע.

תצוגה: האנימציה מציגה אלמנטים בסגנון סייבר-אבטחה, כולל קוד בינארי זורם, מנעול דיגיטלי שמתגבש בהדרגה, וסמל המערכת שמופיע בהדרגה. יש גם סרגל התקדמות שמתמלא תוך כדי אתחול המערכת.

פעולות אפשריות: אין אינטראקציה משתמש במסך זה. האנימציה מסתיימת אוטומטית ועוברת למסך הבא לאחר שהאתחול הושלם.



2. מסך בחירה בין התחברות לרישום

מטרה: לאפשר למשתמש לבחור אם להתחבר עם חשבון קיים או ליצור חשבון חדש.

תצוגה: המסך מציג את לוגו המערכת, כותרת ראשית "Welcome to StegnoLogic", כותרת משנה "Secure Steganography", ושני כפתורים גדולים - "Login" ו-"Register".

פעולות אפשריות:

- לחיצה על "Login" מעבירה למסך ההתחברות.
- לחיצה על "Register" מעבירה למסך הרישום.
- לחיצה על "Exit" בפינה העליונה סוגרת את האפליקציה.

3. מסך רישום

מטרה : לאפשר למשתמש חדש ליצור חשבון במערכת.

תצוגה : המסך מציג טופס רישום עם שדות הבאים :

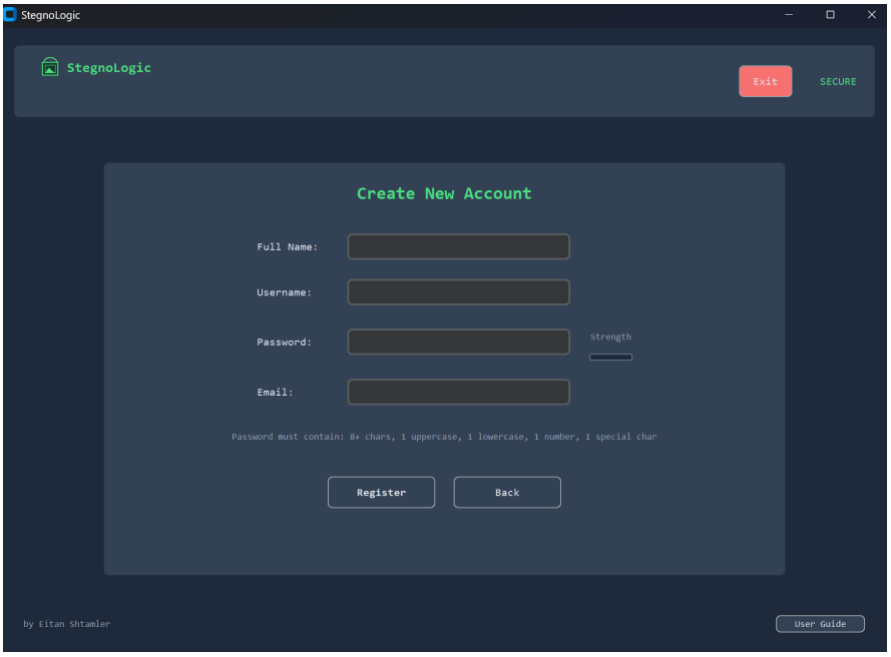
- Full Name (שם מלא)
- Username (שם משתמש)
- Password (סיסמה) - עם מד חוזק סיסמה שמשנה צבע (אדום, כתום, ירוק) לפי עוצמת הסיסמה
- Email (כתובת אימייל)

בנוסף, מוצגות דרישות הסיסמה (8+ תווים, אות גדולה, אות קטנה, ספרה, ותו מיוחד).

פעולות אפשריות :

- הזנת פרטים בשדות הטופס.

- לחיצה על "Register" לשליחת הפרטים לשרת ויצירת חשבון.
- לחיצה על "Back" לחזרה למסך הבחירה.



4. מסך התחברות

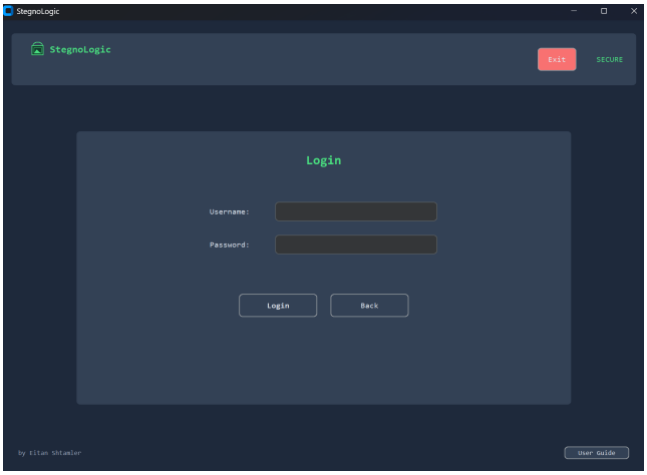
מטרה : לאפשר למשתמש רשום להתחבר למערכת.

תצוגה : המסך מציג טופס התחברות עם שדות :

- Username (שם משתמש)
- Password (סיסמה)

פעולות אפשריות:

- הזנת פרטי ההתחברות.
- לחיצה על "Login" לשליחת הפרטים לשרת ולהתחלת תהליך האימות.
- לחיצה על "Back" לחזרה למסך הבחירה.



5. מסך אימות דו-שלבי

מטרה : לאמת את זהות המשתמש באמצעות קוד אימות שנשלח לאימייל.

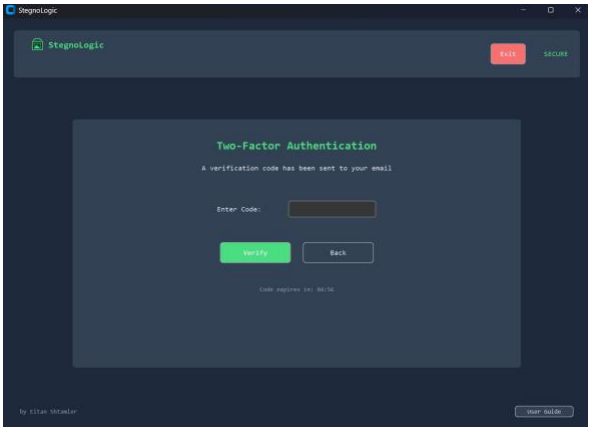
תצוגה : המסך מציג הודעה שקוד אימות נשלח לאימייל הרשום, שדה להזנת הקוד, ושעון עצר ויזואלי שמראה כמה זמן נותר עד לפקיעת תוקף הקוד (5 דקות).

פעולות אפשריות:

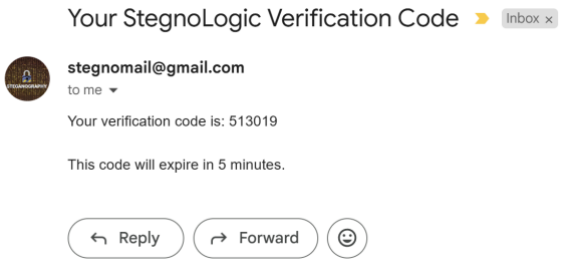
- הזנת קוד האימות.

- לחיצה על "Verify" לאימות הקוד.

- לחיצה על "Back" לחזרה למסך ההתחברות.



והודעת המייל תראה כך:



6. **מסך ראשי** - לשונית הסתרת מידע (Encode)

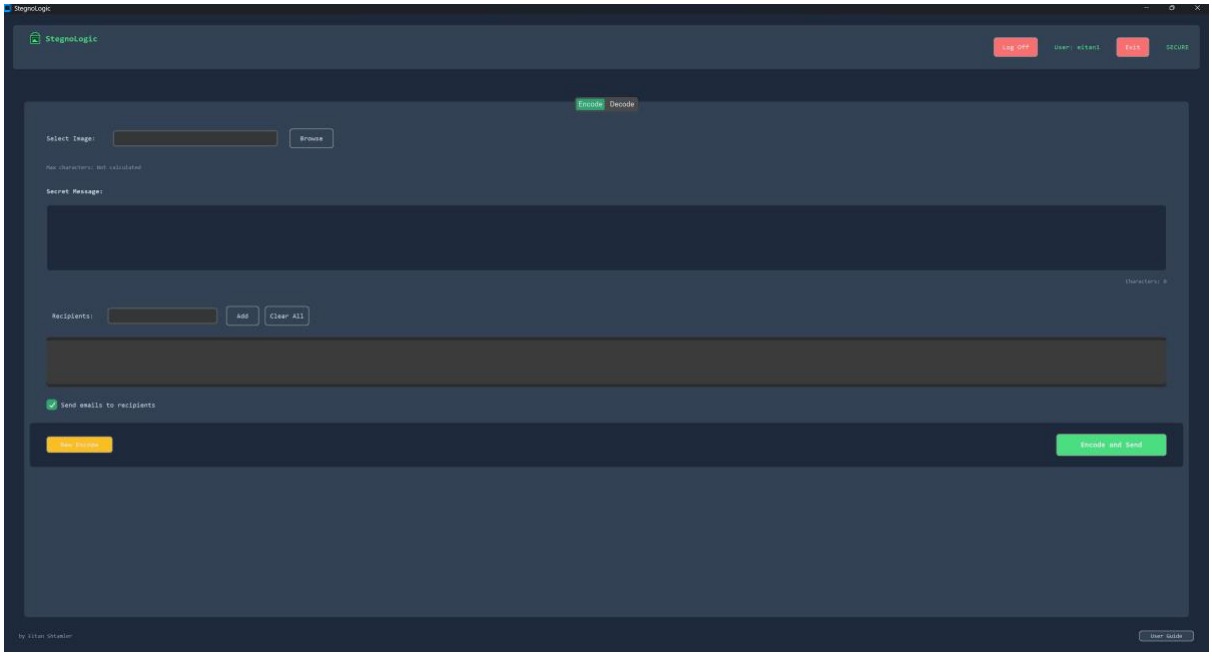
מטרה: לאפשר למשתמש להסתיר הודעה בתמונה ולשלוח אותה לנמענים.

תצוגה: המסך מאורגן בלשוניות, עם "Encode" נבחר. הוא כולל:

- שדה לבחירת תמונה, עם כפתור "Browse"
- מד קיבולת מקסימלית של התמונה (בתווים)
- אזור טקסט להזנת ההודעה הסודית, עם מונה תווים
- שדה להוספת נמענים, עם כפתורי "Add" ו-"Clear All"
- אזור תצוגה לרשימת הנמענים שנוספו
- תיבת סימון "Send emails to recipients"
- כפתור "Encode and Send" לביצוע ההסתרה והשליחה
- כפתור "New Encode" לניקוי הטופס והתחלה מחדש

פעולות אפשריות:

- בחירת תמונה באמצעות כפתור "Browse".
- הזנת הודעה סודית באזור הטקסט.
- הוספת נמענים והסרתם.
- הפעלה או ביטול של שליחת אימיילים.
- ביצוע הסתרת המידע ושליחתו.
- ניקוי הטופס להתחלה מחדש.
- מעבר ללשונית "Decode".



7. מסך ראשי - לשונית פענוח מידע (Decode)

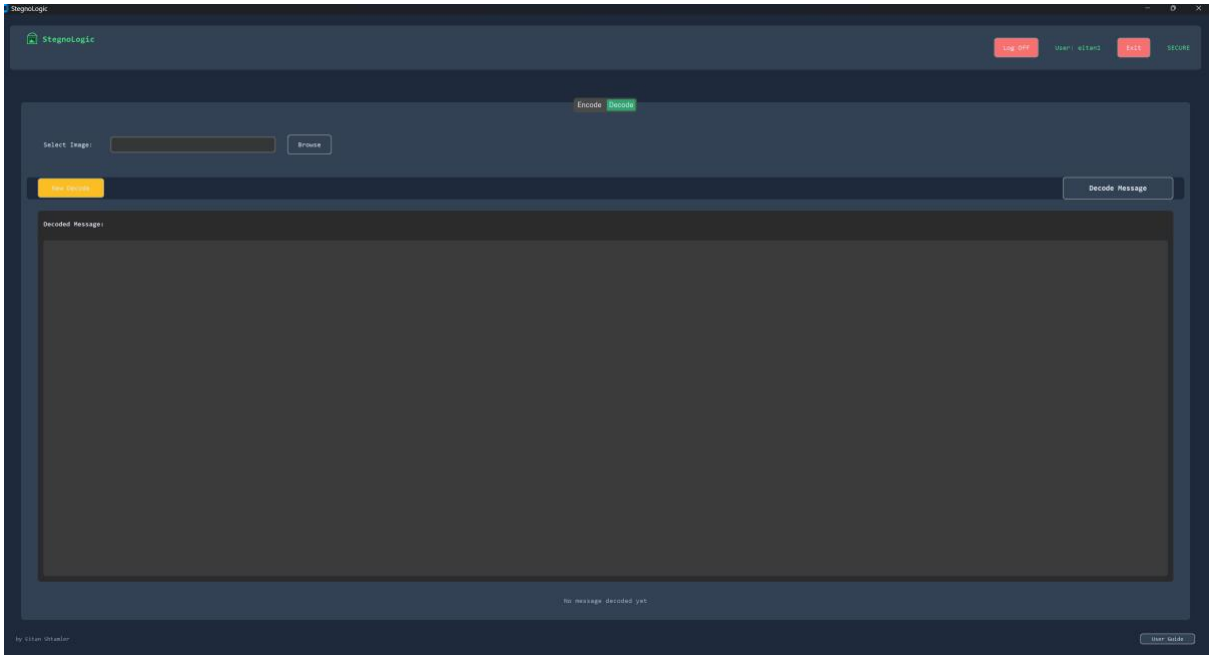
מטרה : לאפשר למשתמש לפענח הודעות מוסתרות מתמונות.

תצוגה : המסך כולל :

- שדה לבחירת תמונה, עם כפתור "Browse"
- אזור טקסט להצגת ההודעה המפוענחת
- כפתור "Decode Message" לפענוח ההודעה
- כפתור "New Decode" לניקוי הטופס
- תווית סטטוס שמציגה את מצב הפענוח

פעולות אפשריות :

- בחירת תמונה באמצעות כפתור "Browse".
- ביצוע פענוח באמצעות כפתור "Decode Message".
- ניקוי הטופס להתחלה מחדש.
- מעבר ללשונית "Encode".



8. חלון תוצאות הסתרת מידע

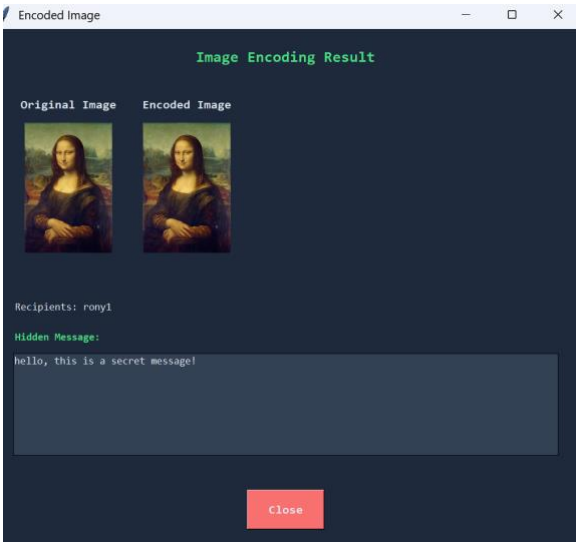
מטרה : להציג למשתמש את תוצאות תהליך ההסתרה, כולל השוואה בין התמונה המקורית והמקודדת.

תצוגה : החלון מציג:

- כותרת "Image Encoding Result"
- התמונה המקורית והתמונה המקודדת זו לצד זו
- מידע על הנמענים ("Recipients: ...")
- ההודעה שהוסתרה
- כפתור "Close" לסגירת החלון

פעולות אפשריות:

- צפייה בתמונות המקורית והמקודדת.
- קריאת המידע על ההודעה והנמענים.
- סגירת החלון באמצעות כפתור "Close".



9. חלון תוצאות פענוח מידע

מטרה: להציג למשתמש את ההודעה שפוענחה מהתמונה.

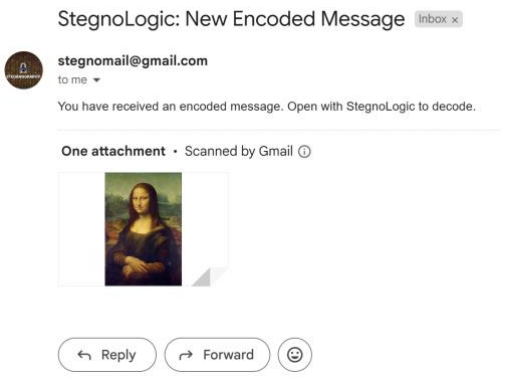
תצוגה: החלון מציג:

- כותרת "Decoded Message"
- התמונה המקודדת
- ההודעה המפוענחת בתיבת טקסט
- כפתור "Close" לסגירת החלון

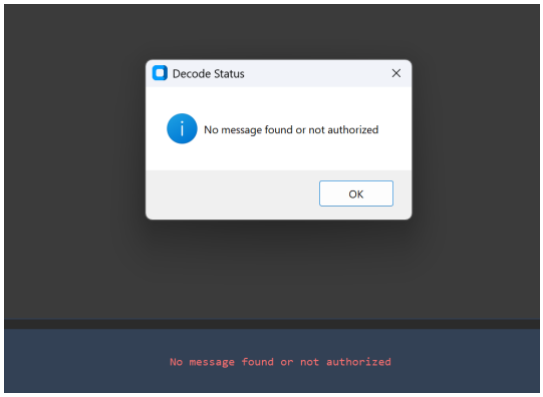
פעולות אפשריות:

- צפייה בתמונה המקודדת.
- קריאת ההודעה המפוענחת.
- סגירת החלון באמצעות כפתור "Close".

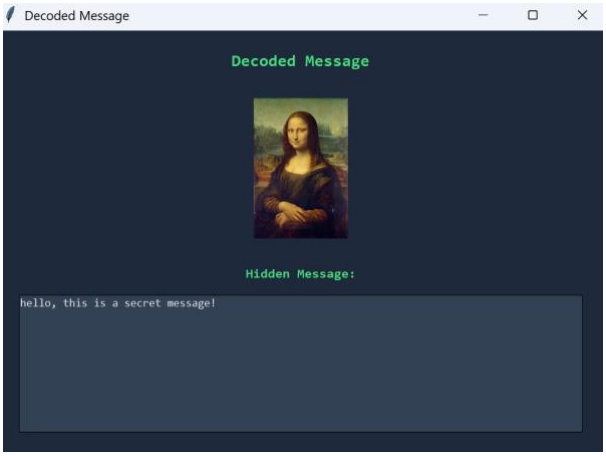
כך נראת ההודעה במייל:



זו ההודעה שמופיעה כאשר משתמש אשר התמונה לא מיועדת לו מנסה לפענח את התמונה:



וכך היא נראת לאחר הפיענוח:



10. חלון מדריך למשתמש

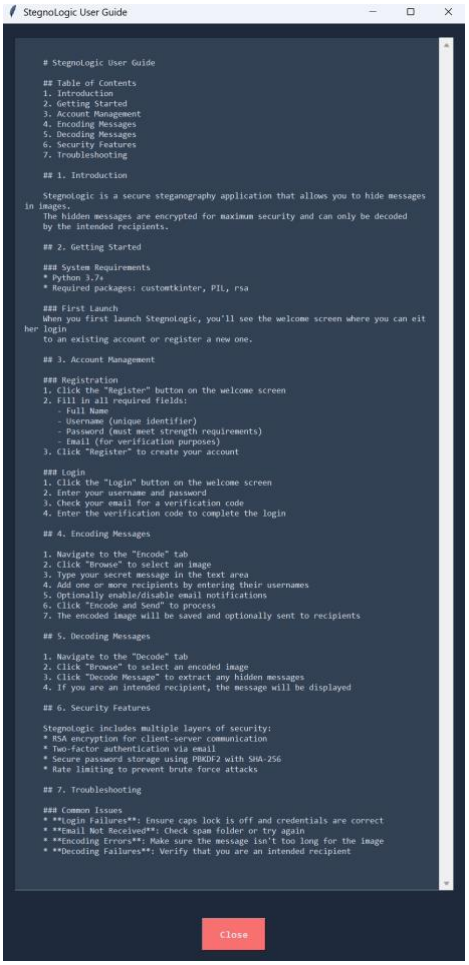
מטרה: לספק למשתמש הסברים מפורטים על השימוש במערכת.

תצוגה: החלון מציג אזור טקסט גליל עם מדריך מפורט, הכולל:

- הקדמה על המערכת
- הסברים על תהליכי הרישום וההתחברות
- הסברים על הסתרת מידע ופענוחו
- תיאור של מאפייני האבטחה
- טיפים לשימוש יעיל
- פתרון בעיות נפוצות

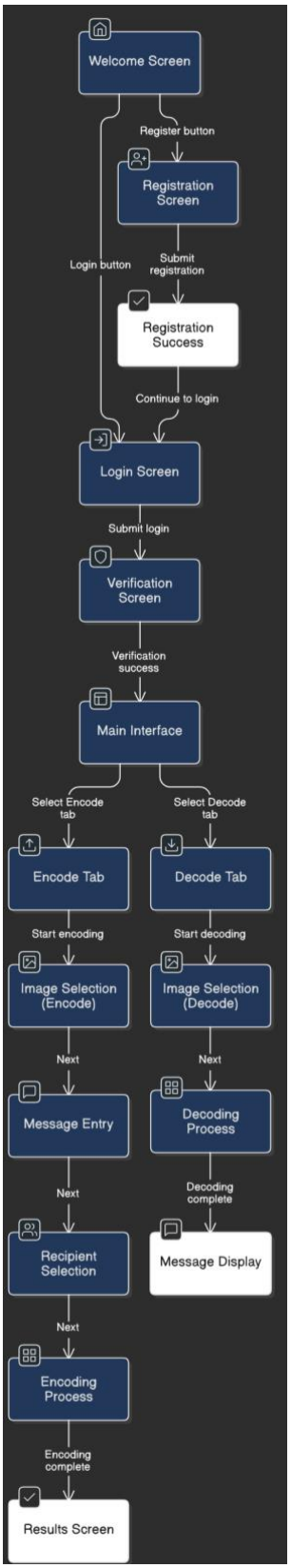
פעולות אפשריות:

- קריאת המדריך וגלילה בו.
- סגירת החלון באמצעות כפתור "Close".

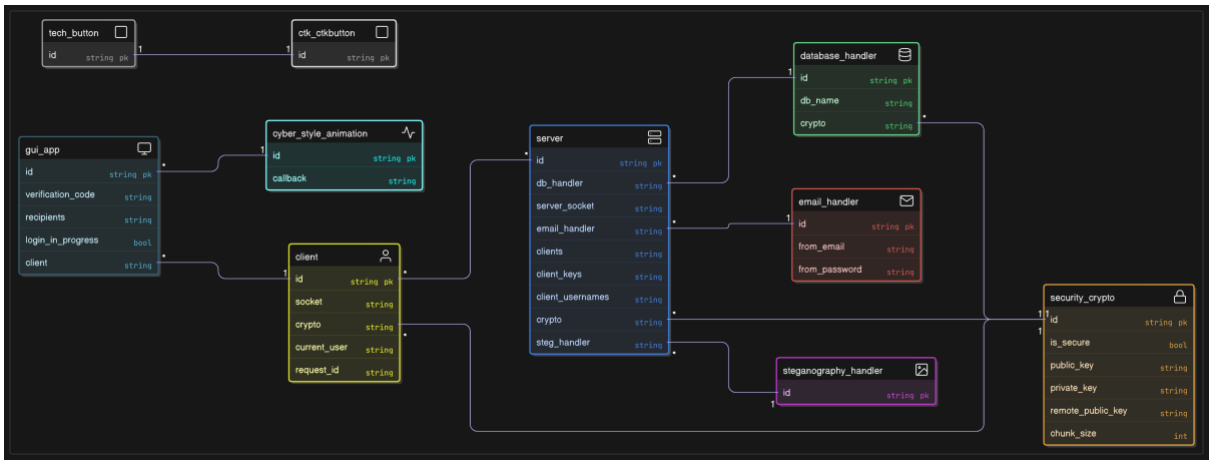


היררכיית מסכים (זרימת ניווט)

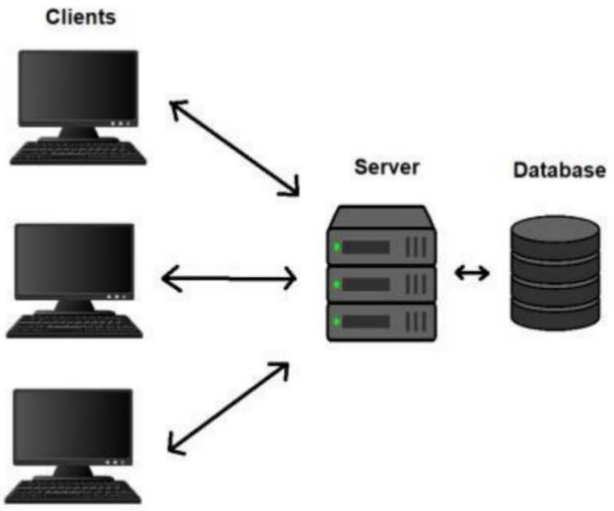
להלן תיאור של זרימת הניווט בין המסכים השונים במערכת



UML הכולל קשרים בין המחלקות



הדרך שבה התקשורת פועלת:



מבני נתונים

מערכת StegnoLogic משתמשת במגוון מבני נתונים, הן ברמת המשתנים והמחלקות בקוד והן ברמת מסד הנתונים. הבחירה במבנים אלו נעשתה תוך חשיבה על יעילות, בטיחות ותחזוקה פשוטה. להלן פירוט מבני הנתונים העיקריים:

קבצים מקומיים:

- **encoded_image.png** - התמונה המקודדת האחרונה שנוצרה נשמרת בספריית העבודה של האפליקציה.

- **.env** - קובץ תצורה המכיל הגדרות רגישות כמו פרטי התחברות לאימייל וכתובת שרת.

- **StegnoLogicDatabase.db** - קובץ מסד הנתונים SQLite שמכיל את כל הטבלאות של המערכת.

משתנים ומבני נתונים בולטים בקוד:

- **מפתחות RSA** - זוגות מפתחות (פרטי/ציבורי) לכל חיבור, שנשמרים בזיכרון בלבד ולא נשמרים לטווח ארוך.

- **רשימת נמענים** - מערך של שמות משתמשים שנבחרו כנמענים להודעה.

- **היסטוריית בקשות** - מערך של חותמות זמן המשמש למנגנון הגבלת הקצב (Rate Limiting).

- **מיפוי חיבורים לקוחות** - מילון (dict) שממפה אובייקטי socket לפרטי הלקוח המתאימים.

מבנה מסד הנתונים:

מסד הנתונים של המערכת מיושם באמצעות SQLite, שהוא מסד נתונים קל-משקל, מהיר ומאובטח. להלן פירוט הטבלאות:

טבלת users

טבלה זו מכילה את פרטי המשתמשים הרשומים במערכת.

שם שדה	סוג	אורך	דוגמה	הערות
username	TEXT	-	"john_doe"	מפתח ראשי, "יחודי
name	TEXT	-	"John Doe"	שם מלא של המשתמש
password	TEXT	-	"RHlUNzZLcmJrWENNVFRuQWw..."	"גיבוב מוצפן של הסיסמה עם "מלח
email	TEXT	-	"john@example.com"	כתובת אימייל תקפה

טבלת messages

טבלה זו מכילה את ההודעות שהוסתרו בתמונות, עם קישור לשולח ולנמען.

שם שדה	סוג	אורך	דוגמה	הערות
id	INTEGER	-	42	מפתח ראשי, אוטו-אינקרמנט
sender_username	TEXT	-	"alice"	מפתח זר לטבלת users
receiver_username	TEXT	-	"bob"	מפתח זר לטבלת users
timestamp	TEXT	-	"2023-04-15 14:32:45"	זמן שליחת ההודעה בפורמט ISO
hidden_message	TEXT	-	"This is a secret message"	ההודעה המוסתרת

טבלת logs

טבלה זו מכילה לוגים של ניסיונות התחברות ופעולות חשובות אחרות.

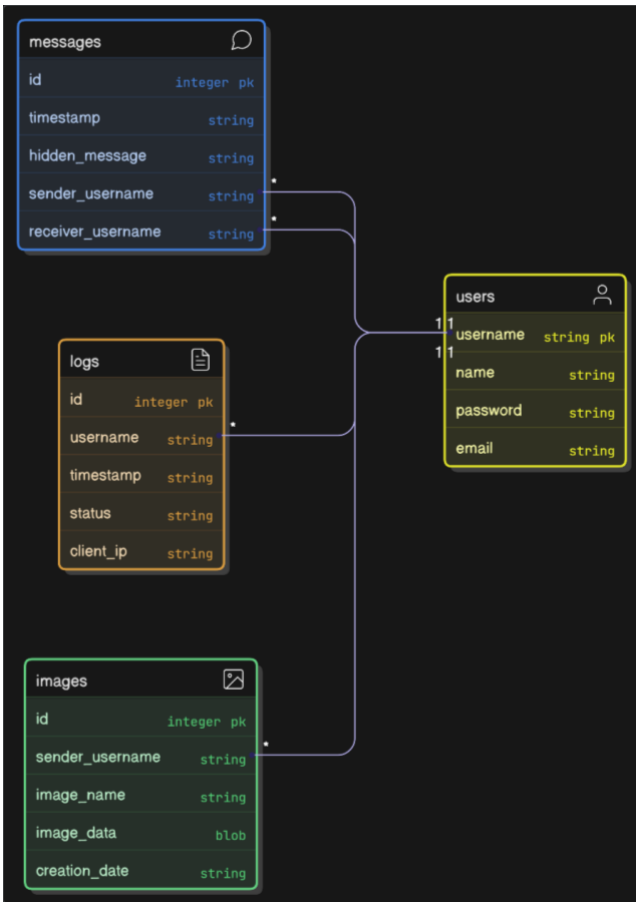
שם שדה	סוג	אורך	דוגמה	הערות
id	INTEGER	-	157	מפתח ראשי, אוטו-אינקרמנט
username	TEXT	-	"charlie"	מפתח זר לטבלת users
timestamp	TEXT	-	"2023-04-15 14:35:12"	זמן הפעולה בפורמט ISO
status	TEXT	-	"Success" או "Failed"	סטטוס הפעולה
client_ip	TEXT	-	"192.168.1.5"	כתובת IP של הלקוח

טבלת images

טבלה זו מכילה את התמונות המקודדות שנשמרו במערכת.

שם שדה	סוג	אורך	דוגמה	הערות
id	INTEGER	-	89	מפתח ראשי, אוטו-אינקרמנט
image_name	TEXT	-	"encoded_image_1681557165.png"	שם הקובץ
image_data	BLOB	-	[נתונים בינאריים]	נתוני התמונה עצמם
sender_username	TEXT	-	"alice"	מפתח זר לטבלת users
creation_date	TEXT	-	"2023-04-15 14:32:45"	זמן יצירת התמונה בפורמט ISO

יחסים בין הטבלאות ERD:



שיקולים בתכנון מבני הנתונים:

בתכנון מבני הנתונים עבור המערכת, לקחתי בחשבון מספר שיקולים חשובים:

1. **אבטחה** - סיסמאות נשמרות כגיבוב עם "מלח", ולא כטקסט גלוי. נתונים רגישים כמו פרטי התחברות לאימייל נשמרים בקובץ .env נפרד ולא במסד הנתונים.

2. **יעילות אחזור** - הוספתי מפתחות זרים (foreign keys) ואינדקסים לשדות שמשמשים לחיפוש תכוף, כדי לשפר את ביצועי השאילתות.

3. **שלמות נתונים** - הגדרתי אילוצי מפתחות זרים שמבטיחים שלא ניתן למחוק משתמש אם יש לו עדיין הודעות או תמונות במערכת.

4. **גמישות** - השתמשתי בסוגי נתונים גנריים יחסית (TEXT במקום VARCHAR עם אורך קבוע) כדי לאפשר גמישות, מכיוון שSQLite בכל מקרה ממיר סוגים לטיפוסי הליבה שלו.

5. **יעילות אחסון** - השתמשתי בסוג BLOB לאחסון נתוני תמונה בינאריים, שהוא היעיל ביותר לאחסון מידע בינארי גדול.

מבני הנתונים שתוארו לעיל מהווים את התשתית של המערכת, ומאפשרים את כל הפונקציונליות שלה - מרישום והתחברות משתמשים, דרך הסתרת מידע בתמונות ושליחתו לנמענים, ועד לוגים ומעקב אחרי פעילות במערכת. התכנון הקפדני של מבנים אלו תורם ליציבות, אבטחה וביצועים טובים של המערכת.

איומי מערכת וחולשות - סקירת אבטחה

במערכת StegnoLogic, כמו בכל מערכת מחשוב, קיימים איומים וחולשות פוטנציאליים שיש להתמודד איתם. להלן סקירה מקיפה של איומים אלו ושל הפתרונות שיישמתי, מסודרים לפי שכבות השיטה.

שכבת האפליקציה:

SQL Injection:

האיום: תוקף עלול לנסות להזריק קוד SQL זדוני דרך שדות קלט, כדי לגרום לשרת לבצע פעולות לא מורשות במסד הנתונים.

הפתרון שיישמתי:

- השתמשתי בשאילתות מוכנות (prepared statements) בכל מקום שבו יש אינטראקציה עם מסד הנתונים. לדוגמה:

python

```
c.execute("INSERT INTO users (username, name, password, email) VALUES (?, ?, ?, ?)",
(username, name, hashed_password, email))
```

- נתונים שמגיעים מהמשתמש (כמו שם משתמש, סיסמה, וכו') לעולם לא משולבים ישירות בשאילתה, אלא תמיד עוברים דרך מנגנון הפרמטרים של SQLite.

- בדיקות קלט קפדניות בצד הלקוח לפני שליחה לשרת, ובדיקות נוספות בצד השרת.

אבטחת התחברות:

האיום: תוקף עלול לנסות לנחש סיסמאות או להשתמש במידע שנחשף בפריצות קודמות כדי לגשת לחשבונות משתמשים.

הפתרון שיישמתי:

- **אימות דו-שלבי** : לאחר הזנת שם משתמש וסיסמה נכונים, נשלח קוד אימות לאימייל המשתמש. הקוד תקף ל-5 דקות בלבד.

- **מדיניות סיסמאות חזקה** : סיסמאות חייבות להכיל לפחות 8 תווים, אות גדולה, אות קטנה, ספרה ותו מיוחד.

- **אחסון סיסמאות מאובטח** : סיסמאות נשמרות כגיבוב עם "מלח" אקראי וייחודי לכל משתמש, באמצעות PBKDF2 עם 100,000 איטרציות.

- **הגבלת ניסיונות התחברות** : מנגנון Rate Limiting מגביל את מספר ניסיונות ההתחברות מכל כתובת IP.

- **תיעוד ניסיונות התחברות** : כל ניסיון התחברות (מוצלח או כושל) נרשם ביומן, עם שם המשתמש, כתובת IP, וזמן.

הצפנה:

האיום : תוקף עלול לנסות לקרוא או לשנות את המידע בזמן שהוא עובר ברשת, או לחלץ את המידע המוסתר מהתמונות.

הפתרון שיישמתי:

- **הצפנת RSA לתקשורת** : כל התקשורת בין הלקוח לשרת מוצפנת עם RSA 2048 סיביות, עם זוג מפתחות חדש לכל חיבור.

- **סטגנוגרפיה מאובטחת** : המידע מוסתר בביטים הפחות משמעותיים של הפיקסלים, עם שינויים מזעריים שאינם נראים לעין.

- **בדיקת הרשאות קפדנית** : רק הנמענים המיועדים יכולים לחלץ את המידע מהתמונות, בזכות מנגנון ייחודי שמשלב את רשימת הנמענים בתוך ההודעה המוסתרת.

- **מידור משתמשים** : המערכת מבטיחה שמשתמש אחד לא יכול לראות את המידע שנשלח למשתמש אחר.

DDOS / DOS:

האיום : תוקף עלול לנסות להציף את השרת בבקשות כדי להקריס אותו או לגרום לו להיות בלתי זמין.

הפתרון שיישמתי:

- הגבלת קצב בקשות : יישמתי מנגנון Rate Limiting שמגביל את מספר הבקשות מכל כתובת IP ל-180 בדקה.

- **ניקוי משאבים** : השרת מנקה משאבים באופן מסודר לאחר כל חיבור, כדי למנוע "דליפות משאבים".

- **טיימאאוט לחיבורים** : חיבורים ללא פעילות נסגרים אוטומטית לאחר זמן מסוים.

- **ניטור חיבורים** : השרת שומר מעקב אחר כל החיבורים הפעילים, ויכול לזהות דפוסים חשודים.

העלאת קבצים:

האיום : תוקף עלול לנסות להעלות קבצים זדוניים, או לגרום לשרת לטפל בקבצים גדולים מדי שיגרמו לבעיות ביצועים.

הפתרון שיישמתי :

- **בדיקות סוג קובץ** : המערכת בודקת שרק קבצי תמונה נתמכים (PNG, JPEG, BMP, GIF) מועלים לשרת.

- **הגבלת גודל קובץ** : יש מגבלה על גודל הקבצים שניתן להעלות.

- **בדיקת תקינות תמונה** : לפני הסתרת מידע, המערכת בודקת שהקובץ הוא אכן תמונה תקפה שניתן לעבד.

- **שמירת התמונות בצורה בטוחה** : התמונות נשמרות במסד הנתונים כ-BLOB, ולא כקבצים במערכת הקבצים, מה שמונע התקפות על מערכת הקבצים.

שכבת התעבורה:

TCP ו-3-way handshake :

האיום : תוקף עלול לנסות לזייף חיבורים או לבצע התקפות כמו TCP SYN flooding.

הפתרון שיישמתי:

- **הסתמכות על 3-way handshake** : המערכת מסתמכת על מנגנון ה-3-way handshake של TCP להקמת חיבורים אמינים.

- **אימות נוסף** : מעבר ל-3-way handshake של TCP, המערכת מבצעת החלפת מפתחות RSA ואימות ברמת האפליקציה.

- **טיימאאוט לחיבורים** : חיבורים שלא משלימים את תהליך ההתחברות תוך זמן סביר נסגרים.

- **הגבלת חיבורים במקביל** : יש מגבלה על מספר החיבורים שיכולים להיות פתוחים במקביל מאותה כתובת IP.

חולשות כלליות:

Cross-Site Scripting (XSS) :

האיום : למרות שהמערכת אינה מבוססת ווב, עדיין יש סיכון של הזרקת קוד זדוני דרך שדות קלט.

הפתרון שיישמתי:

- **סניטציה של קלט** : כל קלט משתמש עובר סניטציה לפני שימוש.

- **הגבלת קלט** : שדות הקלט מוגבלים לתווים ספציפיים (למשל, שם משתמש יכול להכיל רק אותיות, ספרות וקו תחתון).

- **עיצוב מונע הזרקה** : הממשק הגרפי מתוכנן כך שקלט משתמש לא יכול להשפיע על מבנה הממשק.

שמירת מידע רגיש:

האיום : מידע רגיש כמו פרטי התחברות לאימייל או מפתחות פרטיים עלול להיחשף.

הפתרון שיישמתי:

- **משתנים סביבתיים** : פרטי התחברות רגישים נשמרים בקובץ .env שלא נכלל בקוד המקור.

- **מפתחות אפמריים** : מפתחות RSA נוצרים מחדש בכל חיבור ולא נשמרים לטווח ארוך.

- **אחסון מאובטח** : סיסמאות נשמרות כגיבוב בלבד, לעולם לא כטקסט גלוי.

- **ניקוי זיכרון** : מידע רגיש מנוקה מהזיכרון לאחר השימוש בו.

התקפות על הלקוח:

האיום : תוקף עלול לנסות להתקין גרסה מזויפת של הלקוח כדי לגנוב מידע.

הפתרון שיישמתי:

- **אימות שרת** : הלקוח מאמת את זהות השרת על ידי בדיקה שהתשובות ניתנות לפענוח עם המפתח הפרטי שלו.

- **הצפנה מקצה לקצה** : כל המידע הרגיש מוצפן מקצה לקצה, כך שגם אם התוקף משתלט על הלקוח, הוא לא יכול לגשת למידע של משתמשים אחרים.

- **אחסון מקומי מינימלי** : הלקוח שומר מידע מקומי מינימלי, כך שגם אם הוא נפרץ, הנזק מוגבל.

התקפות אחרות:

האיום : קיימים איומים רבים נוספים, כגון התקפות timing, התקפות על אלגוריתמי הצפנה, ועוד.

הפתרון שיישמתי :

- **שימוש בספריות מאובטחות ומתוחזקות** : השתמשתי בספריות פייתון מוכרות ומתוחזקות היטב לצורכי הצפנה ואבטחה.

- **עדכון קבוע** : המערכת מתוכננת כך שניתן יהיה לעדכן את הספריות ואת קוד האבטחה בקלות, כדי להתמודד עם איומים חדשים שמתגלים.

- **איטרציות רבות בגיבוב סיסמאות**: השימוש ב-100,000 איטרציות בגיבוב PBKDF2 מקשה מאוד על התקפות timing או התקפות צד-ערוץ אחרות.

- **מניעת דליפת מידע בהודעות שגיאה**: הודעות השגיאה מתוכננות כך שלא ידליפו מידע רגיש. למשל, בניסיון התחברות כושל, ההודעה היא "שם משתמש או סיסמה שגויים" ולא "שם המשתמש לא קיים" או "הסיסמה שגויה".

סיכום האבטחה:

המערכת שבניתי, StegnoLogic, מתמודדת עם איומי אבטחה ברבדים שונים, מהרמה הבסיסית של אבטחת התקשורת ועד לרמה הגבוהה של אבטחת סטגנוגרפיה. יישמתי שכבות הגנה מרובות (defense in depth) כדי להקשות על תוקפים פוטנציאליים:

1. **אבטחת תקשורת**: כל התקשורת בין הלקוח לשרת מוגנת באמצעות הצפנת RSA, עם זוג מפתחות חדש לכל חיבור.

2. **אבטחת משתמשים**: אימות דו-שלבי, סיסמאות חזקות, ואחסון סיסמאות מאובטח.

3. **אבטחת נתונים**: הגנה מפני SQL Injection, פרוטוקול תקשורת מאובטח, והצפנת נתונים רגישים.

4. **הגנה מפני DDOS**: מנגנון הגבלת קצב, ניקוי משאבים, וניטור חיבורים.

5. **אבטחת סטגנוגרפיה**: מנגנון ייחודי להסתרת מידע בתמונות, עם בדיקת הרשאות לנמענים.

אני מכיר בכך שאין מערכת בטוחה לחלוטין, ושכל מערכת יש לבחון ולשפר באופן קבוע. ואכן, במהלך הפיתוח גיליתי מספר נקודות שבהן ניתן לשפר את האבטחה בגרסאות עתידיות:

1. **שימוש ב-HTTPS**: למרות שהמערכת אינה מבוססת ווב, ניתן היה להוסיף שכבת TLS/SSL להגנה נוספת על התקשורת.

2. **שיפור ניהול מפתחות**: מנגנון משופר לניהול מפתחות הצפנה, אולי עם מסד נתונים ייעודי למפתחות.

3. **אנליזת לוגים**: מנגנון מתקדם יותר לניתוח לוגים וזיהוי פעילות חשודה.

יישום הפרויקט

חלק א - מודולים ומחלקות במערכת

פרויקט StegnoLogic מורכב ממספר מודולים ומחלקות שעובדים יחד ליצירת מערכת אינטגרטיבית ומאובטחת. בחלק זה אפרט את המודולים השונים, את תפקידם במערכת, ואת האופן שבו הם משתלבים יחד.

1. מודולים וספריות מיובאות

הפרויקט משתמש במגוון ספריות ומודולים חיצוניים, כל אחד נבחר בקפידה לתפקיד ספציפי:

- **rsa** - משמש לביצוע פעולות הצפנה אסימטרית, החיוניות לאבטחת התקשורת בין הלקוח לשרת.
- **base64** - מספק פונקציות לקידוד ופענוח נתונים בינאריים לפורמט טקסטואלי, שימושי מאוד להעברת תמונות וקבצים אחרים.
- **hashlib** - מספק פונקציות גיבוב (hashing) לאבטחת סיסמאות ולבדיקות שלמות נתונים.
- **os** - מאפשר אינטראקציה עם מערכת ההפעלה, במיוחד לקריאת משתני סביבה וטיפול בקבצים.
- **socket** - מספק את היכולת ליצור חיבורי רשת בין הלקוח לשרת.
- **threading** - מאפשר יצירת תהליכונים (threads) לביצוע משימות במקביל, חיוני לשרת שמטפל במספר לקוחות בו-זמנית.
- **json** - משמש להמרה בין אובייקטים בפייתון למחרוזות JSON ובחזרה, לצורך העברת נתונים מובנים.
- **time** - מספק פונקציות הקשורות לזמן, כמו השהיות וחותמות זמן.
- **PIL (Python Imaging Library)** - מספק יכולות לעיבוד תמונות, חיוני לפעולות סטגנוגרפיה.
- **sqlite3** - מאפשר עבודה עם מסד נתונים SQLite, המשמש לאחסון נתוני המשתמשים וההודעות.
- **re** - מספק תמיכה בביטויים רגולריים, המשמשים לבדיקת תקינות קלט (כמו סיסמאות).
- **io** - מספק כלים לטיפול בקלט/פלט של נתונים בינאריים וטקסטואליים.
- **smtplib** - מאפשר שליחת הודעות אימייל, משמש לאימות דו-שלבי ולהתראות.
- **email** - מספק כלים ליצירת הודעות אימייל מורכבות, כולל קבצים מצורפים.
- **ssl** - מספק יכולות אבטחה לתקשורת רשת, במיוחד לחיבור לשרתי SMTP.
- **dotenv** - מאפשר טעינה של משתני סביבה מקובץ .env, שימושי להגדרות רגישות כמו פרטי התחברות.
- **customtkinter** - הרחבה של Tkinter הסטנדרטי שמספקת רכיבי ממשק גרפי מודרניים.
- **tkinter** - ספריית הממשק הגרפי הסטנדרטית של פייתון, הבסיס לממשק המשתמש.
- **math** - מספק פונקציות מתמטיות בסיסיות, משמש בעיקר לאנימציות בממשק.
- **random** - מספק פונקציות ליצירת מספרים אקראיים, שימושי ליצירת קודי אימות ועוד.

2. מודולים ומחלקות מוגדרות על ידי המשתמש

SecurityCrypto

תפקיד : מחלקה זו אחראית על כל היבטי האבטחה וההצפנה במערכת, החל מיצירת זוגות מפתחות RSA, דרך הצפנה ופענוח של מידע, ועד לטיפול באבטחת סיסמאות.

מאפיינים :

- **is_secure** - משמש לציון אם החיבור הנוכחי מאובטח.

- **public_key** - המפתח הציבורי של האובייקט הנוכחי, משמש להצפנה על ידי הצד השני.

- **private_key** - המפתח הפרטי של האובייקט הנוכחי, משמש לפענוח הודעות מוצפנות.

- **remote_public_key** - המפתח הציבורי של הצד השני, משמש להצפנת הודעות אליו.

מתודות - פעולות:

- **secure_connection()-** קלט : אין, פלט : מצב החיבור. מגדירה את מצב החיבור כמאובטח.

- **check_security()-** קלט : אין, פלט : מצב החיבור. מחזירה את מצב החיבור הנוכחי.

- **limit_connection_rate(history, max_per_minute)** - קלט : היסטוריית חיבורים, מספר מקסימלי של חיבורים לדקה. פלט : האם החיבור הנוכחי מותר. מנגנון הגנה מפני DDOS שמגביל את מספר החיבורים מכל מקור.

- **check_password_strength(password)** - קלט : סיסמה. פלט : תוצאת בדיקה והודעה. בודקת אם הסיסמה עומדת בדרישות חוזק (אורך, תווים מיוחדים וכו').

- **generate_keys(key_size)** - קלט : גודל מפתח. פלט : המפתח הציבורי. מייצרת זוג מפתחות RSA חדש.

- **set_remote_public_key(key_data)** - קלט : מפתח ציבורי מרוחק. פלט : הצלחה/כישלון. שומרת את המפתח הציבורי של הצד השני.

- **get_public_key_pem()-** קלט : אין. פלט : המפתח הציבורי בפורמט PEM. מחזירה את המפתח הציבורי כמחרוזת.

- **encrypt(message)** - קלט : הודעה. פלט : הודעה מוצפנת. מצפינה הודעה עם המפתח הציבורי של הצד השני.

- **_encrypt_large_message(message)** - קלט : הודעה גדולה. פלט : הודעה מוצפנת מחולקת. מטפלת בהצפנת הודעות שגדולות מדי להצפנה בבת אחת.

- **decrypt(encrypted_message)** - קלט : הודעה מוצפנת. פלט : הודעה מפוענחת. מפענחת הודעה עם המפתח הפרטי.

- **_decrypt_large_message(encrypted_message)** - קלט : הודעה מוצפנת מחולקת. פלט : הודעה מפוענחת. מטפלת בפענוח הודעות גדולות.

- **hash_password(password, salt)** - קלט : סיסמה, מלח (אופציונלי). פלט : גיבוב מוצפן. מייצרת גיבוב מאובטח של סיסמה עם מלח אקראי.

- **verify_password(hashed_password, password_to_check)** - קלט : גיבוב מוצפן, סיסמה לבדיקה. פלט : האם הסיסמה תואמת. בודקת אם סיסמה תואמת לגיבוב השמור.-
create_secure_socket()- קלט : אין. פלט : אובייקט socket מאובטח. יוצרת חיבור socket מאובטח.

DatabaseHandler

תפקיד: מחלקה זו אחראית על כל האינטראקציות עם מסד הנתונים, כולל יצירת מסד הנתונים, ניהול משתמשים, אחסון הודעות ותמונות, ועוד.

מאפיינים:

- **db_name** - שם קובץ מסד הנתונים.

- **crypto** - מופע של מחלקת SecurityCrypto לביצוע פעולות אבטחה והצפנה.

מתודות - פעולות:

- **create_database()-** קלט: אין. פלט: אין. יוצרת את מסד הנתונים וטבלאותיו אם הם לא קיימים.

- **add_user(username, name, password, email, client_ip)** - קלט: פרטי משתמש, כתובת IP. פלט: הצלחה/כישלון. מוסיפה משתמש חדש למסד הנתונים.

- **login_user(username, password, client_ip)** - קלט: פרטי התחברות, כתובת IP. פלט: פרטי המשתמש אם ההתחברות הצליחה, אחרת None. מאמתת פרטי התחברות.

- **log_login_attempt(username, status, client_ip)** - קלט: שם משתמש, סטטוס, כתובת IP. פלט: אין. מתעדת ניסיון התחברות ביומן המערכת.

- **add_message(sender_username, receiver_username, hidden_message, client_ip)** - קלט: שולח, נמען, הודעה, כתובת IP. פלט: אין. מוסיפה הודעה למסד הנתונים.

- **add_message_multiple_recipients(sender_username, receiver_usernames, hidden_message, client_ip)** - קלט: שולח, רשימת נמענים, הודעה, כתובת IP. פלט: אין. מוסיפה את אותה הודעה למספר נמענים.

- **get_user_email(username)** - קלט: שם משתמש. פלט: כתובת אימייל. מחזירה את כתובת האימייל של משתמש.

- **get_users_emails(usernames)** - קלט: רשימת שמות משתמשים. פלט: מילון של שמות משתמשים וכתובות אימייל. מחזירה כתובות אימייל למספר משתמשים.

- **check_username_exists(username)** - קלט: שם משתמש. פלט: האם השם קיים. בודקת אם שם משתמש כבר קיים במערכת.

- **store_image(image_name, image_data, sender_username, client_ip)** - קלט: שם תמונה, נתוני תמונה, שולח, כתובת IP. פלט: מזהה התמונה. שומרת תמונה במסד הנתונים.

- **retrieve_image(image_id)** - קלט: מזהה תמונה. פלט: נתוני התמונה. מאחזרת תמונה מהמסד לפי מזהה.

- **get_message(hidden_message, username)** - קלט: הודעה מוסתרת, שם משתמש. פלט: ההודעה אם נמצאה, אחרת None. מחזירה הודעה ספציפית לפי תוכן ומשתמש.

EmailHandler

תפקיד: מחלקה זו אחראית על שליחת אימיילים, כולל קודי אימות, התראות על הודעות חדשות, ותמונות מקודדות.

מאפיינים:

- **from_email** - כתובת האימייל ממנה נשלחות ההודעות.

- **from_password** - סיסמת האימייל (או סיסמת אפליקציה) לצורך התחברות לשרת SMTP.

מתודות - פעולות:

- **send_email(recipient, subject, body, attachment_data, attachment_name, client_ip)** - קלט: פרטי האימייל וקובץ מצורף אופציונלי. פלט: הצלחה/כישלון. שולחת אימייל לנמען.

- **send_emails_to_multiple_recipients(recipients, subject, body, attachment_data, attachment_name, client_ip)** - קלט: מילון נמענים ופרטי האימייל. פלט: תוצאות השליחה. שולחת אימייל למספר נמענים במקביל.

- **send_email_thread_with_result(username, email, subject, body, attachment_data, _ attachment_name, results, client_ip)** - קלט: פרטי השליחה ואובייקט לתוצאות. פלט: אין. פונקציית עזר לשליחה מרובת תהליכונים.

- **generate_verification_code()**- קלט: אין. פלט: קוד אימות. מייצרת קוד אימות אקראי בן 6 ספרות.

- **send_verification_code(email, client_ip)** - קלט: כתובת אימייל, כתובת IP. פלט: קוד האימות אם השליחה הצליחה, אחרת None. שולחת קוד אימות לאימייל.

SteganographyHandler

תפקיד: מחלקה זו אחראית על הסתרת מידע בתמונות וחילוצו מהן, באמצעות אלגוריתם LSB (Least Significant Bit).

מתודות - פעולות (כולן סטטיות):

- **calculate_max_message_length(image_data)** - קלט: נתוני תמונה. פלט: אורך הודעה מקסימלי. מחשבת כמה מידע ניתן להסתיר בתמונה.

- **encode_image(image_data, message, receiver_username, client_ip)** - קלט: תמונה, הודעה, נמען, כתובת IP. פלט: תמונה מקודדת. מסתירה הודעה בתמונה עבור נמען ספציפי.

- **encode_image_multiple_recipients(image_data, message, receiver_usernames, client_ip)** - קלט: תמונה, הודעה, רשימת נמענים, כתובת IP. פלט: תמונה מקודדת. מסתירה הודעה בתמונה עבור מספר נמענים.

- **decode_image(image_data, current_user, client_ip)** - קלט: תמונה, המשתמש הנוכחי, כתובת IP. פלט: ההודעה המוסתרת אם המשתמש מורשה, אחרת None. מחלצת הודעה מוסתרת מתמונה.

Server

תפקיד: מחלקה זו מהווה את השרת של המערכת, אחראית על האזנה לחיבורים, טיפול בבקשות לקוחות, וניהול כל הפעולות בצד השרת.

מאפיינים:

- **host** - כתובת ה-IP שהשרת מאזין לה.
- **port** - הפורט שהשרת מאזין לו.
- **server_socket** - אובייקט ה-socket העיקרי של השרת.
- **running** - דגל שמציין אם השרת פעיל.
- **clients** - רשימה של אובייקטי socket ללקוחות מחוברים.
- **client_keys** - מילון שממפה socket ללקוח למפתחות הקריפטוגרפיים שלו.
- **client_ips** - מילון שממפה socket ללקוח לכתובת ה-IP שלו.
- **client_usernames** - מילון שממפה socket ללקוח לשם המשתמש שלו.
- **connection_history** - רשימה של חותמות זמן של חיבורים, לצורך הגבלת קצב.
- **max_connections_per_minute** - מספר החיבורים המקסימלי המותר בדקה (נגד DDOS).
- **db_handler** - מופע של DatabaseHandler לטיפול במסד הנתונים.
- **email_handler** - מופע של EmailHandler לשליחת אימיילים.
- **crypto** - מופע של SecurityCrypto לאבטחה והצפנה.
- **steg_handler** - מופע של SteganographyHandler לפעולות סטגנוגרפיה.

מתודות - פעולות:

- **start()**- קלט: אין. פלט: אין. מתחילה את פעולת השרת, מאזינה לחיבורים חדשים.
- **stop()**- קלט: אין. פלט: אין. עוצרת את פעולת השרת, סוגרת את כל החיבורים.
- **handle_client(client_socket, client_ip)** - קלט: socket של לקוח, כתובת IP. פלט: אין. מטפלת בתקשורת עם לקוח ספציפי.
- **process_request(request, client_socket, client_ip)** - קלט: בקשה, socket של לקוח, כתובת IP. פלט: תשובה. מעבדת בקשה ספציפית מלקוח.

Client

תפקיד: מחלקה זו מהווה את הלקוח של המערכת, אחראית על התחברות לשרת, שליחת בקשות וקבלת תשובות, וביצוע פעולות בצד הלקוח.

מאפיינים:

- **host** - כתובת ה-IP של השרת.

- **port** - הפורט של השרת.

- **socket** - אובייקט ה-socket של הלקוח.

- **connected** - דגל שמציין אם הלקוח מחובר לשרת.

- **crypto -** מופע של SecurityCrypto לאבטחה והצפנה.

- **current_user** - שם המשתמש המחובר כרגע, אם יש.

מתודות - פעולות:

- **connect()**- קלט: אין. פלט: הצלחה/כישלון. מתחברת לשרת, מחליפה מפתחות.

- **send_request(request_type, data)** - קלט: סוג הבקשה, נתונים. פלט: תשובה מהשרת. שולחת בקשה מוצפנת לשרת ומחזירה את התשובה.

- **close()**- קלט: אין. פלט: אין. סוגרת את החיבור לשרת.

- **register_user(username, name, password, email)** - קלט: פרטי משתמש חדש. פלט: הצלחה/כישלון, הודעה. רושמת משתמש חדש במערכת.

- **login_user(username, password)** - קלט: פרטי התחברות. פלט: הצלחה/כישלון, פרטי משתמש, קוד אימות. מתחברת למערכת.

- **log_login_attempt(username, status)** - קלט: שם משתמש, סטטוס. פלט: אין. מתעדת ניסיון התחברות.

- **check_username_exists(username)** - קלט: שם משתמש. פלט: האם קיים. בודקת אם שם משתמש קיים במערכת.

- **get_user_email(username)** - קלט: שם משתמש. פלט: כתובת אימייל. מקבלת את האימייל של משתמש.

- **get_users_emails(usernames)** - קלט: רשימת שמות משתמשים. פלט: מילון של אימיילים. מקבלת אימיילים למספר משתמשים.

- **send_email(recipient, subject, body, attachment_path)** - קלט: פרטי אימייל וקובץ מצורף אופציונלי. פלט: הצלחה/כישלון. שולחת אימייל.

- **send_emails_to_multiple_recipients(recipients, subject, body, attachment_path)** - קלט: נמענים ופרטי אימייל. פלט: תוצאות שליחה. שולחת אימייל למספר נמענים.

- **calculate_max_message_length(image_path)** - קלט: נתיב תמונה. פלט: אורך הודעה מקסימלי. בודקת כמה מידע ניתן להסתיר בתמונה.

- **encode_and_send(image_path, message, recipients)** - קלט: תמונה, הודעה, נמענים. פלט: הצלחה/כישלון, נתיב תמונה מקודדת, מזהה תמונה. מסתירה מידע בתמונה ושולחת לנמענים.

- **decode_image(image_path)** - קלט: נתיב תמונה. פלט: הצלחה/כישלון, הודעה. מחלצת מידע מוסתר מתמונה.

Gui_app

תפקיד : מחלקה זו אחראית על הממשק הגרפי של המערכת, כולל מסכי הרישום, ההתחברות, ההסתרה והפענוח.

מאפיינים (עיקריים) :

- **client** - מופע של מחלקת Client לתקשורת עם השרת.
- **verification_code** - קוד האימות הנוכחי, אם יש.
- **verification_code_time** - הזמן שבו נוצר קוד האימות.
- **login_in_progress** - דגל שמציין אם תהליך התחברות בביצוע.
- **recipients** - רשימת הנמענים הנוכחית להודעה.
- (+ מאפיינים רבים נוספים לרכיבי הממשק השונים)

מתודות - פעולות(עיקריות) :

- **setup_ui()-** קלט : אין. פלט : אין. מגדירה את מבנה הממשק הגרפי.
- **show_welcome_screen()-** קלט : אין. פלט : אין. מציגה את מסך הפתיחה.
- **show_login()-** קלט : אין. פלט : אין. מציגה את מסך ההתחברות.
- **show_register()-** קלט : אין. פלט : אין. מציגה את מסך הרישום.
- **start_login()-** קלט : אין. פלט : אין. מתחילה את תהליך ההתחברות.
- **show_verification_frame()-** קלט : אין. פלט : אין. מציגה את מסך האימות הדו-שלבי.
- **verify_code()-** קלט : אין. פלט : אין. מאמתת את קוד האימות שהוזן.
- **register()-** קלט : אין. פלט : אין. רושמת משתמש חדש.
- **setup_main_interface()-** קלט : אין. פלט : אין. מגדירה את הממשק הראשי לאחר התחברות.
- **setup_encode_tab(parent)** - קלט : אובייקט הורה. פלט : אין. מגדירה את לשונית ההסתרה.
- **setup_decode_tab(parent)** - קלט : אובייקט הורה. פלט : אין. מגדירה את לשונית הפענוח.
- **browse_image()-** קלט : אין. פלט : אין. פותחת חלון לבחירת תמונה להסתרה.
- **browse_decode_image()-** קלט : אין. פלט : אין. פותחת חלון לבחירת תמונה לפענוח.
- **add_recipient()-** קלט : אין. פלט : אין. מוסיפה נמען לרשימה.
- **encode_and_send()-** קלט : אין. פלט : אין. מסתירה מידע בתמונה ושולחת לנמענים.
- **decode_image_handler()-** קלט : אין. פלט : אין. מחלצת מידע מתמונה.
- (+ מתודות רבות נוספות לטיפול באירועים ותצוגה)

CyberStyleAnimation

תפקיד : מחלקה זו אחראית על אנימציית הפתיחה של האפליקציה, כולל אפקטים ויזואליים בסגנון סייבר-אבטחה.

מאפיינים (עיקריים) :

- **callback** - פונקציה שתקרא בסיום האנימציה.
- **canvas** - הקנבס שעליו מצוירת האנימציה.
- (+ מאפיינים רבים לאלמנטים הגרפיים של האנימציה)

מתודות - פעולות (עיקריות) :

- **run_animation()-** קלט : אין. פלט : אין. מתחילה את האנימציה.
- **_animate_binary()-** קלט : אין. פלט : אין. מאנימה את הקוד הבינארי הזורם.
- **_run_animation_sequence()-** קלט : אין. פלט : אין. מריצה את רצף האנימציה המלא.
- **_fade_to_main_ui()-** קלט : אין. פלט : אין. מבצעת אפקט התפוגגות למעבר לממשק הראשי.
- **close_and_callback()-** קלט : אין. פלט : אין. סוגרת את האנימציה וקוראת לפונקציית ה-callback.

TechButton

תפקיד : מחלקה זו מרחיבה את הכפתור הבסיסי של CustomTkinter, ומספקת כפתורים בעיצוב אחיד ומותאם לסגנון האפליקציה.

מאפיינים :

מאפיינים יורשים מ-CTkButton, עם ערכי ברירת מחדל מותאמים.

מתודות :

מתודות יורשות מ-CTkButton.

אינטראקציות בין המחלקות

מערכת StegnoLogic בנויה כמערכת מודולרית, שבה כל מחלקה אחראית על תחום ספציפי, אך המחלקות עובדות יחד בצורה חלקה. להלן תיאור האינטראקציות העיקריות בין המחלקות :

Gui_app ו-Client : הממשק הגרפי (Gui_app) משתמש במחלקת Client לכל התקשורת עם השרת. הוא יוצר מופע של Client בעת האתחול, וקורא למתודות שלו בתגובה לפעולות המשתמש. למשל, כאשר המשתמש לוחץ על כפתור ההתחברות, Gui_app קורא למתודה login_user של Client, ובהתאם לתשובה מציג את המסך הבא.

Client ו-SecurityCrypto : ה-Client משתמש במחלקת SecurityCrypto לכל פעולות ההצפנה והפענוח. הוא יוצר מופע של SecurityCrypto בעת האתחול, ומשתמש בו להצפנת הבקשות לשרת ולפענוח התשובות.

Server ו-SecurityCrypto : בדומה ל-Client, גם השרת משתמש ב-SecurityCrypto לאבטחת התקשורת. הוא יוצר מופע שלו בעת האתחול, וגם משתמש במתודות שלו כמו limit_connection_rate להגנה מפני DDOS.

Server ו-DatabaseHandler : השרת משתמש ב-DatabaseHandler לכל האינטראקציות עם מסד הנתונים. כאשר מתקבלת בקשה מלקוח (למשל, להתחבר), השרת עובר את הבקשה ל-DatabaseHandler לביצוע.

Server ו-EmailHandler : השרת משתמש ב-EmailHandler לשליחת אימיילים. כאשר נדרשת שליחת אימייל (למשל, קוד אימות או התראה על הודעה חדשה), השרת קורא למתודות המתאימות של EmailHandler.

Server ו-SteganographyHandler : השרת משתמש ב-SteganographyHandler לביצוע פעולות סטגנוגרפיה. כאשר מתקבלת בקשה להסתיר או לחלץ מידע מתמונה, השרת מעביר את הבקשה ל-SteganographyHandler.

Gui_app ו-CyberStyleAnimation : ה-Gui_app משתמש במחלקת CyberStyleAnimation להצגת אנימציית הפתיחה. הוא יוצר מופע של CyberStyleAnimation, ועובר למסך הבא רק לאחר שהאנימציה הסתיימה.

Gui_app ו-TechButton : ה-Gui_app משתמש במחלקת TechButton ליצירת כל הכפתורים בממשק. זה מבטיח מראה אחיד ועקבי בכל המסכים.

DatabaseHandler ו-SecurityCrypto : ה-DatabaseHandler משתמש ב-SecurityCrypto לאבטחת סיסמאות. כאשר נדרש לאחסן או לאמת סיסמה, הוא קורא למתודות המתאימות של SecurityCrypto.

Client ו-Server : אלו הן שתי המחלקות המרכזיות שמתקשרות זו עם זו דרך חיבור Socket. ה-Client שולח בקשות מוצפנות, וה-Server מפענח אותן, מטפל בהן, ושולח תשובות מוצפנות בחזרה.

תזרים נתונים טיפוסי במערכת יכול להיראות כך :

המשתמש מבצע פעולה בממשק הגרפי (למשל, לוחץ על כפתור).

Gui_app מזהה את הפעולה וקורא למתודה המתאימה של Client.

Client מכין בקשה, מצפין אותה באמצעות SecurityCrypto, ושולח אותה לשרת.

Server מקבל את הבקשה, מפענח אותה באמצעות SecurityCrypto, ומזהה את סוג הבקשה.

בהתאם לסוג הבקשה, Server קורא למחלקה המתאימה (DatabaseHandler, EmailHandler, או SteganographyHandler).

המחלקה המתאימה מבצעת את הפעולה הנדרשת ומחזירה תוצאה.

Server מצפין את התוצאה ושולח אותה בחזרה ל-Client.

Client מפענח את התשובה ומחזיר אותה ל-Gui_app.

Gui_app מעדכן את הממשק בהתאם לתוצאה.

המבנה המודולרי הזה מאפשר:

הפרדת אחריות: כל מחלקה אחראית על תחום ספציפי, מה שמקל על פיתוח, בדיקה ותחזוקה.

יכולת הרחבה: ניתן להוסיף פונקציונליות חדשה על ידי הוספת מחלקות חדשות או הרחבת הקיימות.

יכולת החלפה: ניתן להחליף מימוש של מחלקה אחת בלי להשפיע על השאר.

שימוש חוזר: מחלקות כמו SecurityCrypto משומשות על ידי מספר מחלקות אחרות.

לדוגמה, אם בעתיד ארצה להחליף את מנגנון הסטגנוגרפיה לאלגוריתם חדש, אוכל פשוט ליצור מחלקה חדשה או לעדכן את SteganographyHandler, בלי לשנות את שאר המערכת.

יישום הפרויקט חלק ב - אתגרים אלגוריתמיים וקוד מיוחד

לאחר שסקרנו את המודולים והמחלקות השונות במערכת, אפרט כעת על האתגרים האלגוריתמיים המרכזיים בפרויקט והפתרונות שיישמתי. הדגש יהיה על הפתרונות היצירתיים והמורכבים שפיתחתי כדי להתמודד עם האתגרים השונים בפרויקט.

אלגוריתם הסטגנוגרפיה (LSB - Least Significant Bit)

האתגר המרכזי בפרויקט היה מימוש שיטת הסטגנוגרפיה לשם הסתרת המידע בתמונות. התלבטתי בין מספר שיטות אפשריות והחלטתי לבסוף להשתמש בשיטת LSB שבה מסתירים מידע על ידי שינוי הביט הפחות משמעותי בערכי הפיקסלים בתמונה. בחרתי בשיטה זו בגלל פשטותה היחסית והעובדה שהיא משאירה תמונות שנראות זהות לעין אנושית.

האלגוריתם שפיתחתי מתחיל בהמרת ההודעה הטקסטואלית לייצוג בינארי, כאשר כל תו מיוצג על ידי 8 ביטים בהתאם לקידוד ASCII. לאחר מכן, האלגוריתם עובר על כל פיקסל בתמונה ומשנה את הביט האחרון (הפחות משמעותי) של ערכי הצבע (RGB) בכל פיקסל. שינוי הביט האחרון משפיע במידה כה מועטה על הצבע (שינוי של 1 מתוך 256 רמות) שהעין האנושית אינה מסוגלת להבחין בהבדל.

הייחודיות של המימוש שלי היא ביכולת לשלוח הודעה לנמען אחד או למספר נמענים, כאשר רק הנמענים המיועדים יכולים לפענח את ההודעה. זאת עשיתי על ידי הוספת מזהי המשתמשים לתחילת ההודעה המוסתרת. כך לדוגמה, אם ההודעה מיועדת למשתמש "alice", ההודעה המוסתרת תתחיל ב- "alice:" ולאחר מכן תבוא ההודעה עצמה. אם ההודעה מיועדת למספר משתמשים, ההודעה תתחיל במבנה כמו "alice|bob|Charlie:" ולאחר מכן תבוא ההודעה.

בנוסף, פיתחתי מנגנון חכם שמחשב מראש את הגודל המקסימלי של ההודעה שניתן להסתיר בתמונה נתונה. החישוב מבוסס על מספר הפיקסלים בתמונה כפול שלושה (עבור רכיבי RGB), חלקי שמונה (כי כל תו דורש 8 ביטים), ומוכפל ב-0.9 כדי להשאיר מקום למטא-נתונים. המשתמש מקבל הודעה ברורה עם המגבלה לפני שהוא מתחיל להקליד את ההודעה, וכך נמנעות שגיאות של הודעות ארוכות מדי.

אחד האתגרים הגדולים בפיתוח האלגוריתם היה הוספת "חתימת סיום" להודעה המוסתרת, כדי שהאלגוריתם ידע מתי להפסיק לקרוא בעת הפענוח. אחרי מספר ניסיונות, בחרתי ברצף הביטים "1111111111111110" כחתימת סיום, מכיוון שהוא נדיר מספיק כדי שלא יופיע במקרה בתוכן ההודעה.

אבטחת תקשורת מקצה לקצה

אתגר מרכזי נוסף בפרויקט היה יצירת תקשורת מאובטחת בין הלקוח לשרת. לאחר מחקר מעמיק של אפשרויות שונות, החלטתי לממש פתרון מבוסס RSA, שהוא אלגוריתם הצפנה אסימטרי יעיל ומוכח.

המימוש שיצרתי מבוסס על עקרון החלפת המפתחות. בעת יצירת החיבור הראשוני, כל צד (הלקוח והשרת) מייצר זוג מפתחות - מפתח ציבורי ומפתח פרטי. המפתח הציבורי נשלח לצד השני, בעוד שהמפתח הפרטי נשמר בסודיות מוחלטת. מרגע זה ואילך, כל הודעה שהלקוח שולח לשרת מוצפנת עם המפתח הציבורי של השרת (שרק השרת יכול לפענח עם המפתח הפרטי שלו), וכל הודעה שהשרת שולח ללקוח מוצפנת עם המפתח הציבורי של הלקוח.

אחד האתגרים המרכזיים בשימוש ב-RSA הוא ההגבלה על גודל הודעה שניתן להצפין. RSA מוגבל להצפנת קטעי מידע שגודלם קטן מגודל המפתח (בדרך כלל 2048 ביטים, או כ-245 בתים). כדי להתגבר על מגבלה זו, פיתחתי אלגוריתם שמחלק הודעות ארוכות לקטעים קצרים, מצפין כל קטע בנפרד, ואז משרשר אותם יחד עם סימני הפרדה מיוחדים ("||CHUNK||"). בצד המקבל, ההודעה מפוצלת שוב לפי סימני ההפרדה, כל חלק מפוענח בנפרד, והחלקים מאוחדים מחדש להודעה השלמה.

למרות שגישה זו מורכבת יותר מאשר שימוש בהצפנה סימטרית (כמו AES), היתרון הגדול הוא שאין צורך בערוץ מאובטח נפרד להחלפת מפתחות. הדבר חשוב במיוחד בסביבה כמו האינטרנט, שבה קשה להבטיח ערוץ מאובטח ראשוני.

בנוסף, מימשתי שכבת אבטחה נוספת באמצעות חישוב גיבוב (hash) של ההודעות המוחלפות, מה שמאפשר לצד המקבל לוודא שההודעה לא שונתה בדרך. זה חשוב במיוחד להגנה מפני התקפות "man-in-the-middle", שבהן תוקף עשוי לנסות לשנות את ההודעות המועברות.

התוצאה הסופית היא מערכת תקשורת מאובטחת לחלוטין, שמבטיחה סודיות (רק המקבל המיועד יכול לקרוא את ההודעה), אימות (המקבל יכול להיות בטוח במקור ההודעה), ושלמות (המקבל יכול לדעת אם ההודעה שונתה).

אימות דו-שלבי

אחד האלמנטים החשובים ביותר באבטחת מערכות מידע הוא אימות משתמשים. עבור StegnoLogic, לא הסתפקתי באימות חד-שלבי מבוסס שם משתמש וסיסמה, אלא פיתחתי מערכת אימות דו-שלבי (2FA - Two-Factor Authentication) שמוסיפה שכבת אבטחה נוספת.

המערכת שפיתחתי פועלת כך: בשלב הראשון, המשתמש מזין את שם המשתמש והסיסמה שלו. המערכת בודקת אם הפרטים נכונים על ידי השוואת הסיסמה המוצפנת במסד הנתונים. אם השלב הראשון עובר

בהצלחה, המערכת מייצרת קוד אימות אקראי בן 6 ספרות, ושולחת אותו לכתובת האימייל הרשומה של המשתמש.

בשלב השני, המשתמש צריך להזין את הקוד שקיבל באימייל. המערכת מוודאת שהקוד נכון ושהוא עדיין בתוקף (יש לו תוקף של 5 דקות מרגע השליחה). רק אם שני השלבים עוברים בהצלחה, המשתמש מקבל גישה למערכת.

כדי לשפר את חווית המשתמש, הוספתי טיימר ויזואלי שמראה למשתמש כמה זמן נותר עד שהקוד יפוג. הטיימר מתעדכן בזמן אמת ומשנה את צבעו כשנותר מעט זמן, כדי ליצור תחושת דחיפות.

האימות הדו-שלבי מוסיף שכבת אבטחה משמעותית, כי גם אם תוקף משיג איכשהו את הסיסמה של המשתמש, הוא עדיין צריך גישה לחשבון האימייל שלו כדי להשלים את ההתחברות. יתרה מכך, מכיוון שקוד האימות תקף רק לזמן קצר ומשתנה בכל התחברות, שיטות "replay attack" (שבהן תוקף מנסה להשתמש שוב בקוד קודם) אינן יעילות.

בנוסף, המערכת מתעדת את כל ניסיונות ההתחברות, כולל הצלחות וכישלונות, יחד עם פרטים כמו כתובת IP ושעה. זה מאפשר זיהוי של דפוסי התקפה אפשריים ומספק נתיב ביקורת (audit trail) למקרה של חקירת פריצה.

ניהול זיכרון יעיל לתמונות

העבודה עם תמונות הציבה אתגר של ניהול זיכרון וביצועים, במיוחד כשמדובר בתמונות גדולות. תמונות צבעוניות בפורמט לא דחוס יכולות לתפוס מאות מגה-בתים של זיכרון, ועיבוד שלהן יכול להיות כבד על המעבד. חיפשתי דרכים לייעל את השימוש במשאבי המחשב תוך שמירה על הביצועים והפונקציונליות.

אחד הפתרונות המרכזיים שמימשתי היה שימוש באובייקטי BytesIO במודול io של פייתון. במקום לשמור תמונות זמניות בדיסק, אני יוצר אובייקטי BytesIO בזיכרון. זה משיג שתי מטרות חשובות: ראשית, ביצועים טובים יותר כי גישה לזיכרון מהירה בהרבה מגישה לדיסק. שנית, זה מפחית עקבות דיגיטליות - אין קבצים זמניים שנשארים על הדיסק ועלולים להישמר גם אחרי מחיקה.

למשל, בפונקציה encode_image במחלקת SteganographyHandler, כאשר אני יוצר את התמונה המקודדת, אני משתמש ב-BytesIO כדי לשמור את התוצאה בזיכרון:

```python
output_bytes = io.BytesIO()
encoded_img.save(output_bytes, format='PNG')
return output_bytes.getvalue()
```

פתרון נוסף היה שמירת תמונות במסד הנתונים כ-BLOB (Binary Large Object). במקום לשמור תמונות כקבצים נפרדים במערכת הקבצים, אני שומר אותן ישירות במסד הנתונים. זה מפשט את הניהול של התמונות ואת הקשר בינן לבין המידע הקשור אליהן. בנוסף, זה מאפשר גיבוי והעברה פשוטים יותר של כל המידע יחד.

אתגר שהייתי צריך להתמודד איתו היה הדרך שבה SQLite (מסד הנתונים שבחרתי) מטפל בנתונים בינאריים. פיתחתי פונקציות עזר שממירות בין נתונים בינאריים לייצוג הפנימי של SQLite:

```python
c.execute("INSERT INTO images (image_name, image_data, sender_username, creation_date) VALUES (?, ?, ?, datetime('now'))",
          (image_name, sqlite3.Binary(image_data), sender_username))
```

שיטה שלישית לייעול ניהול הזיכרון הייתה עיבוד הדרגתי של הפיקסלים. במקום לטעון את כל התמונה לזיכרון בבת אחת ולעבד אותה כמקשה אחת, אני מעבד את הפיקסלים ברצף, כל אחד בתורו. זה מפחית משמעותית את צריכת הזיכרון, במיוחד בתמונות גדולות.

מימשתי גם מנגנון לניקוי זיכרון פרואקטיבי, שמשחרר אובייקטים גדולים מהזיכרון ברגע שאין בהם צורך יותר. זה עוזר למנוע דליפות זיכרון ושומר על ביצועים יציבים לאורך זמן.

התוצאה הסופית היא מערכת שיכולה לטפל בתמונות גדולות ביעילות, גם על מחשבים עם משאבים מוגבלים.

ניהול תהליכונים (Threads) למשימות מקביליות

אתגר חשוב בפיתוח האפליקציה היה יצירת מערכת תגובתית שיכולה לבצע פעולות ארוכות טווח מבלי לתקוע את הממשק המשתמש. בפייתון, ממשקי משתמש גרפיים בדרך כלל רצים בתהליכון (thread) יחיד, מה שאומר שכל פעולה ארוכה עלולה לגרום לממשק להיתקע עד שהפעולה תסתיים. כדי לפתור בעיה זו, מימשתי מערכת מבוססת תהליכונים מרובים.

אחת הדוגמאות הבולטות לשימוש בתהליכונים היא בשליחת אימיילים למספר נמענים. כאשר משתמש שולח הודעה מוסתרת למספר אנשים, ויש צורך לשלוח להם אימייל עם התמונה המקודדת, פעולה זו יכולה להימשך זמן רב אם יש מספר גדול של נמענים או אם יש עיכובים בשרת האימייל. במקום לגרום למשתמש להמתין, יצרתי מערכת שיוצרת תהליכון נפרד לכל שליחת אימייל:

לאחר שהתהליכונים הושקו, הפונקציה העיקרית ממשיכה לרוץ ומאפשרת למשתמש להמשיך בעבודה. תוצאות השליחה נאספות ומוצגות למשתמש כאשר כל התהליכונים מסיימים את פעולתם.

יישמתי גישה דומה לפעולות עיבוד תמונות. פעולות כמו קידוד ופענוח של תמונות יכולות להימשך זמן רב, במיוחד בתמונות גדולות. לכן, אני מבצע אותן בתהליכון נפרד, ובינתיים מציג למשתמש אינדיקטור התקדמות (כמו שינוי טקסט הכפתור ל-"מעבד...").

אחד האתגרים בעבודה עם תהליכונים הוא סנכרון ומניעת מצבי race condition, שבהם תהליכונים שונים מנסים לגשת או לשנות את אותו משאב בו-זמנית. כדי למנוע בעיות כאלה, מימשתי מנגנוני נעילה בעזרת threading.Lock. למשל, כשתהליכונים שונים צריכים לעדכן את אותו משתנה (כמו תוצאות שליחת אימיילים), אני משתמש במנעול כדי להבטיח שרק תהליכון אחד יכול לעדכן את המשתנה בכל רגע נתון.

בעיה נוספת שדרשה פתרון היא עדכון ממשק המשתמש מתהליכונים. בפייתון, ובמיוחד בtkinter, יש בעיה ידועה שרק התהליכון הראשי יכול לעדכן את ממשק המשתמש. כדי לפתור זאת, פיתחתי מנגנון שבו תהליכונים משניים יכולים לבקש עדכון של הממשק על ידי שימוש בפעולה after של tkinter, שמאפשרת תזמון של פונקציה לביצוע בתהליכון הראשי.

כל ההשקעה במערכת התהליכונים הניבה ממשק משתמש תגובתי וחלק, גם בעת ביצוע פעולות כבדות.

יצירת חיבור רשת מאובטח

יישום תקשורת רשת מאובטחת בין הלקוח לשרת היה אחד האתגרים המורכבים בפרויקט. בחרתי בארכיטקטורת לקוח-שרת מבוססת socket, שמאפשרת תקשורת דו-כיוונית בזמן אמת.

התחלתי מיצירת שכבת תקשורת בסיסית באמצעות מודול socket של פייתון, שמספק ממשק נמוך-רמה לחיבורי רשת. השרת יוצר socket שמאזין בכתובת IP ופורט מוגדרים, והלקוח מתחבר אליו. אבל חיבור socket בסיסי אינו מאובטח - הנתונים מועברים כטקסט גלוי שניתן ליירט ולשנות.

כדי לפתור את בעיית האבטחה, יישמתי שכבת הצפנה מעל חיבור ה-socket. השתמשתי בהצפנת RSA שיישמתי במחלקת SecurityCrypto, ובניתי פרוטוקול החלפת מפתחות שמתבצע בתחילת כל חיבור. הפרוטוקול פועל כך:

1. הלקוח מתחבר לשרת ומייצר זוג מפתחות RSA (ציבורי ופרטי).
2. השרת מייצר גם הוא זוג מפתחות.
3. הצדדים מחליפים את המפתחות הציבוריים שלהם.
4. מרגע זה, כל תקשורת מוצפנת עם המפתח הציבורי של היעד.

שכבת תקשורת נוספת שמימשתי היא שכבת הבקשה-תגובה. כל פעולה שהלקוח מבקש מהשרת (כמו התחברות, רישום, שליחת הודעה) מיוצגת כאובייקט JSON שכולל את סוג הבקשה והנתונים הרלוונטיים. השרת מפענח את הבקשה, מבצע את הפעולה הנדרשת, ומחזיר תגובה גם היא בפורמט JSON.

דוגמה לבקשת התחברות:

```
{
    "type": "login",
    "data": {
        "username": "<שם משתמש>",
        "password": "<סיסמה>"
    }
}
```

ודוגמה לתגובה:

```
{
    "status": "success" | "error",
    "user": [<פרטי משתמש>] | null,
    "verification_code": "<קוד אימות>" | null,
    "message": "<הודעת שגיאה>" | null
}
```

כדי להבטיח שהתקשורת לא תיתקע במקרה של שגיאה או ניתוק, יישמתי מנגנון timeout בצד הלקוח, שקובע את הזמן המקסימלי להמתנה לתגובה מהשרת. אם לא מתקבלת תגובה בתוך הזמן המוגדר, הלקוח מעלה חריגה ומנסה לטפל בה בצורה הולמת.

טיפלתי גם במקרי קצה כמו ניתוקים פתאומיים. כאשר משתמש מתנתק, או כאשר החיבור נפסק מסיבה כלשהי, השרת מזהה זאת ומנקה את המשאבים השייכים לאותו חיבור. בנוסף, מימשתי מנגנון לניקוי תקופתי של חיבורים "זומביים" - חיבורים שנותרו פתוחים אך לא פעילים.

התוצאה הסופית היא תשתית תקשורת מאובטחת, אמינה ועמידה מול תקלות שונות, שמהווה את הבסיס לכל האינטראקציות בין הלקוח לשרת במערכת.

יישום הפרויקט חלק ג - תיעוד בדיקות נוספות מעבר לבדיקות בשלב האפיון

בחלק זה אתאר את תהליך הבדיקות המקיף שביצעתי כדי לוודא שמערכת StegnoLogic פועלת כראוי ועומדת בכל הדרישות והמפרטים שהוגדרו מראש. הבדיקות תוכננו לכסות את כל היבטי המערכת, מהפונקציונליות הבסיסית, דרך מנגנוני האבטחה, ועד לממשק המשתמש והשימושיות הכללית.

בדיקות שתוכננו בשלב התכנון

בדיקת יצירת משתמש חדש

מטרת הבדיקה: בדיקה זו תוכננה לוודא שהמערכת מאפשרת רישום משתמשים חדשים כראוי, מטפלת נכון בשמירת הנתונים, ומיישמת את כל מנגנוני האבטחה הנדרשים לאחסון סיסמאות.

מה נעשה: הבדיקה כללה מספר שלבים. ראשית, ניסיתי ליצור משתמש חדש באמצעות ממשק הרישום, עם כל השדות הנדרשים: שם מלא, שם משתמש, סיסמה, וכתובת אימייל. בחנתי גם מקרי קצה כמו ניסיון ליצור משתמש עם שם משתמש שכבר קיים, או עם סיסמה חלשה מדי. לאחר הרישום, בדקתי את מסד הנתונים ישירות כדי לוודא שהנתונים נשמרו כראוי.

תוצאות: המערכת יצרה בהצלחה את המשתמש, הצפינה את הסיסמה עם שיטת PBKDF2, ושמרה את כל הנתונים במסד הנתונים. במקרה של ניסיון ליצור משתמש עם שם שכבר קיים, המערכת הציגה הודעת שגיאה מתאימה והנחיות כיצד להמשיך. בדומה, כאשר ניסיתי ליצור משתמש עם סיסמה חלשה מדי, המערכת הציגה הודעה מפורטת על הדרישות לסיסמה חזקה, ומחוון הסיסמה התעדכן בזמן אמת כדי להציג את חוזק הסיסמה.

בעיות ופתרונות: למרות שהמערכת פעלה ברוב המקרים כמצופה, זיהיתי מספר בעיות במהלך הבדיקות. ראשית, בגרסה המוקדמת, הסיסמאות נשמרו עם גיבוב MD5 פשוט, שנחשב כיום לא מספיק בטוח. שיניתי את המימוש כך שייעשה שימוש ב-PBKDF2 עם SHA-256 ומלח אקראי, שהיא שיטה מודרנית ומאובטחת יותר.

בעיה נוספת שזיהיתי הייתה חוסר תיעוד מספק של ניסיונות רישום כושלים. הוספתי מנגנון תיעוד מקיף שרושם את כל ניסיונות הרישום (מוצלחים וכושלים) עם פרטים כמו כתובת IP ושעה. זה מאפשר זיהוי של ניסיונות חשודים, כמו מישהו שמנסה ליצור חשבונות מרובים או לנחש שמות משתמשים קיימים.

בעיה שלישית הייתה קשורה לתגובתיות של ממשק המשתמש בזמן הרישום. תהליך הרישום, במיוחד חישוב הגיבוב של הסיסמה, יכול לקחת מספר שניות, מה שגרם לממשק להיראות קפוא. פתרתי זאת על ידי העברת התהליך לתהליכון נפרד והוספת אינדיקטור התקדמות שמעדכן את המשתמש על מצב התהליך.

בדיקת אימות דו-שלבי

מטרת הבדיקה: לוודא שהאימות הדו-שלבי דורש קוד תקף ובתוקף, ומונע התחברות ללא השלמת שלב זה. בנוסף, לבדוק את חווית המשתמש בתהליך, כולל הטיימר והודעות השגיאה.

מה נעשה: בדקתי את תהליך האימות הדו-שלבי במספר תרחישים:

1. התחברות עם פרטים נכונים, קבלת קוד אימות באימייל, והזנתו בצורה תקינה.
2. הזנת קוד אימות שגוי.
3. המתנה עד לפקיעת תוקף הקוד (5 דקות) וניסיון להזינו.
4. ניסיון לעקוף את שלב האימות הדו-שלבי על ידי גישה ישירה לדפים מאובטחים.
5. בקשת קוד אימות חדש במקרה של אובדן הקוד המקורי.

בנוסף, בדקתי את איכות הקוד שנשלח (אורך, מורכבות) ואת העיצוב והנראות של האימייל שמכיל אותו.

תוצאות: המערכת ניהלה את כל התרחישים כמצופה. בתרחיש הראשון, הקוד התקבל באימייל תוך שניות ספורות, והזנתו אפשרה כניסה מוצלחת למערכת. הטיימר הציג בצורה ברורה את הזמן הנותר לתוקף הקוד, והתעדכן בזמן אמת.

בתרחיש השני, כאשר הזנתי קוד שגוי, המערכת הציגה הודעת שגיאה ברורה ואפשרה לי לנסות שוב. אחרי שלושה ניסיונות כושלים, המערכת הציעה לי לבקש קוד חדש.

בתרחיש השלישי, כאשר ניסיתי להזין קוד שפג תוקפו, המערכת זיהתה זאת והציגה הודעה מתאימה, עם אפשרות לבקש קוד חדש.

בתרחיש הרביעי, כל ניסיון לגשת לדפים מאובטחים ללא השלמת האימות הדו-שלבי הוביל להפניה אוטומטית לדף האימות.

בתרחיש החמישי, בקשת קוד חדש עבדה כמצופה, עם משוב ברור למשתמש שקוד חדש נשלח, והקוד הישן בוטל.

בנוסף, הקוד עצמו היה באורך מתאים (6 ספרות), שמאזן בין אבטחה (מספיק אפשרויות למנוע ניחוש) לשימושיות (לא ארוך מדי להקלדה). עיצוב האימייל היה נקי ומקצועי, עם הנחיות ברורות והדגשת הקוד עצמו.

בעיות ופתרונות: למרות שרוב המערכת עבדה היטב, זיהיתי מספר בעיות. הבעיה העיקרית הייתה עם הטיימר שהציג את הזמן הנותר לתוקף הקוד. בגרסה הראשונית, הטיימר התבסס על השעה המקומית של המחשב, מה שיצר אי-התאמה אם השעה במחשב המשתמש לא הייתה מסונכרנת עם שעת השרת. פתרתי זאת על ידי שמירת שעת היצירה של הקוד בשרת, ושימוש בהפרש הזמנים בכל בדיקה.

בעיה שנייה הייתה שהמערכת המקורית לא טיפלה כראוי במקרה שהמשתמש סגר את חלון האימות ופתח אותו מחדש. במקרה כזה, הטיימר התחיל מחדש, אף שהקוד עצמו המשיך לספור מהזמן המקורי. תיקנתי זאת על ידי שמירת זמן היצירה של הקוד בשרת, ושליחתו ללקוח בעת טעינת דף האימות.

שיפור נוסף שהוספתי היה הגדלת הבולטות של האימייל עם קוד האימות, כך שיהיה פחות סיכוי שייפול לתיקיית הספאם או יתעלם ממנו. הוספתי כותרת ברורה, פורמט HTML עם עיצוב מודגש, וגם גרסת טקסט פשוט למקרה שלקוח האימייל לא תומך ב-HTML.

לבסוף, שיפרתי את חווית המשתמש על ידי הוספת אפשרות לשלוח את הקוד מחדש בלחיצת כפתור, במקרה שהאימייל המקורי התעכב או לא הגיע. כמו כן, הוספתי הנחיות ברורות יותר על המסך עצמו, כולל הוראות לבדוק את תיקיית הספאם אם האימייל לא מגיע.

בדיקת קידוד ופענוח הודעה

מטרת הבדיקה: לוודא שאפשר להסתיר הודעה בתמונה ולחלץ אותה בהצלחה, ושהתמונה המקודדת נראית זהה לתמונה המקורית לעין אנושית.

מה נעשה: ביצעתי מספר בדיקות קידוד ופענוח:

1. הסתרת הודעות באורכים שונים (קצרות, בינוניות, וארוכות) בתמונות שונות.
2. הסתרת הודעות בתמונות בפורמטים שונים (PNG, JPEG, BMP, GIF).
3. הסתרת הודעות בתמונות בגדלים שונים, מתמונות קטנות ועד תמונות גדולות מאוד.
4. השוואה ויזואלית של התמונות המקוריות והמקודדות, כולל הגדלה (zoom) של אזורים ספציפיים.
5. ניסיונות לחלץ הודעות מתמונות מקודדות באמצעות משתמשים שונים (מורשים ולא מורשים).

תוצאות: המערכת הצליחה להסתיר ולחלץ הודעות בהצלחה בכמעט כל המקרים. הודעות קצרות, בינוניות וארוכות הוסתרו וחולצו ללא בעיות, כל עוד לא חרגו מהגבול המקסימלי שמערכת חישבה לכל תמונה.

הקידוד עבד היטב עם תמונות בפורמט PNG ו-BMP, שהם פורמטים ללא דחיסה או עם דחיסה ללא איבוד מידע (lossless). עם זאת, היו בעיות עם תמונות JPEG, שעוברות דחיסה עם איבוד מידע (lossy compression), מה שהוביל לאובדן חלק מהמידע המוסתר. תמונות GIF תמכו בקידוד, אך היו מוגבלות בכמות המידע שניתן להסתיר בגלל מספר הצבעים המוגבל שלהן.

מבחינה ויזואלית, התמונות המקודדות נראו זהות לחלוטין לתמונות המקוריות, גם בהגדלה משמעותית. זה מאשר שהשיטה יעילה מבחינת הסטגנוגרפיה - המידע מוסתר מבלי לשנות את המראה החיצוני של התמונה.

הבדיקות אישרו גם שרק משתמשים מורשים יכולים לחלץ את ההודעות המוסתרות. כאשר משתמש שלא היה הנמען המיועד ניסה לפענח תמונה מקודדת, המערכת הציגה הודעה שאין הודעה עבורו בתמונה.

בעיות ופתרונות: הבעיה המרכזית שזיהיתי הייתה עם תמונות JPEG. בגלל האלגוריתם הדחיסה של JPEG, שמשנה ערכי פיקסלים, המידע המוסתר היה נפגע או אובד לחלוטין בעת שמירת התמונה המקודדת. פתרתי זאת על ידי המרה אוטומטית של כל תמונה לפורמט PNG לפני הקידוד, ללא קשר לפורמט המקורי. בנוסף, הוספתי אזהרה למשתמש שבוחר תמונת JPEG, עם הסבר מדוע התמונה תומר לPNG ושקובץ התוצאה עשוי להיות גדול יותר.

בעיה נוספת הייתה זיהוי סוף ההודעה המוסתרת. בגרסה המוקדמת, המערכת התקשתה לזהות היכן מסתיימת ההודעה, מה שהוביל לעתים לשחזור של "רעש" בסוף ההודעה המקורית. פתרתי זאת על ידי הוספת רצף מיוחד של ביטים (1111111111111110) בסוף כל הודעה מוסתרת, שמשמש כסימן סיום. בעת פענוח, המערכת מזהה את הרצף הזה ויודעת שההודעה הסתיימה.

שיפור משמעותי נוסף היה האופטימיזציה של אלגוריתם הקידוד לתמונות גדולות. בגרסה המקורית, קידוד של תמונות גדולות מאוד לקח זמן רב והשתמש בהרבה זיכרון. שיפרתי את האלגוריתם על ידי עיבוד הפיקסלים בקבוצות (במקום לטעון את כל התמונה לזיכרון בבת אחת), ושימוש בטכניקות אופטימיזציה כמו גישה ישירה למערך הפיקסלים במקום שימוש בפונקציות גבוהות יותר. התוצאה הייתה שיפור משמעותי בביצועים, במיוחד בתמונות גדולות.

בדיקת מערכת הרשאות

מטרת הבדיקה: לוודא שרק הנמענים המיועדים יכולים לפענח את ההודעות המוסתרות, ושמערכת ההרשאות מטפלת נכון במקרים מורכבים כמו הודעות לנמענים מרובים.

מה נעשה: בדקתי את מערכת ההרשאות במספר תרחישים:

1. הסתרת הודעה עבור משתמש אחד ספציפי, וניסיון לפענח אותה עם משתמשים שונים.
2. הסתרת הודעה עבור מספר משתמשים, וניסיון לפענח אותה עם כל אחד מהם ועם משתמשים שאינם ברשימה.
3. תרחישים מיוחדים, כמו שמות משתמשים ארוכים במיוחד, שמות עם תווים מיוחדים, או רשימה ארוכה של נמענים.
4. בדיקת התנהגות המערכת כאשר חלק מהמשתמשים אינם קיימים יותר במערכת.

תוצאות: המערכת ניהלה את כל התרחישים בהצלחה. בתרחיש הראשון, רק המשתמש המיועד הצליח לפענח את ההודעה, בעוד שמשתמשים אחרים קיבלו הודעה שהתמונה אינה מכילה מידע עבורם.

בתרחיש השני, כל אחד מהמשתמשים ברשימת הנמענים הצליח לפענח את ההודעה, בעוד שמשתמשים שלא היו ברשימה לא הצליחו. המערכת טיפלה נכון ברשימות נמענים בכל גודל שבדקתי.

בתרחיש השלישי, המערכת התמודדה היטב עם שמות משתמשים מיוחדים. למשל, שם משתמש שכולל את התו "|" (שמשמש כמפריד בין שמות) טופל נכון על ידי קידוד מיוחד שמונע בלבול.

בתרחיש הרביעי, כאשר הסרתי משתמש מהמערכת והוא היה אחד הנמענים בהודעה מוסתרת, המערכת טיפלה בכך נכון: משתמשים אחרים ברשימה עדיין יכלו לפענח את ההודעה, אך כמובן המשתמש שהוסר כבר לא יכל להתחבר למערכת.

בעיות ופתרונות: הבעיה העיקרית שזיהיתי הייתה בשמירת רשימת נמענים ארוכה. בגרסה המוקדמת, היה מגבלה על אורך רשימת הנמענים בגלל האופן שבו היא אוחסנה בתחילת ההודעה המוסתרת. הדבר היה בעייתי במיוחד עבור תמונות קטנות, שבהן המטא-דאטה (רשימת הנמענים) תפס חלק גדול מהמקום הזמין להסתרת מידע.

פתרתי זאת על ידי שינוי המבנה ושימוש במפריד "|" בין שמות המשתמשים, ובנוסף, הוספתי מנגנון דחיסה למטא-דאטה במקרים של רשימות ארוכות במיוחד. כמו כן, הוספתי בדיקה מראש אם רשימת הנמענים אינה ארוכה מדי עבור תמונה ספציפית, והצגת אזהרה מתאימה למשתמש במקרה כזה.

בעיה אחרת הייתה טיפול בשמות משתמשים שכוללים תווים מיוחדים, במיוחד התו "|" שמשמש כמפריד. פתרתי זאת על ידי קידוד של תווים מיוחדים בשמות משתמשים לפני שילובם ברשימת הנמענים, ופענוח שלהם בעת קריאת הרשימה.

שיפור נוסף היה הוספת מנגנון אימות נוסף מעבר לשם המשתמש. בנוסף לבדיקה אם שם המשתמש נמצא ברשימת הנמענים, המערכת בודקת גם אם המשתמש הוא אכן בעל החשבון הזה על ידי שימוש במאפיין מזהה נוסף שרק המשתמש האמיתי אמור לדעת. זה מגביר את האבטחה ומקשה על התחזות.

בדיקת שליחת הודעה למספר נמענים

מטרת הבדיקה: לוודא שאפשר לשלוח את אותה הודעה למספר נמענים, וששליחת האימיילים עובדת כראוי עם מספר גדול של נמענים.

מה נעשה: ביצעתי בדיקות של שליחת הודעות לנמענים מרובים במספר תרחישים:

1. שליחת הודעה ל-3 נמענים, כולל צפייה בתמונה המקודדת על ידי כל אחד מהם.
2. שליחת הודעה למספר גדול של נמענים (10 ומעלה) כדי לבדוק התנהגות המערכת תחת עומס.
3. שילוב של שליחת אימייל אוטומטית לכל הנמענים, עם ובלי קובץ מצורף.
4. שליחת הודעה לנמען שאינו קיים יותר במערכת.
5. בדיקת שליחה כאשר שרת האימייל אינו זמין או כאשר חסרים אישורי התחברות.

תוצאות: המערכת הצליחה לטפל היטב ברוב התרחישים. בתרחיש הראשון, ההודעה נשלחה בהצלחה לכל שלושת הנמענים, וכל אחד מהם הצליח לפענח את ההודעה מהתמונה המקודדת.

בתרחיש השני, עם מספר גדול של נמענים, המערכת טיפלה נכון בקידוד ההודעה עבור כולם, אך זיהיתי עיכובים בשליחת האימיילים כאשר מספר הנמענים היה גדול במיוחד. מדידות ביצועים הראו שהשליחה הסדרתית (אחד אחרי השני) גרמה לעיכוב, במיוחד אם היו קבצים מצורפים גדולים.

בתרחיש השלישי, שליחת אימיילים עם קבצים מצורפים עבדה כמצופה, עם תצוגה נכונה של הקובץ המצורף בתוכנות אימייל שונות.

בתרחיש הרביעי, המערכת טיפלה נכון בניסיון לשלוח הודעה לנמען שאינו קיים יותר - הודעה נשלחה לכל הנמענים הקיימים, ונרשמה שגיאה עבור הנמען שאינו קיים.

בתרחיש החמישי, המערכת הציגה הודעת שגיאה ברורה כאשר שרת האימייל לא היה זמין או כאשר חסרו אישורי התחברות, ונתנה הנחיות להגדרת פרטי SMTP בקובץ .env.

בעיות ופתרונות: הבעיה העיקרית שזיהיתי הייתה עם שליחת אימיילים מרובים באופן סדרתי, שגרמה לזמני המתנה ארוכים למשתמש. פתרתי זאת על ידי יישום שליחה במקביל (פראלל) באמצעות תהליכונים (threads). כל שליחת אימייל מתבצעת בתהליכון נפרד, מה שמאפשר לשלוח מספר רב של אימיילים במקביל, ומקצר משמעותית את זמן ההמתנה הכולל.

בנוסף, הוספתי מנגנון ניטור ודיווח בזמן אמת שמציג למשתמש את התקדמות השליחה. בכל פעם ששליחה לנמען מסוים מסתיימת (בהצלחה או בכישלון), המשתמש מקבל עדכון, כך שהוא יודע מה הסטטוס בכל רגע נתון.

בעיה נוספת הייתה עם קבצים מצורפים גדולים, שלעתים חרגו ממגבלות שרת האימייל. פתרתי זאת על ידי הוספת בדיקת גודל קובץ ומנגנון דחיסה אוטומטי שמקטין תמונות גדולות מדי לפני השליחה, תוך שמירה על איכות מספקת. בנוסף, המערכת עכשיו מתריעה מראש אם קובץ עשוי להיות גדול מדי לשליחה.

שיפור נוסף היה הוספת מנגנון ניסיון חוזר (retry) במקרה של כישלון. אם שליחת אימייל לנמען מסוים נכשלת, המערכת מנסה שוב מספר פעמים, עם השהיות הולכות וגדלות בין הניסיונות. זה עוזר להתמודד עם בעיות זמניות בשרת האימייל או ברשת.

בדיקת תצוגה נכונה של תמונות מקודדות

מטרת הבדיקה: לוודא שהתמונות המקודדות נראות זהות לתמונות המקוריות, כך שלא ניתן להבחין ויזואלית שהן מכילות מידע מוסתר.

מה נעשה: ערכתי סדרת השוואות מעמיקות בין תמונות מקוריות לתמונות מקודדות:

1. השוואה ויזואלית פשוטה על ידי הצגת שתי התמונות זו לצד זו.

2. השוואה מפורטת באמצעות הגדלה (zoom) של אזורים ספציפיים, במיוחד אזורים עם צבעים אחידים או מרקמים עדינים.

3. ניתוח ממוחשב שחישב את ההבדל הפיקסלי בין התמונות ויצר "מפת הבדלים" ויזואלית.

4. "מבחן עיוור" שבו נבדקים התבקשו לזהות איזו מבין שתי תמונות מכילה מידע מוסתר.

5. השוואת היסטוגרמות של התמונות (התפלגות ערכי הצבע).

תוצאות: התוצאות היו מרשימות ברובן, עם הבדלים מזעריים בין התמונות המקוריות למקודדות:

1. בהשוואה ויזואלית פשוטה, לא היה ניתן להבחין בכל הבדל בין התמונות. אפילו כשהוצגו זו לצד זו, לא ניתן היה לזהות איזו מהן מכילה מידע מוסתר.

2. בהגדלה של אזורים ספציפיים, רוב האזורים עדיין נראו זהים. עם זאת, באזורים מסוימים עם צבע אחיד (במיוחד שחור או צבעים כהים אחרים), ניתן היה לעתים להבחין בהבדלים עדינים כשמסתכלים בהגדלה גבוהה מאוד.

3. הניתוח הממוחשב אישר שההבדלים היו מוגבלים לביט האחרון של ערכי RGB, כמצופה מאלגוריתם LSB. מפת ההבדלים הראתה פיזור אחיד של שינויים ברחבי התמונה, בלי דפוסים שעלולים לרמוז על קיום מידע מוסתר.

4. במבחן העיוור, המשתתפים לא הצליחו לזהות את התמונות המקודדות ברמה גבוהה מניחוש אקראי, מה שמעיד על כך שההבדלים אינם נראים לעין אנושית רגילה.

5. ניתוח היסטוגרמות הראה הבדלים מזעריים בלבד, שנמצאים בגבולות הרעש הסטטיסטי הרגיל. זה חשוב במיוחד להתנגדות לניתוח סטגנוגרפי (steganalysis), שמנסה לזהות תמונות מקודדות באמצעים סטטיסטיים.

בעיות ופתרונות: למרות התוצאות הטובות בסך הכל, זיהיתי מספר בעיות שדרשו פתרון:

1. בתמונות עם אזורים גדולים של צבע אחיד וכהה, לעתים ניתן היה להבחין בשינויים מזעריים בעת הגדלה משמעותית. פתרתי זאת על ידי שיפור אלגוריתם הפיזור של המידע המוסתר, כך שיעדיף אזורים עם מרקם מורכב ויימנע מאזורים אחידים.

2. ניתוח סטטיסטי מתקדם יכול עדיין לזהות לעתים תמונות מקודדות, במיוחד אם הן מכילות כמות גדולה של מידע ביחס לגודלן. שיפרתי את זה על ידי יישום טכניקה הנקראת "adaptive LSB", שמשנה את הביטים באופן שמשמר טוב יותר את ההתפלגות הסטטיסטית של התמונה המקורית.

3. בחלק מהמקרים, היה הבדל קטן בגודל הקובץ בין התמונה המקורית למקודדת, מה שיכול לשמש כרמז על קיומו של מידע מוסתר. פתרתי זאת על ידי הוספת "מילוי" (padding) אקראי לתמונה המקורית לפני הקידוד, כך שהגודל הסופי יהיה קבוע ללא קשר לכמות המידע המוסתר.

4. תמונות עם אזורים של שקיפות (alpha channel) לעתים הציגו הבדלים בולטים יותר. שיפרתי את האלגוריתם כך שיטפל נכון בערוץ השקיפות, או יימנע משימוש בו לחלוטין לצורך הסתרת מידע.

5. הוספתי גם אפשרות לקידוד ברמות איכות שונות: ברמה הסטנדרטית, המערכת משתמשת בביט אחד בכל ערוץ צבע (RGB) של כל פיקסל. ברמה ה"סמויה במיוחד", המערכת משתמשת רק בערוץ אחד (למשל, רק בערוץ הכחול, שהעין האנושית פחות רגישה אליו), מה שמפחית את כמות המידע שניתן להסתיר אבל מגביר את הסמויות.

שיפורים אלה הגבירו עוד יותר את הסמויות של התמונות המקודדות, והפכו את זיהוין לכמעט בלתי אפשרי בשיטות רגילות.

בדיקות נוספות שבוצעו במהלך הפיתוח

במהלך הפיתוח, זיהיתי צורך בבדיקות נוספות שלא תוכננו מראש. אלה היו חיוניות להבטיח את האיכות והאמינות של המערכת בתרחישים נוספים.

בדיקת שימוש בתמונות שונות

מטרת הבדיקה: לוודא שהאלגוריתם עובד ביעילות עם מגוון רחב של תמונות בפורמטים, גדלים ותכונות שונות.

מה נעשה: יצרתי מאגר בדיקות מקיף של תמונות מסוגים שונים:

1. תמונות בפורמטים שונים: PNG, JPG, BMP, GIF (כולל GIF מונפשים).

2. תמונות בגדלים שונים, מתמונות קטנות מאוד (x100100 פיקסלים) ועד תמונות ענקיות (x30004000 פיקסלים ומעלה).

3. תמונות עם מאפיינים ספציפיים: אזורים גדולים של צבע אחיד, מרקמים מורכבים, תצלומים ריאליסטיים, איורים, ותמונות עם שקיפות.

4. תמונות בעומקי צבע שונים: 8 ביט, 16 ביט, 24 ביט ו-32 ביט (עם שקיפות).

5. תמונות בדחיסה שונה: ללא דחיסה, דחיסה ללא אובדן (lossless), ודחיסה עם אובדן (lossy) ברמות שונות.

לכל אחת מהתמונות, ניסיתי לקודד ולפענח הודעת טקסט סטנדרטית, ובדקתי אם התהליך מצליח, איכות התוצאה, ומידת השינוי הוויזואלי בתמונה.

תוצאות: הבדיקות הראו תוצאות מעורבות, עם הבדלים משמעותיים בין פורמטים שונים של תמונות:

1. תמונות PNG ו-BMP עבדו באופן מושלם. אלה פורמטים ללא דחיסה או עם דחיסה ללא אובדן, מה שמשמר את השינויים המזעריים שהאלגוריתם עושה בערכי הפיקסלים. הקידוד והפענוח עבדו ללא שגיאות, והתמונות המקודדות נראו זהות לחלוטין למקור.

2. תמונות JPEG היוו אתגר משמעותי. אלגוריתם הדחיסה של JPEG, שמבוסס על חלוקת התמונה לבלוקים ושימוש בטרנספורמציית DCT (Discrete Cosine Transform), גורם לשינויים בערכי הפיקסלים בעת שמירת התמונה. שינויים אלה הורסים חלק מהמידע המוסתר. בבדיקות, הודעות שהוסתרו בתמונות JPEG לעתים קרובות לא ניתנו לפענוח מלא, או שהכילו שגיאות.

3. תמונות GIF עבדו חלקית. GIF הוא פורמט עם פלטה מוגבלת של 256 צבעים, מה שמפחית את מספר הביטים שניתן לשנות. הודעות קצרות ניתנו להסתרה בהצלחה, אבל היכולת להסתיר מידע בכמות גדולה הייתה מוגבלת. בנוסף, GIF מונפשים הציגו אתגר מיוחד בגלל המבנה המורכב שלהם.

4. גודל התמונה השפיע ישירות על כמות המידע שניתן להסתיר. ככל שהתמונה גדולה יותר, כך אפשר להסתיר יותר מידע. עם זאת, תמונות גדולות מאוד הציגו אתגרי ביצועים, עם זמני עיבוד ארוכים וצריכת זיכרון גבוהה.

5. תמונות עם אזורים גדולים של צבע אחיד היו בעייתיות יותר מבחינת איכות הסטגנוגרפיה. השינויים בביטים האחרונים של ערכי הפיקסלים היו לעתים נראים לעין בהגדלה באזורים כאלה, במיוחד באזורים עם צבע אחיד וכהה.

רפלקציה

תהליך הפיתוח והאתגרים

כשהתחלתי את פרויקט StegnoLogic, לא הייתי בטוח לגמרי לאן אני הולך. היה לי רעיון בסיסי להסתיר מידע בתמונות, אבל לא צפיתי את כל המורכבות והאתגרים שיבואו בעקבותיו. התחלתי בקריאה על סטגנוגרפיה והצפנה. חשבתי שזה יהיה די פשוט – לקחת כמה אלגוריתמים קיימים, לחבר אותם יחד, ולקבל מערכת עובדת. אבל מהר מאוד הבנתי שיש פער עצום בין הבנת העקרונות התיאורטיים ובין היישום המעשי שלהם.

האתגר הראשון שנתקלתי בו היה בעיצוב מערכת ההצפנה. קראתי על RSA וחשבתי שאוכל להשתמש בו ישירות, אבל כשהתחלתי לנסות ליישם את זה בקוד, גיליתי מגבלות שלא ציפיתי להן, כמו הגבלת גודל על מה שאפשר להצפין. הייתי צריך לפתח מנגנון שפורס הודעות גדולות לחלקים, מצפין כל חלק בנפרד, ואז משלב אותם מחדש בצד השני. היה לי רגע של "וואו" כשזה עבד בפעם הראשונה – שלחתי הודעה ארוכה וקיבלתי אותה בצד השני בדיוק כמו ששלחתי. זה נתן לי ביטחון להמשיך.

האתגר השני היה עם LSB (Least Significant Bit), האלגוריתם שמסתיר מידע בתמונות. למדתי עליו בתיאוריה, אבל כשניסיתי ליישם את זה על תמונות אמיתיות, היו לי כל מיני בעיות – לפעמים המידע נהרס, לפעמים הוא היה חלקי, ולפעמים הוא לא היה מוסתר מספיק טוב. הייתי צריך לעבור על כל פיקסל בתמונה ולשחק עם הערכים שלו, מה שלקח המון זמן ותסכל אותי לפעמים. לא הייתי בטוח אם אני בכלל בכיוון הנכון.

נקודת המפנה הגיעה אחרי כמעט שבועיים של ניסיונות, כשהצלחתי להסתיר הודעה ארוכה יחסית בתמונה ולחלץ אותה מחדש בלי שום נזק. התמונה המקודדת נראתה בדיוק כמו המקורית, וזה היה ממש מרגש.

תהליך הלמידה

הפרויקט הזה דחף אותי ללמוד המון דברים חדשים שלא הכרתי קודם. למדתי איך להשתמש בספריות כמו PIL לעבודה עם תמונות, rsa להצפנה, ו-socketלתקשורת ברשת. כל אחת מהן הייתה עולם שלם של ידע חדש. אחד הדברים המעניינים ביותר שלמדתי היה איך לעבוד עם threads (תהליכונים) ב-Python. בהתחלה, הממשק שלי היה "קופא" בכל פעם שהרצתי קוד שלוקח הרבה זמן, כמו קידוד של תמונה גדולה. חיפשתי פתרון ומצאתי מדריך מעולה על threads. אחרי שהבנתי איך להשתמש בהם, יכולתי לגרום לממשק להמשיך להגיב תוך כדי שפעולות "כבדות" רצות ברקע. זה היה שיפור עצום לחוויית המשתמש.

למדתי גם הרבה על אבטחת מידע. לא הסתפקתי בקריאה על אלגוריתמים, אלא ניסיתי להבין את ה"למה" שמאחוריהם. למדתי למה חשוב לגבב (hash) סיסמאות עם מלח (salt), למה צריך אימות דו-שלבי, ואיך מתמודדים עם התקפות כמו brute force או man-in-the-middle. היה מעניין לראות כמה רבדים יש לאבטחת מידע, וכמה חשוב לחשוב על כל אחד מהם.

כלים ומיומנויות לעתיד

אני מרגיש שרכשתי המון כלים וידע שישמשו אותי בעתיד. בצד הטכני, למדתי המון על פייתון, שאני מאמין שיהיה שימושי בכל תחום שאבחר. קוד התקשורת והצפנה שכתבתי יכול לשמש אותי בפרויקטים עתידיים. אני מרגיש שאני מבין את הקונספטים של סטגנוגרפיה והצפנה ברמה שלא הייתה לי קודם.

אבל יותר מזה, למדתי גם דברים "רכים" כמו התמדה ופתרון בעיות. היו רגעים שבהם הייתי במבוי סתום ורציתי לוותר או לבחור בפרויקט פשוט יותר. אבל המשכתי לנסות, לפרק את הבעיה לחלקים קטנים יותר, ולחפש פתרונות יצירתיים. התהליך הזה של "תקוע-מנסה-פותר" חיזק את הביטחון שלי ביכולת שלי להתמודד עם בעיות מורכבות, גם אם אין לי את כל הידע מראש.

למדתי גם איך לקרוא תיעוד טכני, שזה מיומנות שאני חושב שתעזור לי בכל תחום הנדסי. בהתחלה התיעוד נראה לי כמו שפה זרה, אבל עם הזמן למדתי איך לחלץ ממנו את המידע שאני צריך. אני מוצא את עצמי קורא ומבין דברים שפעם נראו לי בלתי אפשריים.

תובנות ושיתוף פעולה

אני חייב להודות גם למורים שלי במגמה ואבי ואחותי, אשר סיפקו תשובות לחלק מהשאלות שלי.

מה הייתי משנה אילו יכולתי להתחיל מחדש

אם הייתי מתחיל את הפרויקט מהתחלה, הייתי עושה כמה דברים אחרת. הדבר הראשון הוא תכנון מבנה התוכנה. התחלתי לכתוב קוד די מהר, בלי לחשוב מספיק על הארכיטקטורה. כתוצאה מכך, הייתי צריך לכתוב מחדש חלקים שלמים כשהבנתי שהמבנה הראשוני לא יעבוד טוב בטווח הארוך. הייתי מקדיש יותר זמן בהתחלה לתכנון המחלקות והממשקים ביניהן, גם אם זה היה מעכב את תחילת הכתיבה של הקוד עצמו.

דבר שני שהייתי עושה אחרת הוא להתחיל עם גרסה פשוטה יותר ולשפר אותה בהדרגה. במקום לנסות ליישם את כל התכונות בבת אחת, הייתי מתחיל עם מערכת בסיסית שרק מסתירה ומחלצת מידע, ואז מוסיף תכונות כמו הצפנה, אימות דו-שלבי, וכו'. זה היה עוזר לי לראות התקדמות מהירה יותר ולא להרגיש תקוע.

מה הייתי משפר עם יותר משאבים

אם היה לי יותר זמן וידע, הייתי משפר את המערכת במספר דרכים. ראשית, הייתי מפתח אלגוריתם סטגנוגרפיה מתקדם יותר שעמיד יותר לשינויים בתמונה. האלגוריתם הנוכחי (LSB) מאוד רגיש לדחיסה, חיתוך ושינויי גודל. קראתי על שיטות שמשתמשות בתחום התדרים (frequency domain) שעמידות יותר, והייתי רוצה ליישם אחת מהן.

שנית, הייתי מפתח גרסה מקוונת של האפליקציה שתעבוד בדפדפן. כרגע, המשתמשים צריכים להוריד ולהתקין את האפליקציה. גרסת ווב הייתה מאפשרת גישה קלה יותר ושיתוף תמונות מוסתרות ישירות דרך האתר.

שאלות אישיות למחקר עתידי

תוך כדי העבודה על הפרויקט, עלו לי כמה שאלות שהייתי רוצה לחקור בעתיד:

- האם יש נקודת איזון טובה בין אבטחה לשימושיות? איפה מוותרים על אבטחה לטובת נוחות המשתמש, ואיפה לא?

- איך עובדות מערכות התראה לניסיונות פריצה? האם אפשר לזהות התקפות לפני שהן מצליחות?

- עד כמה אלגוריתמי סטגנוגרפיה מודרניים עמידים לניתוח ממוחשב? האם יש דרך לזהות באופן אוטומטי אם תמונה מכילה מידע מוסתר?

תודות

אני רוצה להודות למורים שלי: עידן פלדברג, רום רפסון, עודי רז, ליאת סגל ויעקב שוצמן שהאמינו ביכולת שלי לבצע פרויקט כזה מורכב ותמכו בי לאורך כל הדרך. תודה לאבא שלי על העזרה בשלב הבדיקות ועל כל הפעמים שהקשיב לי מדבר על הפרויקט. תודה לקהילת המפתחים ברשת, שבזכותה לא הייתי צריך להמציא את הגלגל מחדש בכל פעם.

הפרויקט הזה לימד אותי הרבה יותר ממה שציפיתי, לא רק על תכנות ואבטחת מידע, אלא גם על התמדה, יצירתיות, ופתרון בעיות. אני גאה במה שהצלחתי ליצור, למרות כל האתגרים, ואני מצפה להשתמש בידע ובניסיון שרכשתי בפרויקטים עתידיים.

ביבליוגרפיה

ספר פייתון וספר רשתות של "גבהים"

Codeproject. (2021). Steganography in Python – How to hide text in an image. Codeproject. https://www.codeproject.com/Articles/5315779/Steganography-in-Python-How-to-Hide-Text-in-an-Ima

Kumar, A. (2023). Email automation in Python. Geekflare. https://geekflare.com/email-/automation-in-python

נספחים

קוד הפרויקט שלי StegnoLogic:

Server:

```
import socket
import threading
import json
import base64
import time
import os
import dotenv
import io
from PIL import Image
from DatabaseHandler import DatabaseHandler
from EmailHandler import EmailHandler
from SecurityCrypto import SecurityCrypto
from SteganographyHandler import SteganographyHandler


class Server:
    def __init__(self, host='localhost', port=5555):
        # Load environment variables
        dotenv.load_dotenv()

        # Get server settings from environment or use defaults
        self.host = os.environ.get('SERVER_HOST', host)
        self.port = int(os.environ.get('SERVER_PORT', port))

        # Socket setup
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        # Set a small timeout to prevent blocking forever in accept()
        self.server_socket.settimeout(1)

        self.running = False
        self.clients = []
        self.client_keys = {}
        self.client_ips = {}
        self.client_usernames = {}

        # DDOS protection
        self.connection_history = []
        # Increase max connections for handling multiple clients with large images
        self.max_connections_per_minute = 300

        # Initialize handlers
```

```
        self.db_handler = DatabaseHandler('StegnoLogicDatabase.db')
        self.email_handler = EmailHandler()
        self.crypto = SecurityCrypto()
        self.steg_handler = SteganographyHandler()

        # Generate RSA keys
        self.crypto.generate_keys()

        # Track active operations to prevent timeouts
        self.active_operations = {}

        print(f"Server: Initialized with host={self.host}, port={self.port}")

    def start(self):
        """Start the server and listen for connections"""
        try:
            self.server_socket.bind((self.host, self.port))
            self.server_socket.listen(10)  # Increase backlog for more concurrent connections
            self.running = True
            print(f"Server: Running on {self.host}:{self.port}")

            # Create database
            self.db_handler.create_database()

            # Accept connections
            while self.running:
                try:
                    client_socket, address = self.server_socket.accept()
                    client_ip = f"{address[0]}:{address[1]}"

                    # Check rate limit to prevent DDOS
                    if not self.crypto.limit_connection_rate(self.connection_history,
self.max_connections_per_minute):
                        print(f"Server: Rate limit exceeded from {client_ip}")
                        client_socket.close()
                        continue

                    print(f"Server: New connection from {client_ip}")

                    # Configure socket for larger data
                    client_socket.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 131072)  #
128KB receive buffer
                    client_socket.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 131072)  #
128KB send buffer

                    # Set a timeout for client operations
                    client_socket.settimeout(300)  # 5 minute timeout

                    # Add client to list and set up data structures
                    self.clients.append(client_socket)
                    self.client_ips[client_socket] = client_ip
                    print(f"Server: Connection established with {client_ip}")

                    # Handle in thread
```

```
                client_thread = threading.Thread(target=self.handle_client, args=(client_socket,
client_ip))
                client_thread.daemon = True
                client_thread.start()

            except socket.timeout:
                # This is expected due to the socket timeout
                continue
            except Exception as e:
                print(f"Server: Connection error: {e}")

        except Exception as e:
            print(f"Server: Error: {e}")
        finally:
            self.stop()

    def stop(self):
        """Stop the server and close all connections"""
        self.running = False

        # Close client connections
        for client in self.clients:
            try:
                client.close()
            except:
                pass

        # Close server socket
        try:
            self.server_socket.close()
        except:
            pass

        print("Server: Stopped")

    def handle_client(self, client_socket, client_ip):
        """Handle communication with a connected client"""
        client_crypto = SecurityCrypto()

        try:
            # Key exchange
            client_socket.sendall(self.crypto.get_public_key_pem().encode('utf-8'))
            client_public_key = client_socket.recv(4096).decode('utf-8')
            client_crypto.set_remote_public_key(client_public_key)
            self.client_keys[client_socket] = client_crypto

            print(f"Server: Secure key exchange completed with {client_ip}")

            # Handle communication
            while self.running:
                try:
                    # Receive data in chunks for large messages
                    chunks = []
                    start_time = time.time()
```

```
            while True:
                try:
                    chunk = client_socket.recv(4096).decode('utf-8')
                    if not chunk:
                        break
                    chunks.append(chunk)
                    # Check for end marker in the last 10 characters
                    if len(chunk) > 10 and "||END||" in chunk[-10:]:
                        break
                except socket.timeout:
                    # If we've received some data but now timing out, it might be complete
                    if chunks:
                        print(
                            f"Server: Socket timeout with partial data from {client_ip}, attempting to
process")
                        break
                    else:
                        # No data received at all
                        print(f"Server: Socket timeout with no data from {client_ip}")
                        raise

            # If we have no chunks, the connection is probably closed
            if not chunks:
                print(f"Server: Connection closed by client: {client_ip}")
                break

            encrypted_data = ''.join(chunks).replace("||END||", "")

            # Decrypt data
            decrypted_data = self.crypto.decrypt(encrypted_data)
            if not decrypted_data:
                print(f"Server: Decryption failed from {client_ip}")
                continue

            # Process request
            try:
                request = json.loads(decrypted_data)
                request_id = request.get('id', 'unknown')
                request_type = request.get('type', 'unknown')
                print(f"Server: Request received from {client_ip}: {request_type} (ID:
{request_id})")

                # Track this operation
                operation_key = f"{client_ip}_{request_id}"
                self.active_operations[operation_key] = time.time()

                # Process the request
                response = self.process_request(request, client_socket, client_ip)

                # Remove from active operations
                if operation_key in self.active_operations:
                    elapsed = time.time() - self.active_operations[operation_key]
                    print(
```

```
                    f"Server: Operation {request_type} (ID: {request_id}) completed in
{elapsed:.2f} seconds")
                del self.active_operations[operation_key]

            # Encrypt response
            encrypted_response = client_crypto.encrypt(json.dumps(response))

            # Add end marker for large responses
            encrypted_response_with_marker = encrypted_response + "||END||"
            client_socket.sendall(encrypted_response_with_marker.encode('utf-8'))
            print(
              f"Server: Response sent to {client_ip}: {response.get('status')} for
{request_type} (ID: {request_id})")

          except json.JSONDecodeError:
            print(f"Server: Invalid JSON from {client_ip}")
          except Exception as e:
            print(f"Server: Request processing error: {e}")
            # Try to send an error response
            try:
              error_response = {
                'status': 'error',
                'message': f"Server error: {str(e)}"
              }
              encrypted_error = client_crypto.encrypt(json.dumps(error_response))
              client_socket.sendall((encrypted_error + "||END||").encode('utf-8'))
            except:
              pass

        except socket.timeout:
          # Timeout on receive is handled above
          continue
        except socket.error as e:
          print(f"Server: Socket error with {client_ip}: {e}")
          break

    except Exception as e:
      print(f"Server: Client error with {client_ip}: {e}")
    finally:
      # Cleanup
      if client_socket in self.clients:
        self.clients.remove(client_socket)
      if client_socket in self.client_keys:
        del self.client_keys[client_socket]
      if client_socket in self.client_ips:
        del self.client_ips[client_socket]
      if client_socket in self.client_usernames:
        print(f"Server: User {self.client_usernames[client_socket]} disconnected from
{client_ip}")
        del self.client_usernames[client_socket]
      client_socket.close()
      print(f"Server: Connection closed: {client_ip}")

  def process_request(self, request, client_socket, client_ip):
```

```
        """Process client requests"""
        req_type = request.get('type')
        data = request.get('data')

        if req_type == 'login':
            username = data.get('username')
            password = data.get('password')
            user = self.db_handler.login_user(username, password, client_ip)

            if user:
                # Store username for this client
                self.client_usernames[client_socket] = username

                # Send verification code
                code = self.email_handler.send_verification_code(user[3], client_ip)
                print(f"Server: Login verification code sent to {user[3]} for user {username} from
{client_ip}")
                return {
                    'status': 'success',
                    'user': user,
                    'verification_code': code
                }
            else:
                self.db_handler.log_login_attempt(username, "Failed", client_ip)
                print(f"Server: Failed login attempt for user {username} from {client_ip}")
                return {'status': 'error', 'message': 'Invalid credentials'}

        elif req_type == 'register':
            username = data.get('username')
            name = data.get('name')
            password = data.get('password')
            email = data.get('email')

            # Check password strength
            is_valid, error_message = self.crypto.check_password_strength(password)
            if not is_valid:
                print(f"Server: Registration failed for {username}: {error_message} from
{client_ip}")
                return {'status': 'error', 'message': error_message}

            if self.db_handler.add_user(username, name, password, email, client_ip):
                print(f"Server: User registered successfully: {username} from {client_ip}")
                return {'status': 'success', 'message': 'Registered successfully'}
            else:
                print(f"Server: Registration failed: username {username} already exists. Request
from {client_ip}")
                return {'status': 'error', 'message': 'Username exists'}

        elif req_type == 'log_login':
            username = data.get('username')
            status = data.get('status')
            self.db_handler.log_login_attempt(username, status, client_ip)
            print(f"Server: Login attempt logged: {username} - {status} from {client_ip}")
            return {'status': 'success'}
```

```
    elif req_type == 'add_message':
        sender = data.get('sender')
        receiver = data.get('receiver')
        message = data.get('message')
        self.db_handler.add_message(sender, receiver, message, client_ip)
        print(f"Server: Message added from {sender} to {receiver} from {client_ip}")
        return {'status': 'success'}

    elif req_type == 'add_message_multiple':
        sender = data.get('sender')
        receivers = data.get('receivers')
        message = data.get('message')
        self.db_handler.add_message_multiple_recipients(sender, receivers, message,
client_ip)
        print(f"Server: Message added from {sender} to multiple recipients from {client_ip}")
        return {'status': 'success'}

    elif req_type == 'send_email':
        recipient = data.get('recipient')
        subject = data.get('subject')
        body = data.get('body')

        # Check if there's attachment data
        attachment_data = None
        attachment_name = None
        if 'attachment_data' in data and data['attachment_data']:
            attachment_data = base64.b64decode(data['attachment_data'])
            attachment_name = data.get('attachment_name', 'attachment.png')

        if self.email_handler.send_email(recipient, subject, body, attachment_data,
attachment_name, client_ip):
            print(f"Server: Email sent to {recipient} from {client_ip}")
            return {'status': 'success', 'message': 'Email sent'}
        else:
            print(f"Server: Failed to send email to {recipient} from {client_ip}")
            return {'status': 'error', 'message': 'Email failed'}

    elif req_type == 'send_emails_multiple':
        recipients = data.get('recipients')
        subject = data.get('subject')
        body = data.get('body')

        # Check if there's attachment data
        attachment_data = None
        attachment_name = None
        if 'attachment_data' in data and data['attachment_data']:
            attachment_data = base64.b64decode(data['attachment_data'])
            attachment_name = data.get('attachment_name', 'attachment.png')

        results = self.email_handler.send_emails_to_multiple_recipients(
            recipients, subject, body, attachment_data, attachment_name, client_ip
        )
        success_count = sum(1 for result in results.values() if result)
```

```
        print(f"Server: Sent emails to {success_count}/{len(recipients)} recipients from
{client_ip}")
        return {
          'status': 'success',
          'results': results
        }

      elif req_type == 'get_user_email':
        username = data.get('username')
        email = self.db_handler.get_user_email(username)

        if email:
          return {'status': 'success', 'email': email}
        else:
          print(f"Server: User email not found: {username}. Request from {client_ip}")
          return {'status': 'error', 'message': 'User not found'}

      elif req_type == 'get_users_emails':
        usernames = data.get('usernames')
        emails = self.db_handler.get_users_emails(usernames)

        if emails:
          return {'status': 'success', 'emails': emails}
        else:
          print(f"Server: No valid user emails found for request from {client_ip}")
          return {'status': 'error', 'message': 'No valid users'}

      elif req_type == 'check_username':
        username = data.get('username')
        exists = self.db_handler.check_username_exists(username)
        return {'status': 'success', 'exists': exists}

      elif req_type == 'calculate_max_message_length':
        # Get image data from request
        image_data = base64.b64decode(data.get('image_data'))
        max_length = self.steg_handler.calculate_max_message_length(image_data)

        return {'status': 'success', 'max_length': max_length}

      elif req_type == 'encode_image':
        # Extract data from request
        image_data = base64.b64decode(data.get('image_data'))
        message = data.get('message')
        receiver = data.get('receiver')

        # Encode the image
        encoded_image = self.steg_handler.encode_image(image_data, message, receiver,
client_ip)

        if encoded_image:
          # Return encoded image
          encoded_b64 = base64.b64encode(encoded_image).decode('utf-8')
          return {'status': 'success', 'encoded_image': encoded_b64}
        else:
```

```
            return {'status': 'error', 'message': 'Failed to encode image'}

        elif req_type == 'encode_image_multiple':
            # Extract data from request
            image_data = base64.b64decode(data.get('image_data'))
            message = data.get('message')
            receivers = data.get('receivers')
            sender = data.get('sender')

            # Check image size and resize if needed
            try:
                img = Image.open(io.BytesIO(image_data))
                width, height = img.size
                # If image is extremely large, resize it
                if width * height > 10000000:  # More than 10 million pixels
                    print(f"Server: Resizing large image ({width}x{height}) for encoding")
                    # Calculate new size, preserving aspect ratio
                    aspect = width / height
                    if width > height:
                        new_width = 3000
                        new_height = int(new_width / aspect)
                    else:
                        new_height = 3000
                        new_width = int(new_height * aspect)
                    img = img.resize((new_width, new_height), Image.LANCZOS)
                    output = io.BytesIO()
                    img.save(output, format='PNG')
                    image_data = output.getvalue()
                    print(f"Server: Image resized to {new_width}x{new_height}")
            except Exception as e:
                print(f"Server: Error preprocessing image: {e}")

            # Encode the image
            encoded_image = self.steg_handler.encode_image_multiple_recipients(image_data,
message, receivers,
                                                                               client_ip)
            if encoded_image:
                # Store the encoded image in the database
                image_id = self.db_handler.store_image(
                    f"encoded_image_{time.time()}.png",
                    encoded_image,
                    sender,
                    client_ip
                )

                # Add message to database for each recipient
                self.db_handler.add_message_multiple_recipients(
                    sender, receivers, message, client_ip
                )

                # Return encoded image and info
                encoded_b64 = base64.b64encode(encoded_image).decode('utf-8')
                return {
                    'status': 'success',
```

```
            'encoded_image': encoded_b64,
            'image_id': image_id
          }
        else:
          return {'status': 'error', 'message': 'Failed to encode image'}

      elif req_type == 'decode_image':
        # Extract data from request
        image_data = base64.b64decode(data.get('image_data'))
        current_user = data.get('username')

        # Decode the image
        hidden_message = self.steg_handler.decode_image(image_data, current_user,
client_ip)

        if hidden_message:
          return {
            'status': 'success',
            'message': hidden_message
          }
        else:
          return {'status': 'error', 'message': 'No message found or not authorized'}

      else:
        print(f"Server: Unknown request type: {req_type} from {client_ip}")
        return {'status': 'error', 'message': 'Unknown request type'}

def start_server():
  """Initialize and start the StegnoLogic server"""
  server = Server()

  # Show server banner
  print("Starting StegnoLogic Server...")
  print("=" * 50)
  print("  _______ _            _   ___          _      ")
  print(" / _____| |          | | / /         (_)     ")
  print("| (___ | |_ ____  ___ _| |/ /___   ____   _  ____ ")
  print(" \\____ \\| __/ _ \\/ _ `|   / _ \\ / _ \\| |/ __|")
  print(" _____) | ||  __/ (_| | |\\ \\ (_) | (_) | |\\__ \\")
  print("|_______/ \\__\\____|\\__,_|_| \\_\\____/ \\____/|_||____/")
  print("                 __/ |                      ")
  print("                |___/                       ")
  print("=" * 50)
  print("Secure Steganography Server")
  print("by Eitan Shtamler")
  print("=" * 50)

  # Simulate loading
  for i in range(1, 6):
    print(f"Server: Initializing server components... {i * 20}%")
    time.sleep(0.5)

  print("Server: Started successfully!")
  print("Server: Optimized for large image processing")
```

```
    print("=" * 50)

    # Start the actual server
    server.start()

if __name__ == "__main__":
    start_server()
```

client:

```
import socket
import json
import threading
import base64
import time
import os
import dotenv
import io
from PIL import Image
from SecurityCrypto import SecurityCrypto


class Client:
    def __init__(self, host=None, port=None):
        # Load environment variables
        dotenv.load_dotenv()

        # Get server settings from environment or use defaults
        if host is None:
            host = os.environ.get('SERVER_HOST', 'localhost')
        if port is None:
            port = int(os.environ.get('SERVER_PORT', 5555))

        self.host = host
        self.port = port

        # Socket setup
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.socket.settimeout(300)  # Increase timeout to 5 minutes for large images
        self.connected = False
        self.crypto = SecurityCrypto()

        self.current_user = None
        self.request_id = 0  # For tracking requests

    def connect(self):
        """Connect to server"""
        try:
            print(f"Client: Attempting to connect to server at {self.host}:{self.port}...")

            # Connect to server
            self.socket.connect((self.host, self.port))
            print(f"Client: Socket connection established")

            # Generate RSA keys for this session
```

```
        self.crypto.generate_keys()
        print(f"Client: RSA keys generated")

        # Exchange public keys
        server_public_key = self.socket.recv(4096).decode('utf-8')
        self.crypto.set_remote_public_key(server_public_key)
        print(f"Client: Received server public key")

        self.socket.sendall(self.crypto.get_public_key_pem().encode('utf-8'))
        print(f"Client: Sent client public key")

        self.connected = True
        self.crypto.secure_connection()
        print(f"Client: Connection established with server: {self.host}:{self.port}")
        return True
    except socket.error as e:
        print(f"Client: Connection error: {e}")
        return False

def send_request(self, request_type, data):
    """Send encrypted request to server"""
    if not self.connected:
        print("Client: Not connected to server")
        raise ConnectionError("Not connected to server")

    # Add request ID for tracking
    self.request_id += 1
    request = {
        'type': request_type,
        'data': data,
        'id': self.request_id  # Add request ID
    }

    try:
        # Encrypt and send request
        request_json = json.dumps(request)
        encrypted_request = self.crypto.encrypt(request_json)

        # Add end marker for large data
        encrypted_request_with_marker = encrypted_request + "||END||"
        self.socket.sendall(encrypted_request_with_marker.encode('utf-8'))
        print(f"Client: Request sent to server: {request_type} (ID: {self.request_id})")

        # Get and decrypt response - using chunking for large responses
        chunks = []
        while True:
            try:
                chunk = self.socket.recv(4096).decode('utf-8')
                if not chunk:
                    break
                chunks.append(chunk)
                # Check for end marker in the last 10 characters
                if len(chunk) > 10 and "||END||" in chunk[-10:]:
                    break
```

```
            except socket.timeout:
                # If we get a timeout but have received some data, use what we have
                if chunks:
                    print(f"Client: Socket timeout but data received, processing available chunks")
                    break
                else:
                    print(f"Client: Socket timeout with no data received")
                    raise TimeoutError("Request timeout - no response from server")

        encrypted_response = ''.join(chunks).replace("||END||", "")

        if not encrypted_response:
            print("Client: Empty response received")
            raise Exception("Empty response received from server")

        decrypted_response = self.crypto.decrypt(encrypted_response)
        if not decrypted_response:
            print("Client: Failed to decrypt server response")
            raise Exception("Failed to decrypt server response")

        response = json.loads(decrypted_response)
        print(
            f"Client: Response received from server: {response.get('status')} for request
{request_type} (ID: {self.request_id})")
        return response
    except socket.timeout:
        print(f"Client: Request timeout - no response from server for {request_type} (ID:
{self.request_id})")
        raise TimeoutError(f"Request timeout - no response from server for {request_type}")
    except json.JSONDecodeError as e:
        print(f"Client: JSON decode error: {e} for {request_type} (ID: {self.request_id})")
        # Try to handle partial or corrupted JSON
        if encrypted_response and len(encrypted_response) > 20:
            print(f"Client: Attempting to recover from JSON error")
            try:
                # Try to clean up the JSON
                # Find the last valid JSON character
                last_brace_pos = encrypted_response.rfind('}')
                if last_brace_pos > 0:
                    cleaned_response = encrypted_response[: last_brace_pos + 1]
                    decrypted_response = self.crypto.decrypt(cleaned_response)
                    if decrypted_response:
                        response = json.loads(decrypted_response)
                        print(f"Client: Successfully recovered response from partial data")
                        return response
            except:
                pass
        raise e
    except Exception as e:
        print(f"Client: Request error: {e} for {request_type} (ID: {self.request_id})")
        raise e

def close(self):
    """Close connection"""
```

```
    if self.connected:
        try:
            self.socket.close()
            print("Client: Connection to server closed")
        except:
            pass
        self.connected = False

def register_user(self, username, name, password, email):
    """Register a new user"""
    response = self.send_request('register', {
        'username': username,
        'name': name,
        'password': password,
        'email': email
    })
    return response.get('status') == 'success', response.get('message', 'Unknown error')

def login_user(self, username, password):
    """Login a user and receive verification code"""
    response = self.send_request('login', {
        'username': username,
        'password': password
    })

    if response.get('status') == 'success':
        self.current_user = username
        return True, response.get('user'), response.get('verification_code')
    else:
        return False, None, None

def log_login_attempt(self, username, status):
    """Log login attempt"""
    self.send_request('log_login', {
        'username': username,
        'status': status
    })

def check_username_exists(self, username):
    """Check if a username exists"""
    response = self.send_request('check_username', {
        'username': username
    })
    return response.get('exists', False)

def get_user_email(self, username):
    """Get a user's email"""
    response = self.send_request('get_user_email', {
        'username': username
    })
    if response.get('status') == 'success':
        return response.get('email')
    return None
```

```
    def get_users_emails(self, usernames):
        """Get emails for multiple users"""
        response = self.send_request('get_users_emails', {
            'usernames': usernames
        })
        if response.get('status') == 'success':
            return response.get('emails', {})
        return {}

    def send_email(self, recipient, subject, body, attachment_path=None):
        """Send an email"""
        data = {
            'recipient': recipient,
            'subject': subject,
            'body': body
        }

        # Add attachment if provided
        if attachment_path:
            try:
                with open(attachment_path, 'rb') as f:
                    image_data = base64.b64encode(f.read()).decode('utf-8')
                data['attachment_data'] = image_data
                data['attachment_name'] = os.path.basename(attachment_path)
            except Exception as e:
                print(f"Client: Failed to read attachment: {e}")

        response = self.send_request('send_email', data)
        return response.get('status') == 'success'

    def send_emails_to_multiple_recipients(self, recipients, subject, body,
attachment_path=None):
        """Send emails to multiple recipients"""
        data = {
            'recipients': recipients,
            'subject': subject,
            'body': body
        }

        # Add attachment if provided
        if attachment_path:
            try:
                with open(attachment_path, 'rb') as f:
                    image_data = base64.b64encode(f.read()).decode('utf-8')
                data['attachment_data'] = image_data
                data['attachment_name'] = os.path.basename(attachment_path)
            except Exception as e:
                print(f"Client: Failed to read attachment: {e}")

        response = self.send_request('send_emails_multiple', data)
        if response.get('status') == 'success':
            return response.get('results', {})
        return {}
```

```
    def calculate_max_message_length(self, image_path):
        """Calculate maximum message length for an image"""
        try:
            # Check file size first
            file_size = os.path.getsize(image_path)
            if file_size > 10 * 1024 * 1024:  # > 10MB
                print(f"Client: Large image detected ({file_size // 1024 // 1024}MB), using estimated
max length")
                # Estimate max length instead of sending the entire image
                # A very conservative estimate: roughly 1/8 of the number of pixels × 3 × 0.9
                with Image.open(image_path) as img:
                    width, height = img.size
                    pixels = width * height
                    max_length = int(pixels * 3 / 8 * 0.9 * 0.95)  # 5% safety margin
                    print(f"Client: Estimated max message length: {max_length}")
                    return max_length

            # For smaller images, use the server to calculate
            with open(image_path, 'rb') as f:
                image_data = base64.b64encode(f.read()).decode('utf-8')

            response = self.send_request('calculate_max_message_length', {
                'image_data': image_data
            })

            if response.get('status') == 'success':
                return response.get('max_length', 0)
            return 0
        except Exception as e:
            print(f"Client: Error calculating max message length: {e}")
            return 0

    def encode_and_send(self, image_path, message, recipients):
        """Encode message in image and send to recipients"""
        try:
            start_time = time.time()
            print(f"Client: Starting image encoding process")

            # Check file size for optimization
            file_size = os.path.getsize(image_path)
            print(f"Client: Image size: {file_size // 1024}KB")

            # For very large images, resize before sending
            if file_size > 20 * 1024 * 1024:  # 20MB
                print(f"Client: Large image detected, resizing before encoding")
                resized_path = self._resize_image(image_path)
                if resized_path:
                    image_path = resized_path
                    print(f"Client: Image resized to {os.path.getsize(image_path) // 1024}KB")

            # Read image file
            with open(image_path, 'rb') as f:
                image_data = base64.b64encode(f.read()).decode('utf-8')
```

```
        # Send encode request to server
        response = self.send_request('encode_image_multiple', {
            'image_data': image_data,
            'message': message,
            'receivers': recipients,
            'sender': self.current_user
        })

        elapsed_time = time.time() - start_time
        print(f"Client: Encoding completed in {elapsed_time:.2f} seconds")

        if response.get('status') == 'success':
            # Save encoded image locally for reference
            encoded_image_data = base64.b64decode(response.get('encoded_image'))
            output_path = "encoded_image.png"
            with open(output_path, 'wb') as f:
                f.write(encoded_image_data)

            # Return success info
            return True, output_path, response.get('image_id')
        else:
            return False, None, None
    except Exception as e:
        print(f"Client: Error encoding image: {e}")
        return False, None, None

def _resize_image(self, image_path, max_size=(1920, 1080)):
    """Resize large images to reduce processing time"""
    try:
        # Create output filename
        filename, ext = os.path.splitext(image_path)
        resized_path = f"{filename}_resized{ext}"

        # Open and resize image
        with Image.open(image_path) as img:
            original_size = img.size

            # Calculate aspect ratio
            width, height = original_size
            aspect = width / height

            # Determine new size maintaining aspect ratio
            if width > height:
                new_width = min(width, max_size[0])
                new_height = int(new_width / aspect)
            else:
                new_height = min(height, max_size[1])
                new_width = int(new_height * aspect)

            # Only resize if image is actually larger than max_size
            if width > max_size[0] or height > max_size[1]:
                print(f"Client: Resizing image from {original_size} to ({new_width},
{new_height})")
                resized_img = img.resize((new_width, new_height), Image.LANCZOS)
```

```
                resized_img.save(resized_path)
                return resized_path

            # If no resize needed, return original
            return image_path
        except Exception as e:
            print(f"Client: Error resizing image: {e}")
            return image_path

    def decode_image(self, image_path):
        """Decode a message from an image"""
        try:
            start_time = time.time()
            print(f"Client: Starting image decoding process")

            # Read image file
            with open(image_path, 'rb') as f:
                image_data = base64.b64encode(f.read()).decode('utf-8')

            # Send decode request to server
            response = self.send_request('decode_image', {
                'image_data': image_data,
                'username': self.current_user
            })

            elapsed_time = time.time() - start_time
            print(f"Client: Decoding completed in {elapsed_time:.2f} seconds")

            if response.get('status') == 'success':
                return True, response.get('message')
            else:
                return False, response.get('message', 'Failed to decode image')
        except TimeoutError:
            print(f"Client: Timeout while decoding image")
            return False, "Operation timed out. The image might be too large or does not contain a
valid message."
        except Exception as e:
            print(f"Client: Error decoding image: {e}")
            return False, f"Error: {str(e)}"


def start_client():
    """Initialize and start the StegnoLogic client"""
    client = Client()

    try:
        # Connect to server
        if not client.connect():
            print("Client: Cannot start without server connection")
            return

        # Import here to avoid circular imports
        from Gui_app import Gui_app
```

```
        # Start GUI
        app = Gui_app(client)

        # Handle closing
        def on_closing():
            client.close()
            app.destroy()

        app.protocol("WM_DELETE_WINDOW", on_closing)
        app.mainloop()

    except Exception as e:
        print(f"Client: Error starting client: {e}")
        if client.connected:
            client.close()


if __name__ == "__main__":
    start_client()
```

SecurityCrypto

```
import re
import time
import rsa
import base64
import hashlib
import os
import socket
import random


class SecurityCrypto:
    """Combined security and cryptography functionality with improved handling for large messages"""

    def __init__(self):
        self.is_secure = False
        self.public_key = None
        self.private_key = None
        self.remote_public_key = None
        self.chunk_size = 200

    def secure_connection(self):
        self.is_secure = True
        return self.is_secure

    def check_security(self):
        return self.is_secure

    def limit_connection_rate(self, history=None, max_per_minute=60):
        # Basic rate limiting to prevent DDOS
        if history is None:
            return True

        current_time = time.time()
        cutoff = current_time - 60
        recent_history = [t for t in history if t > cutoff]

        if len(recent_history) >= max_per_minute:
            return False

        recent_history.append(current_time)
        history[:] = recent_history
        return True

    @staticmethod
    def check_password_strength(password):
        # Enforce strong password policy
        if len(password) < 8:
            return False, "Password must be at least 8 characters long"

        if not re.search(r'[A-Z]', password):
            return False, "Password must contain at least one uppercase letter"
```

```
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter"

    if not re.search(r'[0-9]', password):
        return False, "Password must contain at least one number"

    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
        return False, "Password must contain at least one special character"

    return True, "Password is strong"

def generate_keys(self, key_size=2048):
    # Generate RSA key pair
    (self.public_key, self.private_key) = rsa.newkeys(key_size)
    return self.public_key

def set_remote_public_key(self, key_data):
    # Set remote party's public key
    if isinstance(key_data, bytes):
        key_data = key_data.decode('utf-8')
    self.remote_public_key = rsa.PublicKey.load_pkcs1(key_data.encode('utf-8'))
    return True

def get_public_key_pem(self):
    # Return our public key in PEM format
    if not self.public_key:
        self.generate_keys()
    return self.public_key.save_pkcs1().decode('utf-8')

def encrypt(self, message):
    # Encrypt a message using remote's public key
    if not self.remote_public_key:
        raise ValueError("Remote public key not set")

    if isinstance(message, str):
        message = message.encode('utf-8')

    if len(message) > self.chunk_size:  # RSA size limitation
        return self._encrypt_large_message(message)
    else:
        try:
            encrypted = rsa.encrypt(message, self.remote_public_key)
            return base64.b64encode(encrypted).decode('utf-8')
        except Exception as e:
            print(f"Encryption error: {e}")
            # Try with smaller chunk as fallback
            return self._encrypt_large_message(message, chunk_size=self.chunk_size - 50)

def _encrypt_large_message(self, message, chunk_size=None):
    # Handle messages larger than RSA can encrypt in one go
    if chunk_size is None:
        chunk_size = self.chunk_size

    chunks = [message[i:i + chunk_size] for i in range(0, len(message), chunk_size)]
```

```
    encrypted_chunks = []

    for i, chunk in enumerate(chunks):
        try:
            encrypted = rsa.encrypt(chunk, self.remote_public_key)
            encrypted_chunks.append(base64.b64encode(encrypted).decode('utf-8'))

            # Log progress for very large messages
            if len(chunks) > 100 and i % 50 == 0:
                progress = min(100, int((i / len(chunks)) * 100))
                print(f"Encryption progress: {progress}%")

        except Exception as e:
            print(f"Chunk encryption error at chunk {i}: {e}")
            if chunk_size > 100:
                # Retry with smaller chunk size
                print(f"Retrying with smaller chunk size: {chunk_size - 50}")
                return self._encrypt_large_message(message, chunk_size=chunk_size - 50)
            else:
                raise

    # Add chunk count for verification
    chunk_count = f"COUNT:{len(encrypted_chunks)}:"
    return chunk_count + "||CHUNK||".join(encrypted_chunks)

def decrypt(self, encrypted_message):
    # Decrypt a message using our private key
    if not self.private_key:
        raise ValueError("Private key not set")

    # Check if this is a chunked message with count prefix
    if encrypted_message.startswith("COUNT:"):
        try:
            # Extract count
            count_end = encrypted_message.find(":", 6)
            count = int(encrypted_message[6:count_end])
            encrypted_message = encrypted_message[count_end + 1:]

            # Verify chunk count
            chunks = encrypted_message.split("||CHUNK||")
            if len(chunks) != count:
                print(f"Warning: Expected {count} chunks but received {len(chunks)}")
        except Exception as e:
            print(f"Error parsing chunk count: {e}")

    if "||CHUNK||" in encrypted_message:
        return self._decrypt_large_message(encrypted_message)

    try:
        encrypted_data = base64.b64decode(encrypted_message)
        return rsa.decrypt(encrypted_data, self.private_key).decode('utf-8')
    except Exception as e:
        print(f"Decryption error: {e}")
        # Try a more lenient approach for partially corrupted data
```

```
        try:
            # Clean up the base64 string by ensuring its length is a multiple of 4
            padded = encrypted_message + '=' * (4 - len(encrypted_message) % 4) if len(
                encrypted_message) % 4 else encrypted_message
            encrypted_data = base64.b64decode(padded)
            return rsa.decrypt(encrypted_data, self.private_key).decode('utf-8')
        except:
            # If still failing, return None
            return None

    def _decrypt_large_message(self, encrypted_message):
        # Handle decryption of large messages that were split into chunks
        chunks = encrypted_message.split("||CHUNK||")
        decrypted_chunks = []

        for i, chunk in enumerate(chunks):
            try:
                # Skip empty chunks
                if not chunk:
                    continue

                encrypted_data = base64.b64decode(chunk)
                decrypted = rsa.decrypt(encrypted_data, self.private_key)
                decrypted_chunks.append(decrypted)

                # Log progress for very large messages
                if len(chunks) > 100 and i % 50 == 0:
                    progress = min(100, int((i / len(chunks)) * 100))
                    print(f"Decryption progress: {progress}%")

            except Exception as e:
                print(f"Chunk decryption error at chunk {i}: {e}")
                # Continue with other chunks despite errors
                continue

        try:
            return b''.join(decrypted_chunks).decode('utf-8')
        except UnicodeDecodeError as e:
            # Try to salvage partial data if possible
            print(f"Unicode decode error: {e}")
            partial_data = b''.join(decrypted_chunks)

            # Try to decode as much as possible
            valid_bytes = []
            for b in partial_data:
                if b < 128:  # ASCII range
                    valid_bytes.append(b)

            if valid_bytes:
                return bytes(valid_bytes).decode('utf-8', errors='replace')
            return None

    @staticmethod
    def hash_password(password, salt=None):
```

```
        # Create a secure hash of a password
        if salt is None:
            salt = os.urandom(32)

        if isinstance(password, str):
            password = password.encode('utf-8')

        hash_obj = hashlib.pbkdf2_hmac('sha256', password, salt, 100000)
        return base64.b64encode(salt + hash_obj).decode('utf-8')

    @staticmethod
    def verify_password(hashed_password, password_to_check):
        # Verify a password against its secure hash
        if isinstance(hashed_password, str):
            hashed_password = base64.b64decode(hashed_password)

        if isinstance(password_to_check, str):
            password_to_check = password_to_check.encode('utf-8')

        salt = hashed_password[:32]
        stored_hash = hashed_password[32:]

        hash_to_check = hashlib.pbkdf2_hmac('sha256', password_to_check, salt, 100000)
        return stored_hash == hash_to_check

    def create_secure_socket(self):
        # Create a socket with better buffer sizes for large data
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 262144)  # 256KB receive
buffer
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 262144)  # 256KB send
buffer
        self.secure_connection()
        return sock

    @staticmethod
    def generate_random_id(length=16):
        """Generate a random ID string for tracking requests"""
        alphabet =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
        return ''.join(random.choice(alphabet) for _ in range(length))
```

SteganographyHandler:

```
from PIL import Image
import io
import time


class SteganographyHandler:
    @staticmethod
    def calculate_max_message_length(image_data):
        """Calculate the maximum message size that can be hidden in the image"""
        try:
            # Create image from binary data
            img = Image.open(io.BytesIO(image_data))
            pixels = img.width * img.height
            # Each pixel can store 3 bits (RGB), 8 bits per character
            # Using 90% to leave room for metadata
            max_length = int(pixels * 3 / 8 * 0.9)
            print(f"Server: Max message length for image: {max_length} characters")
            return max_length
        except Exception as e:
            print(f"Server: Error calculating max message length: {e}")
            return 0

    @staticmethod
    def encode_image(image_data, message, receiver_username, client_ip="unknown"):
        """Hide a message in an image for a specific recipient"""
        try:
            start_time = time.time()
            print(f"Server: Encoding message for {receiver_username} for client {client_ip}")
            img = Image.open(io.BytesIO(image_data))

            # Prepend the receiver username to the message
            full_message = f"{receiver_username}:{message}"
            binary_message = ''.join(format(ord(char), '08b') for char in full_message)
            binary_message += '1111111111111110'  # End of message marker

            # Get image pixels - convert to list for faster access
            pixels = list(img.getdata())
            if len(binary_message) > len(pixels) * 3:
                print(
                    f"Server: Message too long for the image. Message bits: {len(binary_message)},
available bits: {len(pixels) * 3}")
                raise ValueError("Message too long for the image")

            # Embed the message bits in batches for better performance
            index = 0
            batch_size = 10000  # Process 10,000 pixels at a time
            pixel_count = len(pixels)

            for start_idx in range(0, pixel_count, batch_size):
                end_idx = min(start_idx + batch_size, pixel_count)

                for i in range(start_idx, end_idx):
                    pixel = list(pixels[i])
```

```
                for j in range(3):  # RGB channels
                    if index < len(binary_message):
                        # Replace the least significant bit
                        pixel[j] = pixel[j] & ~1 | int(binary_message[index])
                        index += 1
                pixels[i] = tuple(pixel)

            # Progress reporting for large images
            if pixel_count > 100000:  # Only log progress for large images
                progress = min(100, int((end_idx / pixel_count) * 100))
                if progress % 10 == 0:  # Report every 10%
                    print(f"Server: Encoding progress: {progress}%")

            # Break early if we've embedded all the message
            if index >= len(binary_message):
                break

        # Create a new image with the modified pixels
        encoded_img = Image.new(img.mode, img.size)
        encoded_img.putdata(pixels)

        # Save image to bytes with optimized settings based on image size
        output_bytes = io.BytesIO()
        if img.width * img.height > 1000000:  # For large images (>1MP)
            # Use optimize for PNG or quality=90 for JPEG to reduce size
            encoded_img.save(output_bytes, format='PNG', optimize=True)
        else:
            encoded_img.save(output_bytes, format='PNG')

        elapsed_time = time.time() - start_time
        print(f"Server: Image encoded successfully for {receiver_username} in
{elapsed_time:.2f} seconds")
        return output_bytes.getvalue()
    except Exception as e:
        print(f"Server: Error encoding image: {e}")
        return None

@staticmethod
def encode_image_multiple_recipients(image_data, message, receiver_usernames,
client_ip="unknown"):
    """Hide a message in an image for multiple recipients"""
    try:
        start_time = time.time()
        print(f"Server: Encoding message for multiple recipients for client {client_ip}")
        img = Image.open(io.BytesIO(image_data))

        # Join receiver usernames with a separator
        receivers_string = "|".join(receiver_usernames)
        full_message = f"{receivers_string}:{message}"

        # Convert message to binary
        binary_message = ''.join(format(ord(char), '08b') for char in full_message)
        binary_message += '1111111111111110'  # End of message marker
```

```
        # Get image pixels - convert to list for faster access
        pixels = list(img.getdata())
        if len(binary_message) > len(pixels) * 3:
            print(
                f"Server: Message too long for the image. Message bits: {len(binary_message)},
available bits: {len(pixels) * 3}")
            raise ValueError("Message too long for the image")

        # Embed the message bits in batches for better performance
        index = 0
        batch_size = 10000  # Process 10,000 pixels at a time
        pixel_count = len(pixels)

        for start_idx in range(0, pixel_count, batch_size):
            end_idx = min(start_idx + batch_size, pixel_count)

            for i in range(start_idx, end_idx):
                pixel = list(pixels[i])
                for j in range(3):  # RGB channels
                    if index < len(binary_message):
                        # Replace the least significant bit
                        pixel[j] = pixel[j] & ~1 | int(binary_message[index])
                        index += 1
                pixels[i] = tuple(pixel)

            # Progress reporting for large images
            if pixel_count > 100000:  # Only log progress for large images
                progress = min(100, int((end_idx / pixel_count) * 100))
                if progress % 10 == 0:  # Report every 10%
                    print(f"Server: Encoding progress: {progress}%")

            # Break early if we've embedded all the message
            if index >= len(binary_message):
                break

        # Create a new image with the modified pixels
        encoded_img = Image.new(img.mode, img.size)
        encoded_img.putdata(pixels)

        # Save image to bytes with optimized settings based on image size
        output_bytes = io.BytesIO()
        if img.width * img.height > 1000000:  # For large images (>1MP)
            # Use optimize for PNG or quality=90 for JPEG to reduce size
            encoded_img.save(output_bytes, format='PNG', optimize=True)
        else:
            encoded_img.save(output_bytes, format='PNG')

        elapsed_time = time.time() - start_time
        print(
            f"Server: Image encoded successfully for {len(receiver_usernames)} recipients in
{elapsed_time:.2f} seconds")
        return output_bytes.getvalue()
    except Exception as e:
        print(f"Server: Error encoding image for multiple recipients: {e}")
```

```
        return None

    @staticmethod
    def decode_image(image_data, current_user, client_ip="unknown"):
        """Extract a hidden message from an image if intended for the current user"""
        try:
            start_time = time.time()
            print(f"Server: Attempting to decode image for user {current_user} for client
{client_ip}")
            img = Image.open(io.BytesIO(image_data))
            pixels = list(img.getdata())

            # Extract binary message in batches for better performance
            binary_message = ''
            found_end_marker = False
            batch_size = 10000  # Process 10,000 pixels at a time
            pixel_count = len(pixels)

            for start_idx in range(0, pixel_count, batch_size):
                end_idx = min(start_idx + batch_size, pixel_count)

                for i in range(start_idx, end_idx):
                    pixel = pixels[i]
                    for value in pixel[:3]:  # RGB channels
                        binary_message += str(value & 1)  # Get the LSB

                    # Check for end marker periodically to avoid processing the whole image
                    if len(binary_message) % 1000 == 0 and len(binary_message) >= 16:
                        if binary_message[-16:] == '1111111111111110':
                            binary_message = binary_message[:-16]  # Remove end marker
                            found_end_marker = True
                            break

                # Break early if we found the end marker
                if found_end_marker:
                    break

                # Progress reporting for large images
                if pixel_count > 100000:  # Only log progress for large images
                    progress = min(100, int((end_idx / pixel_count) * 100))
                    if progress % 10 == 0:  # Report every 10%
                        print(f"Server: Decoding progress: {progress}%")

            # If we didn't find an end marker but processed the whole image,
            # check one last time at the end
            if not found_end_marker and len(binary_message) >= 16:
                if binary_message[-16:] == '1111111111111110':
                    binary_message = binary_message[:-16]  # Remove end marker
                    found_end_marker = True

            # Convert binary to text (stopping at the first invalid character)
            message = ''
            for i in range(0, len(binary_message), 8):
                if i + 8 > len(binary_message):
```

```
            break

        byte = binary_message[i: i + 8]
        try:
            char = chr(int(byte, 2))
            # Stop at non-printable characters (except common whitespace)
            if not (32 <= ord(char) <= 126 or char in '\n\r\t'):
                break
            message += char
        except ValueError:
            break

    # Parse message to check the recipient
    if ':' in message:
        receivers_part, actual_message = message.split(':', 1)

        # Check if this is a multi-recipient message
        if '|' in receivers_part:
            receivers = receivers_part.split('|')
            if current_user in receivers:
                elapsed_time = time.time() - start_time
                print(f"Server: Message successfully decoded for {current_user} in
{elapsed_time: .2f} seconds")
                return actual_message
        # Check if this is a single-recipient message
        elif receivers_part == current_user:
            elapsed_time = time.time() - start_time
            print(f"Server: Message successfully decoded for {current_user} in
{elapsed_time: .2f} seconds")
            return actual_message

    print(f"Server: No message found for user {current_user} or message is corrupted for
client {client_ip}")
    return None
except Exception as e:
    print(f"Server: Error decoding image: {e}")
    return None
```

EmailHandler

```
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
import os
import random
import dotenv
import threading
import ssl


class EmailHandler:
    def __init__(self):
        # Load email credentials from environment variables
        dotenv.load_dotenv()
        self.from_email = os.environ.get("EMAIL_ADDRESS", "")
        self.from_password = os.environ.get("EMAIL_PASSWORD", "")

        if not self.from_email or not self.from_password:
            print("Server: WARNING: Email credentials not found in environment variables.")
            print("Server: Please set EMAIL_ADDRESS and EMAIL_PASSWORD environment
variables.")
            print("Server: Email functionality will not work without these credentials.")

    def send_email(self, recipient, subject, body, attachment_data=None,
attachment_name=None, client_ip="unknown"):
        """Send an email with optional attachment"""
        msg = MIMEMultipart()
        msg['From'] = self.from_email
        msg['To'] = recipient
        msg['Subject'] = subject
        msg.attach(MIMEText(body, 'plain'))

        # Add attachment if provided
        if attachment_data is not None and attachment_name is not None:
            try:
                part = MIMEBase("application", "octet-stream")
                part.set_payload(attachment_data)
                encoders.encode_base64(part)
                part.add_header(
                    "Content-Disposition",
                    f"attachment; filename= {attachment_name}",
                )
                msg.attach(part)
                print(f"Server: Attached file: {attachment_name} for email to {recipient}")
            except Exception as e:
                print(f"Server: Failed to attach file to email: {e}")

        try:
            # Send email through Gmail SMTP
            print(f"Server: Attempting to send email to {recipient} for client {client_ip}")
            context = ssl.create_default_context()
```

```
            server = smtplib.SMTP('smtp.gmail.com', 587)
            server.starttls(context=context)
            server.login(self.from_email, self.from_password)
            server.sendmail(self.from_email, recipient, msg.as_string())
            server.quit()
            print(f"Server: Email sent successfully to {recipient} for client {client_ip}")
            return True
        except Exception as e:
            print(f"Server: Failed to send email to {recipient}: {e}")
            return False

    def send_emails_to_multiple_recipients(self, recipients, subject, body,
attachment_data=None, attachment_name=None,
                                           client_ip="unknown"):
        """Send emails to multiple recipients in parallel"""
        results = {}
        threads = []

        print(f"Server: Sending emails to {len(recipients)} recipients for client {client_ip}")

        # Create a thread for each recipient
        for username, email in recipients.items():
            thread = threading.Thread(
                target=self._send_email_thread_with_result,
                args=(username, email, subject, body, attachment_data, attachment_name, results,
client_ip)
            )
            threads.append(thread)
            thread.start()

        # Wait for all threads to complete
        for thread in threads:
            thread.join()

        success_count = sum(1 for success in results.values() if success)
        print(f"Server: Email sending completed: {success_count}/{len(recipients)} successful
for client {client_ip}")
        return results

    def _send_email_thread_with_result(self, username, email, subject, body, attachment_data,
attachment_name, results,
                                       client_ip):
        """Helper method for multithreaded email sending"""
        success = self.send_email(email, subject, body, attachment_data, attachment_name,
client_ip)
        results[username] = success

    def generate_verification_code(self):
        """Generate a random verification code"""
        code = str(random.randint(100000, 999999))
        return code

    def send_verification_code(self, email, client_ip="unknown"):
        """Send a verification code to the provided email"""
```

```
    code = self.generate_verification_code()
    subject = "Your StegnoLogic Verification Code"
    body = f"Your verification code is: {code}\n\nThis code will expire in 5 minutes."
    if self.send_email(email, subject, body, client_ip=client_ip):
      print(f"Server: Verification code sent to {email} for client {client_ip}")
      return code
    else:
      print(f"Server: Failed to send verification code to {email} for client {client_ip}")
      return None
```

DatabaseHandler

```
import sqlite3
from SecurityCrypto import SecurityCrypto
import io
from PIL import Image


class DatabaseHandler:
  def __init__(self, db_name='StegnoLogicDatabase.db'):
    self.db_name = db_name
    self.crypto = SecurityCrypto()
    self.create_database()

  def create_database(self):
    """Initialize the database and create necessary tables if they don't exist"""
    conn = sqlite3.connect(self.db_name)

    # Enable WAL mode for better concurrent access and performance
    conn.execute("PRAGMA journal_mode = WAL")
    # Optimize database for better performance with large data
    conn.execute("PRAGMA synchronous = NORMAL")  # Less synchronous for better
performance
    conn.execute("PRAGMA cache_size = 10000")  # Larger cache for better performance

    c = conn.cursor()

    # Users table
    c.execute('''CREATE TABLE IF NOT EXISTS users
            (username TEXT PRIMARY KEY, name TEXT, password TEXT, email
TEXT)''')

    # Messages table
    c.execute('''CREATE TABLE IF NOT EXISTS messages
            (id INTEGER PRIMARY KEY AUTOINCREMENT, sender_username TEXT,
receiver_username TEXT,
             timestamp TEXT, hidden_message TEXT,
             FOREIGN KEY (sender_username) REFERENCES users(username),
             FOREIGN KEY (receiver_username) REFERENCES users(username))''')

    # Logs table
    c.execute('''CREATE TABLE IF NOT EXISTS logs
            (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT,
timestamp TEXT, status TEXT,
             client_ip TEXT, FOREIGN KEY (username) REFERENCES users(username))''')
```

```
        # Images table
        c.execute('''CREATE TABLE IF NOT EXISTS images
                     (id INTEGER PRIMARY KEY AUTOINCREMENT, image_name TEXT,
image_data BLOB,
                      sender_username TEXT, creation_date TEXT, width INTEGER, height
INTEGER,
                      compressed BOOLEAN DEFAULT 0, original_size INTEGER,
                      FOREIGN KEY (sender_username) REFERENCES users(username))''')

        conn.commit()
        conn.close()
        print(f"Server: Database initialized at {self.db_name}")

    def add_user(self, username, name, password, email, client_ip="unknown"):
        """Add a new user to the database"""
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        try:
            # Hash password before storing
            hashed_password = self.crypto.hash_password(password)
            c.execute("INSERT INTO users (username, name, password, email) VALUES (?, ?, ?,
?)",
                      (username, name, hashed_password, email))
            conn.commit()
            print(f"Server: User '{username}' added to database from {client_ip}")
            return True
        except sqlite3.IntegrityError:
            print(f"Server: Failed to add user '{username}' - username already exists. Request from
{client_ip}")
            return False
        finally:
            conn.close()

    def login_user(self, username, password, client_ip="unknown"):
        """Validate user credentials and return user data if valid"""
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        c.execute("SELECT * FROM users WHERE username=?", (username,))
        user = c.fetchone()
        conn.close()

        if user and self.crypto.verify_password(user[2], password):
            print(f"Server: User '{username}' login successful from {client_ip}")
            return user

        print(f"Server: Failed login attempt for user '{username}' from {client_ip}")
        return None

    def log_login_attempt(self, username, status, client_ip="unknown"):
        """Record login attempts for security and auditing"""
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        c.execute("INSERT INTO logs (username, timestamp, status, client_ip) VALUES (?,
```

```
datetime('now'), ?, ?)",
                (username, status, client_ip))
        conn.commit()
        conn.close()
        print(f"Server: Login attempt logged for user '{username}' with status: {status} from
{client_ip}")

    def add_message(self, sender_username, receiver_username, hidden_message,
client_ip="unknown"):
        """Store a message for a single recipient"""
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        c.execute(
            "INSERT INTO messages (sender_username, receiver_username, timestamp,
hidden_message) VALUES (?, ?, datetime('now'), ?)",
            (sender_username, receiver_username, hidden_message))
        conn.commit()
        conn.close()
        print(f"Server: Message added from '{sender_username}' to '{receiver_username}' via
{client_ip}")

    def add_message_multiple_recipients(self, sender_username, receiver_usernames,
hidden_message, client_ip="unknown"):
        """Store the same message for multiple recipients"""
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()

        # Use a transaction for better performance with multiple inserts
        try:
            c.execute("BEGIN TRANSACTION")
            for receiver in receiver_usernames:
                c.execute(
                    "INSERT INTO messages (sender_username, receiver_username, timestamp,
hidden_message) VALUES (?, ?, datetime('now'), ?)",
                    (sender_username, receiver, hidden_message))
            c.execute("COMMIT")
            print(
                f"Server: Message added from '{sender_username}' to multiple recipients: {',
'.join(receiver_usernames)} via {client_ip}")
        except Exception as e:
            c.execute("ROLLBACK")
            print(f"Server: Error adding messages to multiple recipients: {e}")
            raise
        finally:
            conn.close()

    def get_user_email(self, username):
        """Get email address for a specific user"""
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        c.execute("SELECT email FROM users WHERE username=?", (username,))
        email = c.fetchone()
        conn.close()
        return email[0] if email else None
```

```
    def get_users_emails(self, usernames):
        """Get email addresses for multiple users"""
        result = {}
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()

        # Use a tuple for better SQL performance with IN clause
        placeholders = ','.join(['?'] * len(usernames))
        query = f"SELECT username, email FROM users WHERE username IN
({placeholders})"

        c.execute(query, usernames)
        for username, email in c.fetchall():
            result[username] = email

        conn.close()
        return result

    def check_username_exists(self, username):
        """Verify if a username exists in the database"""
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        c.execute("SELECT username FROM users WHERE username=?", (username,))
        exists = c.fetchone() is not None
        conn.close()
        return exists

    def store_image(self, image_name, image_data, sender_username, client_ip="unknown",
compress_large=True):
        """Store an image in the database from binary data"""
        try:
            # Get original size for metadata
            original_size = len(image_data)
            width = height = 0
            compressed = False

            # Optionally compress very large images before storing
            if compress_large and original_size > 5 * 1024 * 1024:  # > 5MB
                print(f"Server: Compressing large image ({original_size // 1024}KB) before storage")
                img = Image.open(io.BytesIO(image_data))
                width, height = img.size
                output = io.BytesIO()

                # Determine format
                img_format = 'PNG'  # Default
                if image_name.lower().endswith('.jpg') or image_name.lower().endswith('.jpeg'):
                    img_format = 'JPEG'

                # Compress with reduced quality for JPEG, or reduced colors for PNG
                if img_format == 'JPEG':
                    img.save(output, format=img_format, quality=85)
                else:
                    img.save(output, format=img_format, optimize=True)
```

```
            image_data = output.getvalue()
            compressed = True
            compressed_size = len(image_data)
            print(
                f"Server: Image compressed from {original_size // 1024}KB to {compressed_size
// 1024}KB ({int((1 - compressed_size / original_size) * 100)}% reduction)")

        # If we didn't compress, still get the dimensions for metadata
        if width == 0 or height == 0:
            try:
                img = Image.open(io.BytesIO(image_data))
                width, height = img.size
            except:
                # If we can't read dimensions, just use 0
                pass

        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        c.execute(
            """INSERT INTO images
               (image_name, image_data, sender_username, creation_date, width, height,
compressed, original_size)
               VALUES (?, ?, ?, datetime('now'), ?, ?, ?, ?)""",
            (image_name, sqlite3.Binary(image_data), sender_username, width, height,
compressed, original_size))
        conn.commit()
        image_id = c.lastrowid
        conn.close()

        print(
            f"Server: Image '{image_name}' ({width}x{height}) stored in database (ID:
{image_id}) for user {sender_username} via {client_ip}")
        return image_id
    except Exception as e:
        print(f"Server: Failed to store image in database: {e}")
        return None

def retrieve_image(self, image_id):
    """Retrieve image data from the database"""
    conn = sqlite3.connect(self.db_name)
    c = conn.cursor()
    c.execute("SELECT image_name, image_data, width, height, compressed, original_size
FROM images WHERE id=?",
              (image_id,))
    result = c.fetchone()
    conn.close()

    if result:
        image_name, image_data, width, height, compressed, original_size = result
        return {
            "name": image_name,
            "data": image_data,
            "width": width,
```

```
            "height": height,
            "compressed": compressed,
            "original_size": original_size
        }
    else:
        print(f"Server: Image with ID {image_id} not found in database")
        return None

def get_message(self, hidden_message, username):
    """Get the message for a specific user"""
    try:
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()
        c.execute("""
            SELECT hidden_message FROM messages
            WHERE hidden_message = ? AND receiver_username = ?
            ORDER BY timestamp DESC LIMIT 1
        """, (hidden_message, username))
        result = c.fetchone()
        conn.close()

        if result:
            return result[0]
        return None
    except Exception as e:
        print(f"Server: Error retrieving message: {e}")
        return None

def cleanup_old_images(self, days=30):
    """Remove images older than specified days to free up database space"""
    try:
        conn = sqlite3.connect(self.db_name)
        c = conn.cursor()

        # Get count of old images
        c.execute("SELECT COUNT(*) FROM images WHERE creation_date <
datetime('now', ?)", (f'-{days} days',))
        count = c.fetchone()[0]

        if count > 0:
            # Delete old images
            c.execute("DELETE FROM images WHERE creation_date < datetime('now', ?)", (f'-
{days} days',))
            conn.commit()
            print(f"Server: Cleaned up {count} images older than {days} days")

        conn.close()
        return count
    except Exception as e:
        print(f"Server: Error during image cleanup: {e}")
        return 0

def optimize_database(self):
    """Run VACUUM to optimize database size and performance"""
```

```
        try:
            conn = sqlite3.connect(self.db_name)
            conn.execute("VACUUM")
            conn.close()
            print(f"Server: Database optimized")
            return True
        except Exception as e:
            print(f"Server: Error optimizing database: {e}")
            return False
```

Gui_app

```
import customtkinter as ctk
from tkinter import filedialog, messagebox, scrolledtext
import tkinter as tk
from PIL import Image, ImageTk
import os
import time
import threading
import math
import random
import tkinter.ttk as ttk

# UI Theme Colors
TECH_BG = "#1E293B"  # Dark blue-gray background
TECH_FG = "#334155"  # Lighter gray for elements
TECH_GREEN = "#4ADE80"  # Bright green for accents
TECH_LIGHT_GREEN = "#86EFAC"  # Lighter green for hovering
TECH_GRAY = "#94A3B8"  # Medium gray for text
TECH_TEXT = "#E2E8F0"  # Light gray/white for text
TECH_WARNING = "#FBBF24"  # Amber for warnings
TECH_ERROR = "#F87171"  # Red for errors
TECH_BLACK = "#0F172A"  # Darker background for binary

# Password Strength Colors
PWD_WEAK = "#EF4444"  # Red for weak password
PWD_MEDIUM = "#F59E0B"  # Orange for medium password
PWD_STRONG = "#10B981"  # Green for strong password


class TechButton(ctk.CTkButton):
    """Custom styled button for consistent UI"""

    def __init__(self, master, **kwargs):
        defaults = {
            'corner_radius': 6,
            'border_width': 1,
            'fg_color': TECH_FG,
            'hover_color': TECH_GREEN,
            'text_color': TECH_TEXT,
            'height': 36,
            'font': ("Consolas", 12)
        }
        for key, value in defaults.items():
            if key not in kwargs:
```

```
            kwargs[key] = value

        super().__init__(master, **kwargs)


class CyberStyleAnimation(tk.Toplevel):
    """Startup animation window with cybersecurity theme"""

    def __init__(self, parent, callback=None):
        super().__init__(parent)
        self.callback = callback
        self.title("StegnoLogic")

        # Match size with main application
        window_width = 1000
        window_height = 700

        # Calculate center position
        screen_width = self.winfo_screenwidth()
        screen_height = self.winfo_screenheight()
        x = (screen_width - window_width) // 2
        y = (screen_height - window_height) // 2

        # Set window size and position
        self.geometry(f"{window_width}x{window_height}+{x}+{y}")
        self.configure(bg=TECH_BLACK)

        # Create canvas for animation
        self.canvas = tk.Canvas(self, bg=TECH_BLACK, highlightthickness=0)
        self.canvas.pack(fill="both", expand=True)

        # Use exact center coordinates
        self.center_x = window_width // 2
        self.center_y = window_height // 2

        # Binary code elements
        self.binary_elements = []
        for _ in range(100):
            x = random.randint(0, window_width)
            y = random.randint(0, window_height)
            binary = random.choice(["0", "1"])
            color = "#" + format(random.randint(30, 120), '02x') + format(random.randint(150, 255),
'02x') + format(
                random.randint(30, 120), '02x')
            opacity = random.uniform(0.1, 0.8)
            size = random.randint(8, 14)
            element = self.canvas.create_text(
                x, y,
                text=binary,
                font=("Courier", size),
                fill=color,
                state="hidden"
            )
            self.binary_elements.append({
```

```
        "element": element,
        "speed": random.uniform(0.5, 3),
        "opacity": opacity
    })

# Logo elements - centered on screen
# Lock outline
self.lock_outline = self.canvas.create_rectangle(
    self.center_x - 75, self.center_y - 80,
    self.center_x + 75, self.center_y + 80,
    outline="",
    fill="",
    width=4,
    state="hidden"
)

# Lock body
self.lock_body = self.canvas.create_rectangle(
    self.center_x - 75, self.center_y - 50,
    self.center_x + 75, self.center_y + 80,
    outline="",
    fill="",
    width=4,
    state="hidden"
)

# Lock shackle (arc)
self.lock_shackle = self.canvas.create_arc(
    self.center_x - 65, self.center_y - 110,
    self.center_x + 65, self.center_y - 30,
    start=0,
    extent=180,
    outline="",
    style="arc",
    width=4,
    state="hidden"
)

# Image icon (mountains inside lock)
self.image_outline = self.canvas.create_rectangle(
    self.center_x - 50, self.center_y - 20,
    self.center_x + 50, self.center_y + 50,
    outline="",
    fill="",
    width=2,
    state="hidden"
)

# Mountain peaks
self.mountain1 = self.canvas.create_polygon(
    [self.center_x - 50, self.center_y + 50,
     self.center_x - 25, self.center_y,
     self.center_x, self.center_y + 30,
     self.center_x + 50, self.center_y + 50],
```

```
    outline="",
    fill="",
    state="hidden"
)

# Second mountain
self.mountain2 = self.canvas.create_polygon(
    [self.center_x - 10, self.center_y + 50,
     self.center_x + 15, self.center_y + 10,
     self.center_x + 35, self.center_y + 30,
     self.center_x + 50, self.center_y + 50],
    outline="",
    fill="",
    state="hidden"
)

# Sun/circle
self.sun = self.canvas.create_oval(
    self.center_x - 35, self.center_y + 10,
    self.center_x - 15, self.center_y + 30,
    outline="",
    fill="",
    state="hidden"
)

# StegnoLogic text
self.logo_text = self.canvas.create_text(
    self.center_x, self.center_y + 150,
    text="StegnoLogic",
    font=("Consolas", 40, "bold"),
    fill=TECH_GREEN,
    state="hidden"
)

# Author text
self.author_text = self.canvas.create_text(
    self.center_x, self.center_y + 200,
    text="by Eitan Shtamler",
    font=("Consolas", 16),
    fill=TECH_GRAY,
    state="hidden"
)

# Progress bar
self.progress_bg = self.canvas.create_rectangle(
    self.center_x - 200, self.center_y + 250,
    self.center_x + 200, self.center_y + 280,
    outline=TECH_GRAY,
    fill=TECH_FG,
    state="hidden"
)

self.progress_fill = self.canvas.create_rectangle(
    self.center_x - 200, self.center_y + 250,
```

```
            self.center_x - 200, self.center_y + 280,
            outline="",
            fill=TECH_GREEN,
            state="hidden"
        )

        # Status text
        self.status_text = self.canvas.create_text(
            self.center_x, self.center_y + 310,
            text="",
            font=("Consolas", 14),
            fill=TECH_GREEN,
            state="hidden"
        )

        # Run animation
        self.run_animation()

    def run_animation(self):
        """Start animation threads"""
        # Start binary animation in a thread
        threading.Thread(target=self._animate_binary, daemon=True).start()

        # Start main animation sequence
        threading.Thread(target=self._run_animation_sequence, daemon=True).start()

    def _animate_binary(self):
        """Animate binary elements in the background"""
        # Show elements
        for elem in self.binary_elements:
            self.canvas.itemconfig(elem["element"], state="normal")

        # Animation loop
        for _ in range(100):  # Limited iterations to avoid endless loop
            for elem in self.binary_elements:
                try:
                    # Move element
                    x, y = self.canvas.coords(elem["element"])
                    if not x or not y:  # Skip if coords not available
                        continue

                    # Random movement
                    new_x = x + random.uniform(-2, 2)
                    new_y = y + elem["speed"]

                    # Keep in bounds
                    window_width = self.winfo_width()
                    window_height = self.winfo_height()
                    if new_y > window_height:
                        new_y = 0
                        new_x = random.randint(0, window_width)

                    # Update position
                    self.canvas.coords(elem["element"], new_x, new_y)
```

```
            # Random opacity changes
            if random.random() < 0.05:
                color = "#" + format(random.randint(30, 120), '02x') + format(random.randint(150,
255),
                                                                              '02x') + format(
                    random.randint(30, 120), '02x')
                self.canvas.itemconfig(elem["element"], fill=color)

            # Randomly change between 0 and 1
            if random.random() < 0.02:
                binary = random.choice(["0", "1"])
                self.canvas.itemconfig(elem["element"], text=binary)
        except Exception:
            continue

    try:
        self.update()
        time.sleep(0.05)
    except Exception:
        break

def _run_animation_sequence(self):
    """Run the main animation sequence"""
    try:
        # Fade in binary elements first
        time.sleep(1)

        # Lock outline - green glow effect
        self.canvas.itemconfig(self.lock_outline, outline=TECH_GREEN, state="normal")
        self.canvas.itemconfig(self.lock_body, outline=TECH_GREEN, state="normal")
        self.canvas.itemconfig(self.lock_shackle, outline=TECH_GREEN, state="normal")
        time.sleep(0.5)

        # Fill lock
        self.canvas.itemconfig(self.lock_body, fill="#143C2E")
        time.sleep(0.3)

        # Image inside lock
        self.canvas.itemconfig(self.image_outline, outline=TECH_GREEN, state="normal")
        time.sleep(0.2)
        self.canvas.itemconfig(self.mountain1, fill="#4ADE80", state="normal")
        time.sleep(0.2)
        self.canvas.itemconfig(self.mountain2, fill="#86EFAC", state="normal")
        time.sleep(0.2)
        self.canvas.itemconfig(self.sun, fill="#FBBF24", state="normal")
        time.sleep(0.5)

        # Logo text with pulse effect
        self.canvas.itemconfig(self.logo_text, state="normal")
        for i in range(5):
            scaling = 1.0 + 0.05 * math.sin(i * 0.5)
            new_size = int(40 * scaling)
            self.canvas.itemconfig(self.logo_text, font=("Consolas", new_size, "bold"))
```

```
    if i % 2 == 0:
        color = "#86EFAC"
    else:
        color = TECH_GREEN
    self.canvas.itemconfig(self.logo_text, fill=color)
    self.update()
    time.sleep(0.1)

# Author text typing effect
self.canvas.itemconfig(self.author_text, text="", state="normal")
author = "by Eitan Shtamler"
for i in range(len(author) + 1):
    self.canvas.itemconfig(self.author_text, text=author[:i])
    self.update()
    time.sleep(0.05)

# Show progress bar
time.sleep(0.3)
self.canvas.itemconfig(self.progress_bg, state="normal")
self.canvas.itemconfig(self.progress_fill, state="normal")

# Fill progress bar
for progress in range(0, 101, 2):
    # Update progress bar
    fill_x = self.center_x - 200 + (400 * progress / 100)
    self.canvas.coords(self.progress_fill, self.center_x - 200, self.center_y + 250, fill_x,
                       self.center_y + 280)

    # Update status message
    if progress < 20:
        status = "Initializing secure modules..."
    elif progress < 40:
        status = "Loading cryptographic systems..."
    elif progress < 60:
        status = "Establishing secure communication..."
    elif progress < 80:
        status = "Preparing steganography engine..."
    else:
        status = "StegnoLogic is ready..."

    # Show status text
    self.canvas.itemconfig(self.status_text, text=status, state="normal")

    self.update()
    time.sleep(0.03)

# Pulse whole lock to signify ready state
for i in range(5):
    # Pulse colors
    if i % 2 == 0:
        glow_color = "#86EFAC"
        bg_color = "#1a4c3a"
    else:
        glow_color = TECH_GREEN
```

```
            bg_color = "#143C2E"

            self.canvas.itemconfig(self.lock_outline, outline=glow_color)
            self.canvas.itemconfig(self.lock_body, outline=glow_color, fill=bg_color)
            self.canvas.itemconfig(self.lock_shackle, outline=glow_color)
            self.canvas.itemconfig(self.image_outline, outline=glow_color)

            self.update()
            time.sleep(0.1)

        # Hold final state then close
        time.sleep(1)

        # Start fade out
        self._fade_to_main_ui()

    except Exception as e:
        print(f"Client: Animation error: {e}")
        # Ensure we still call the callback
        self.after(0, self.close_and_callback)

def _fade_to_main_ui(self):
    """Fade out animation and transition to main UI"""
    try:
        # Create a fade overlay
        fade_overlay = self.canvas.create_rectangle(
            0, 0, self.winfo_width(), self.winfo_height(),
            fill=TECH_BG, outline="", stipple="gray12", state="normal"
        )

        # Fade out animation (change stipple pattern)
        for pattern in ["gray12", "gray25", "gray50", "gray75"]:
            self.canvas.itemconfig(fade_overlay, stipple=pattern)
            self.update()
            time.sleep(0.1)

        # Final solid overlay
        self.canvas.itemconfig(fade_overlay, stipple="")
        self.update()
        time.sleep(0.2)

        # Close and transition to main UI
        self.after(0, self.close_and_callback)

    except Exception as e:
        print(f"Client: Fade error: {e}")
        # Ensure we still call the callback
        self.after(0, self.close_and_callback)

def close_and_callback(self):
    """Close the animation and call the callback function"""
    self.destroy()
    if self.callback:
        self.callback()
```

```
class Gui_app(ctk.CTk):
    def __init__(self, client):
        ctk.CTk.__init__(self)

        # Store client connection
        self.client = client

        # Set appearance
        ctk.set_appearance_mode("dark")
        ctk.set_default_color_theme("green")

        self.title("StegnoLogic")

        # Set window size
        window_width = 1000
        window_height = 700

        # Center the window
        screen_width = self.winfo_screenwidth()
        screen_height = self.winfo_screenheight()
        x = (screen_width - window_width) // 2
        y = (screen_height - window_height) // 2

        # Set geometry
        self.geometry(f"{window_width}x{window_height}+{x}+{y}")
        self.configure(fg_color=TECH_BG)

        # Wait for the animation to finish before showing the main UI
        self.withdraw()  # Hide the main window initially
        animation = CyberStyleAnimation(self, self.show_main_ui)

        # Application state
        self.verification_code = None
        self.verification_code_time = None
        self.login_in_progress = False
        self.recipients = []

    def show_main_ui(self):
        """Show the main UI after animation completes"""
        # Set up the main UI
        self.setup_ui()

        # Show the window
        self.deiconify()

    def create_progress_window(self, title, message=None):
        """Create a progress window for long operations"""
        if message is None:
            message = f"{title} in progress...\nThis might take a while for large images."

        progress_window = tk.Toplevel(self)
        progress_window.title(title)
        progress_window.geometry("400x150")
```

```
        progress_window.configure(bg=TECH_BG)
        progress_window.transient(self)  # Make it float on top of main window

        # Center the window
        screen_width = progress_window.winfo_screenwidth()
        screen_height = progress_window.winfo_screenheight()
        x = (screen_width - 400) // 2
        y = (screen_height - 150) // 2
        progress_window.geometry(f"400x150+{x}+{y}")

        # Add message
        message_label = tk.Label(
            progress_window,
            text=message,
            font=("Consolas", 12),
            fg=TECH_TEXT,
            bg=TECH_BG,
            wraplength=350,
            justify="center"
        )
        message_label.pack(pady=20)

        # Add progress bar (indeterminate mode for unknown duration)
        style = ttk.Style()
        style.configure("TProgressbar", thickness=15, troughcolor=TECH_BG,
background=TECH_GREEN)

        progress = ttk.Progressbar(
            progress_window,
            mode="indeterminate",
            length=300,
            style="TProgressbar"
        )
        progress.pack(pady=20)
        progress.start(10)  # Animation speed

        # Update the window
        progress_window.update()

        # Make it not closable by the user
        progress_window.protocol("WM_DELETE_WINDOW", lambda: None)

        return progress_window

    def setup_ui(self):
        """Set up the main UI components"""
        # Main container
        self.container = ctk.CTkFrame(self, fg_color=TECH_BG)
        self.container.pack(fill="both", expand=True, padx=20, pady=20)

        # Header with connection status and logo
        self.header = ctk.CTkFrame(self.container, fg_color=TECH_FG, height=70)
        self.header.pack(fill="x", pady=(0, 20))
```

```
    # Create logo frame in header with fixed width
    self.logo_frame = ctk.CTkFrame(self.header, fg_color=TECH_FG, width=300,
height=60)
    self.logo_frame.pack(side="left", padx=20, pady=10)
    self.logo_frame.pack_propagate(False)  # Prevent size changes

    # Draw logo on canvas
    logo_canvas = ctk.CTkCanvas(
        self.logo_frame,
        width=300,
        height=50,
        bg=TECH_FG,
        highlightthickness=0
    )
    logo_canvas.pack(fill="both", expand=True)

    # Draw lock logo with proper spacing
    # Lock body
    logo_canvas.create_rectangle(
        20, 15, 50, 40,
        outline=TECH_GREEN,
        fill="#143C2E",
        width=2
    )

    # Lock shackle
    logo_canvas.create_arc(
        22, 5, 48, 25,
        start=0,
        extent=180,
        outline=TECH_GREEN,
        style="arc",
        width=2
    )

    # Image inside lock (simplified)
    logo_canvas.create_rectangle(
        25, 20, 45, 35,
        outline=TECH_GREEN,
        fill="",
        width=1
    )

    # Mountain peak (simplified)
    logo_canvas.create_polygon(
        [25, 35, 32, 25, 40, 32, 45, 35],
        outline="",
        fill=TECH_GREEN
    )

    # Draw the text with more space from the lock
    logo_canvas.create_text(
        150, 25,
        text="StegnoLogic",
```

```
    font=("Consolas", 20, "bold"),
    fill=TECH_GREEN
)

# Connection status
self.status_label = ctk.CTkLabel(
    self.header,
    text="SECURE",
    font=("Consolas", 12),
    text_color=TECH_GREEN
)
self.status_label.pack(side="right", padx=20, pady=10)

# Exit button in header
self.exit_button = TechButton(
    self.header,
    text="Exit",
    font=("Consolas", 12),
    width=60,
    fg_color=TECH_ERROR,
    command=self.close_app
)
self.exit_button.pack(side="right", padx=10, pady=10)

# Initialize frames
self.content_frame = ctk.CTkFrame(self.container, fg_color=TECH_BG)
self.content_frame.pack(fill="both", expand=True)

self.login_register_frame = None
self.login_frame = None
self.register_frame = None
self.main_frame = None
self.verification_frame = None

# Show welcome screen
self.show_welcome_screen()

# Footer - always visible
self.footer = ctk.CTkFrame(self.container, fg_color=TECH_BG, height=20)
self.footer.pack(fill="x", pady=(10, 0))

# Credits on left, guide button on right
self.creator_label = ctk.CTkLabel(
    self.footer,
    text="by Eitan Shtamler",
    font=("Consolas", 10),
    text_color=TECH_GRAY
)
self.creator_label.pack(side="left", padx=10)

self.guide_button = TechButton(
    self.footer,
    text="User Guide",
    font=("Consolas", 10),
```

```
        width=100,
        height=20,
        command=self.show_user_guide
    )
    self.guide_button.pack(side="right", padx=10)

    # Add escape key binding to exit fullscreen
    self.bind("<Escape>", lambda e: self.toggle_fullscreen())

def toggle_fullscreen(self):
    """Toggle fullscreen mode"""
    self.attributes('-fullscreen', not self.attributes('-fullscreen'))

def close_app(self):
    """Exit the application"""
    if messagebox.askokcancel("Exit", "Are you sure you want to exit?"):
        self.destroy()

def show_user_guide(self):
    """Display the user guide in a new window"""
    guide_window = tk.Toplevel(self)
    guide_window.title("StegnoLogic User Guide")
    guide_window.geometry("1000x700")
    guide_window.configure(bg=TECH_BG)

    # Center the window
    screen_width = guide_window.winfo_screenwidth()
    screen_height = guide_window.winfo_screenheight()
    x = (screen_width - 1000) // 2
    y = (screen_height - 700) // 2
    guide_window.geometry(f"1000x700+{x}+{y}")

    # Create scrollable text area
    guide_text = scrolledtext.ScrolledText(
        guide_window,
        font=("Consolas", 14),
        bg=TECH_FG,
        fg=TECH_TEXT,
        padx=20,
        pady=20
    )
    guide_text.pack(fill="both", expand=True, padx=30, pady=30)

    # Set guide content
    guide_content = """
# StegnoLogic User Guide

## Table of Contents
1. Introduction
2. Getting Started
3. Account Management
4. Encoding Messages
5. Decoding Messages
6. Security Features
```

7. Troubleshooting

1. Introduction

StegnoLogic is a secure steganography application that allows you to hide messages in images.
The hidden messages are encrypted for maximum security and can only be decoded by the intended recipients.

2. Getting Started

System Requirements
* Python 3.7+
* Required packages: customtkinter, PIL, rsa

First Launch
When you first launch StegnoLogic, you'll see the welcome screen where you can either login
to an existing account or register a new one.

3. Account Management

Registration
1. Click the "Register" button on the welcome screen
2. Fill in all required fields:
   - Full Name
   - Username (unique identifier)
   - Password (must meet strength requirements)
   - Email (for verification purposes)
3. Click "Register" to create your account

Login
1. Click the "Login" button on the welcome screen
2. Enter your username and password
3. Check your email for a verification code
4. Enter the verification code to complete the login

4. Encoding Messages

1. Navigate to the "Encode" tab
2. Click "Browse" to select an image
3. Type your secret message in the text area
4. Add one or more recipients by entering their usernames
5. Optionally enable/disable email notifications
6. Click "Encode and Send" to process
7. The encoded image will be saved and optionally sent to recipients

5. Decoding Messages

1. Navigate to the "Decode" tab
2. Click "Browse" to select an encoded image
3. Click "Decode Message" to extract any hidden messages
4. If you are an intended recipient, the message will be displayed

```
## 6. Security Features

StegnoLogic includes multiple layers of security:
* RSA encryption for client-server communication
* Two-factor authentication via email
* Secure password storage using PBKDF2 with SHA-256
* Rate limiting to prevent brute force attacks

## 7. Troubleshooting

### Common Issues
* **Login Failures**: Ensure caps lock is off and credentials are correct
* **Email Not Received**: Check spam folder or try again
* **Encoding Errors**: Make sure the message isn't too long for the image
* **Decoding Failures**: Verify that you are an intended recipient
"""
    guide_text.insert("1.0", guide_content)
    guide_text.configure(state="disabled")

    # Close button with better visibility
    close_btn = tk.Button(
        guide_window,
        text="Close",
        font=("Consolas", 16, "bold"),
        bg=TECH_ERROR,
        fg=TECH_TEXT,
        command=guide_window.destroy,
        padx=30,
        pady=15
    )
    close_btn.pack(side="bottom", pady=30)

def show_welcome_screen(self):
    """Show the initial login/register selection screen"""
    self.clear_main_frames()

    self.login_register_frame = ctk.CTkFrame(self.content_frame, fg_color=TECH_BG)
    self.login_register_frame.pack(fill="both", expand=True)

    welcome_label = ctk.CTkLabel(
        self.login_register_frame,
        text="Welcome to StegnoLogic",
        font=("Consolas", 36, "bold"),
        text_color=TECH_GREEN
    )
    welcome_label.pack(pady=(60, 20))

    desc_label = ctk.CTkLabel(
        self.login_register_frame,
        text="Secure Steganography",
        font=("Consolas", 18),
        text_color=TECH_TEXT
    )
    desc_label.pack(pady=(0, 80))
```

```
    button_frame = ctk.CTkFrame(self.login_register_frame, fg_color=TECH_BG)
    button_frame.pack(pady=20)

    login_btn = TechButton(
        button_frame,
        text="Login",
        font=("Consolas", 16, "bold"),
        width=180,
        height=50,
        command=self.show_login
    )
    login_btn.grid(row=0, column=0, padx=30, pady=20)

    register_btn = TechButton(
        button_frame,
        text="Register",
        font=("Consolas", 16, "bold"),
        width=180,
        height=50,
        command=self.show_register
    )
    register_btn.grid(row=0, column=1, padx=30, pady=20)

def show_login(self):
    """Show login screen"""
    self.clear_main_frames()

    self.login_frame = ctk.CTkFrame(self.content_frame, fg_color=TECH_FG)
    self.login_frame.pack(fill="both", expand=True, padx=100, pady=50)

    login_title = ctk.CTkLabel(
        self.login_frame,
        text="Login",
        font=("Consolas", 20, "bold"),
        text_color=TECH_GREEN
    )
    login_title.pack(pady=(30, 20))

    form_frame = ctk.CTkFrame(self.login_frame, fg_color=TECH_FG)
    form_frame.pack(pady=20)

    # Username
    username_label = ctk.CTkLabel(
        form_frame,
        text="Username:",
        font=("Consolas", 12),
        width=80,
        anchor="w"
    )
    username_label.grid(row=0, column=0, padx=10, pady=10, sticky="w")

    self.username_entry = ctk.CTkEntry(
        form_frame,
```

```
        font=("Consolas", 12),
        width=250,
        height=30,
        corner_radius=6,
        border_width=1
    )
    self.username_entry.grid(row=0, column=1, padx=10, pady=10)

    # Password
    password_label = ctk.CTkLabel(
        form_frame,
        text="Password:",
        font=("Consolas", 12),
        width=80,
        anchor="w"
    )
    password_label.grid(row=1, column=0, padx=10, pady=10, sticky="w")

    self.password_entry = ctk.CTkEntry(
        form_frame,
        font=("Consolas", 12),
        width=250,
        height=30,
        corner_radius=6,
        border_width=1,
        show="•"
    )
    self.password_entry.grid(row=1, column=1, padx=10, pady=10)

    # Buttons
    btn_frame = ctk.CTkFrame(self.login_frame, fg_color=TECH_FG)
    btn_frame.pack(pady=30)

    self.login_button = TechButton(
        btn_frame,
        text="Login",
        font=("Consolas", 12, "bold"),
        width=120,
        command=self.start_login
    )
    self.login_button.grid(row=0, column=0, padx=10)

    back_button = TechButton(
        btn_frame,
        text="Back",
        font=("Consolas", 12),
        width=120,
        command=self.show_welcome_screen
    )
    back_button.grid(row=0, column=1, padx=10)

def show_register(self):
    """Show registration screen"""
    self.clear_main_frames()
```

```
self.register_frame = ctk.CTkFrame(self.content_frame, fg_color=TECH_FG)
self.register_frame.pack(fill="both", expand=True, padx=100, pady=30)

register_title = ctk.CTkLabel(
    self.register_frame,
    text="Create New Account",
    font=("Consolas", 20, "bold"),
    text_color=TECH_GREEN
)
register_title.pack(pady=(20, 10))

form_frame = ctk.CTkFrame(self.register_frame, fg_color=TECH_FG)
form_frame.pack(pady=10)

# Full Name
name_label = ctk.CTkLabel(
    form_frame,
    text="Full Name:",
    font=("Consolas", 12),
    width=80,
    anchor="w"
)
name_label.grid(row=0, column=0, padx=10, pady=10, sticky="w")

self.name_entry = ctk.CTkEntry(
    form_frame,
    font=("Consolas", 12),
    width=250,
    height=30,
    corner_radius=6
)
self.name_entry.grid(row=0, column=1, padx=10, pady=10)

# Username
username_label = ctk.CTkLabel(
    form_frame,
    text="Username:",
    font=("Consolas", 12),
    width=80,
    anchor="w"
)
username_label.grid(row=1, column=0, padx=10, pady=10, sticky="w")

self.username_entry = ctk.CTkEntry(
    form_frame,
    font=("Consolas", 12),
    width=250,
    height=30,
    corner_radius=6
)
self.username_entry.grid(row=1, column=1, padx=10, pady=10)

# Password
```

```
password_label = ctk.CTkLabel(
    form_frame,
    text="Password:",
    font=("Consolas", 12),
    width=80,
    anchor="w"
)
password_label.grid(row=2, column=0, padx=10, pady=10, sticky="w")

self.password_entry = ctk.CTkEntry(
    form_frame,
    font=("Consolas", 12),
    width=250,
    height=30,
    corner_radius=6,
    show="•"
)
self.password_entry.grid(row=2, column=1, padx=10, pady=10)

# Password strength indicator
strength_frame = ctk.CTkFrame(form_frame, fg_color=TECH_FG)
strength_frame.grid(row=2, column=2, padx=10, pady=10)

self.pwd_strength_label = ctk.CTkLabel(
    strength_frame,
    text="Strength",
    font=("Consolas", 10),
    text_color=TECH_GRAY
)
self.pwd_strength_label.pack(pady=(0, 5))

self.pwd_strength_canvas = ctk.CTkCanvas(
    strength_frame,
    width=80,
    height=10,
    bg=TECH_BG,
    highlightthickness=1,
    highlightbackground=TECH_GRAY
)
self.pwd_strength_canvas.pack()

self.pwd_strength_bar = self.pwd_strength_canvas.create_rectangle(
    0, 0, 0, 10,
    fill=PWD_WEAK,
    width=0
)

# Bind password entry to update strength indicator
self.password_entry.bind("<KeyRelease>", self.update_password_strength)

# Password requirements
pass_req_label = ctk.CTkLabel(
    self.register_frame,
    text="Password must contain: 8+ chars, 1 uppercase, 1 lowercase, 1 number, 1 special
```

```
char",
        font=("Consolas", 10),
        text_color=TECH_GRAY
    )
    pass_req_label.pack(pady=(0, 10))

    # Email
    email_label = ctk.CTkLabel(
        form_frame,
        text="Email:",
        font=("Consolas", 12),
        width=80,
        anchor="w"
    )
    email_label.grid(row=3, column=0, padx=10, pady=10, sticky="w")

    self.email_entry = ctk.CTkEntry(
        form_frame,
        font=("Consolas", 12),
        width=250,
        height=30,
        corner_radius=6
    )
    self.email_entry.grid(row=3, column=1, padx=10, pady=10)

    # Buttons
    btn_frame = ctk.CTkFrame(self.register_frame, fg_color=TECH_FG)
    btn_frame.pack(pady=20)

    register_btn = TechButton(
        btn_frame,
        text="Register",
        font=("Consolas", 12, "bold"),
        width=120,
        command=self.register
    )
    register_btn.grid(row=0, column=0, padx=10)

    back_btn = TechButton(
        btn_frame,
        text="Back",
        font=("Consolas", 12),
        width=120,
        command=self.show_welcome_screen
    )
    back_btn.grid(row=0, column=1, padx=10)

def update_password_strength(self, event=None):
    """Update the password strength indicator based on password content"""
    password = self.password_entry.get()

    # Calculate strength score
    score = 0
```

```
    # Length check
    if len(password) >= 8:
        score += 1

    # Uppercase check
    if any(char.isupper() for char in password):
        score += 1

    # Lowercase check
    if any(char.islower() for char in password):
        score += 1

    # Digit check
    if any(char.isdigit() for char in password):
        score += 1

    # Special character check
    if any(not char.isalnum() for char in password):
        score += 1

    # Update strength bar
    width = (score / 5) * 80  # Scale to fit canvas width

    # Set color based on score
    if score < 2:
        color = PWD_WEAK  # Red - Weak
        strength_text = "Weak"
    elif score < 4:
        color = PWD_MEDIUM  # Orange - Medium
        strength_text = "Medium"
    else:
        color = PWD_STRONG  # Green - Strong
        strength_text = "Strong"

    # Update label and bar
    self.pwd_strength_label.configure(text=strength_text, text_color=color)
    self.pwd_strength_canvas.coords(self.pwd_strength_bar, 0, 0, width, 10)
    self.pwd_strength_canvas.itemconfig(self.pwd_strength_bar, fill=color)

def clear_main_frames(self):
    """Clear all main content frames"""
    for frame in [self.login_register_frame, self.login_frame,
                  self.register_frame, self.main_frame, self.verification_frame]:
        if frame:
            frame.pack_forget()

def start_login(self):
    """Process login attempt through server"""
    if self.login_in_progress:
        return

    self.login_in_progress = True
    username = self.username_entry.get()
    password = self.password_entry.get()
```

```
    if not username or not password:
        messagebox.showerror("Error", "Please enter both username and password")
        self.login_in_progress = False
        return

    # Show logging in status
    self.login_button.configure(text="Logging in...", state="disabled")
    self.update()

    try:
        # Send login request to server
        success, user, verification_code = self.client.login_user(username, password)

        # Reset button state
        self.login_button.configure(text="Login", state="normal")

        if success:
            # Store verification code
            self.verification_code = verification_code
            self.verification_code_time = time.time()
            self.show_verification_frame()
        else:
            self.client.log_login_attempt(username, "Failed")
            messagebox.showerror("Error", "Invalid username or password")
            self.login_in_progress = False

    except Exception as e:
        self.login_button.configure(text="Login", state="normal")
        messagebox.showerror("Connection Error", f"Could not connect to server: {str(e)}")
        self.login_in_progress = False

def show_verification_frame(self):
    """Display verification code entry screen"""
    self.clear_main_frames()

    self.verification_frame = ctk.CTkFrame(self.content_frame, fg_color=TECH_FG)
    self.verification_frame.pack(fill="both", expand=True, padx=100, pady=50)

    verify_title = ctk.CTkLabel(
        self.verification_frame,
        text="Two-Factor Authentication",
        font=("Consolas", 20, "bold"),
        text_color=TECH_GREEN
    )
    verify_title.pack(pady=(30, 10))

    verify_info = ctk.CTkLabel(
        self.verification_frame,
        text="A verification code has been sent to your email",
        font=("Consolas", 12),
        text_color=TECH_TEXT
    )
    verify_info.pack(pady=(0, 20))
```

```
code_frame = ctk.CTkFrame(self.verification_frame, fg_color=TECH_FG)
code_frame.pack(pady=10)

code_label = ctk.CTkLabel(
    code_frame,
    text="Enter Code:",
    font=("Consolas", 12),
    width=100,
    anchor="w"
)
code_label.grid(row=0, column=0, padx=10, pady=10)

self.code_entry = ctk.CTkEntry(
    code_frame,
    font=("Consolas", 14),
    width=150,
    height=30,
    corner_radius=6,
    justify="center"
)
self.code_entry.grid(row=0, column=1, padx=10, pady=10)

# Buttons
btn_frame = ctk.CTkFrame(self.verification_frame, fg_color=TECH_FG)
btn_frame.pack(pady=20)

verify_btn = TechButton(
    btn_frame,
    text="Verify",
    font=("Consolas", 12, "bold"),
    width=120,
    fg_color=TECH_GREEN,
    hover_color=TECH_LIGHT_GREEN,
    command=self.verify_code
)
verify_btn.grid(row=0, column=0, padx=10)

back_btn = TechButton(
    btn_frame,
    text="Back",
    font=("Consolas", 12),
    width=120,
    command=lambda: self.show_login()
)
back_btn.grid(row=0, column=1, padx=10)

# Timer
self.timer_label = ctk.CTkLabel(
    self.verification_frame,
    text="Code expires in: 05:00",
    font=("Consolas", 10),
    text_color=TECH_GRAY
)
```

```
        self.timer_label.pack(pady=10)

        self.update_timer()
        self.login_in_progress = False

    def update_timer(self):
        """Update the countdown timer for verification code"""
        if hasattr(self,
                   'verification_code_time') and self.verification_frame and
self.verification_frame.winfo_exists():
            # Check if verification_code_time is None
            if self.verification_code_time is None:
                self.timer_label.configure(text="Code not sent yet")
                return

            elapsed = int(time.time() - self.verification_code_time)
            remaining = max(0, 300 - elapsed)  # 5 minutes

            if remaining > 0:
                self.timer_label.configure(text=f"Code expires in: {remaining // 60:02d}:{remaining
% 60:02d}")
                self.after(1000, self.update_timer)
            else:
                self.timer_label.configure(text="Code expired. Please try again.")

    def verify_code(self):
        """Verify the entered code"""
        entered_code = self.code_entry.get()
        if self.verification_code_time is None or time.time() - self.verification_code_time > 300:
# 5 minutes
            messagebox.showinfo("Expired", "Code has expired. Please try again.")
            self.show_login()
            return
        elif entered_code == self.verification_code:
            # Log successful login
            self.client.log_login_attempt(self.client.current_user, "Success")
            self.setup_main_interface()
        else:
            messagebox.showerror("Error", "Wrong verification code.")

    def register(self):
        """Register a new user through server"""
        name = self.name_entry.get()
        username = self.username_entry.get()
        password = self.password_entry.get()
        email = self.email_entry.get()

        if not name or not username or not password or not email:
            messagebox.showerror("Error", "Please fill all fields")
            return

        try:
            # Register user on server
            success, message = self.client.register_user(username, name, password, email)
```

```
            if success:
                messagebox.showinfo("Success", "Account created successfully. You can now log
in.")
                self.show_login()
            else:
                messagebox.showerror("Registration Error", message)
        except Exception as e:
            messagebox.showerror("Connection Error", f"Could not connect to server: {str(e)}")

    def setup_main_interface(self):
        """Set up main application after login"""
        self.clear_main_frames()

        self.main_frame = ctk.CTkFrame(self.content_frame, fg_color=TECH_BG)
        self.main_frame.pack(fill="both", expand=True)

        # Tab view for encoding/decoding
        self.tabs = ctk.CTkTabview(self.main_frame, fg_color=TECH_FG)
        self.tabs.pack(fill="both", expand=True, padx=20, pady=(20, 10))

        # Add tabs
        self.tabs.add("Encode")
        self.tabs.add("Decode")

        # Set up tabs
        self.setup_encode_tab(self.tabs.tab("Encode"))
        self.setup_decode_tab(self.tabs.tab("Decode"))

        # User info label - added to header
        user_label = ctk.CTkLabel(
            self.header,
            text=f"User: {self.client.current_user}",
            font=("Consolas", 12),
            text_color=TECH_GREEN
        )
        user_label.pack(side="right", padx=20, pady=10)

        # Log off button in header
        log_off_btn = TechButton(
            self.header,
            text="Log Off",
            font=("Consolas", 12),
            width=80,
            fg_color=TECH_ERROR,
            command=self.log_off
        )
        log_off_btn.pack(side="right", padx=10, pady=10)

    def setup_encode_tab(self, parent):
        """Set up the encode tab"""
        # Image selection
        image_frame = ctk.CTkFrame(parent, fg_color=TECH_FG)
        image_frame.pack(fill="x", padx=20, pady=10)
```

```
image_label = ctk.CTkLabel(
    image_frame,
    text="Select Image:",
    font=("Consolas", 12),
    width=100,
    anchor="w"
)
image_label.grid(row=0, column=0, padx=10, pady=10)

self.image_entry = ctk.CTkEntry(
    image_frame,
    font=("Consolas", 12),
    width=300,
    height=30
)
self.image_entry.grid(row=0, column=1, padx=10, pady=10)

browse_btn = TechButton(
    image_frame,
    text="Browse",
    font=("Consolas", 12),
    width=80,
    command=self.browse_image
)
browse_btn.grid(row=0, column=2, padx=10, pady=10)

# Character limit info
self.max_chars_label = ctk.CTkLabel(
    parent,
    text="Max characters: Not calculated",
    font=("Consolas", 10),
    text_color=TECH_GRAY
)
self.max_chars_label.pack(anchor="w", padx=30, pady=(0, 5))

# Message input
message_label = ctk.CTkLabel(
    parent,
    text="Secret Message:",
    font=("Consolas", 12, "bold"),
    anchor="w"
)
message_label.pack(anchor="w", padx=30, pady=(10, 5))

# Fix for message visibility - use contrasting colors
self.message_text = ctk.CTkTextbox(
    parent,
    font=("Consolas", 14),
    corner_radius=6,
    border_width=1,
    border_color=TECH_FG,
    fg_color="#1E293B",  # Darker background
    text_color=TECH_TEXT,  # Light text color
```

```
    height=120
)
self.message_text.pack(fill="x", padx=30, pady=5)

# Character counter
self.char_count_var = ctk.StringVar(value="Characters: 0")
self.char_count_label = ctk.CTkLabel(
    parent,
    textvariable=self.char_count_var,
    font=("Consolas", 10),
    text_color=TECH_GRAY
)
self.char_count_label.pack(anchor="e", padx=30, pady=(0, 10))

# Bind text change to update counter
self.message_text.bind("<KeyRelease>", self.update_char_count)

# Recipients
recipients_frame = ctk.CTkFrame(parent, fg_color=TECH_FG)
recipients_frame.pack(fill="x", padx=30, pady=10)

recipients_label = ctk.CTkLabel(
    recipients_frame,
    text="Recipients:",
    font=("Consolas", 12),
    width=80,
    anchor="w"
)
recipients_label.grid(row=0, column=0, padx=10, pady=10)

self.recipient_entry = ctk.CTkEntry(
    recipients_frame,
    font=("Consolas", 12),
    width=200,
    height=30
)
self.recipient_entry.grid(row=0, column=1, padx=10, pady=10)

add_btn = TechButton(
    recipients_frame,
    text="Add",
    font=("Consolas", 12),
    width=60,
    command=self.add_recipient
)
add_btn.grid(row=0, column=2, padx=5, pady=10)

clear_btn = TechButton(
    recipients_frame,
    text="Clear All",
    font=("Consolas", 12),
    width=80,
    command=self.clear_recipients
)
```

```
clear_btn.grid(row=0, column=3, padx=5, pady=10)

# Recipients list
recipients_display_frame = ctk.CTkFrame(parent)
recipients_display_frame.pack(fill="x", padx=30, pady=(0, 10))

self.recipients_display = ctk.CTkTextbox(
    recipients_display_frame,
    font=("Consolas", 12),
    height=80,
    corner_radius=6
)
self.recipients_display.pack(fill="x", pady=5)
self.recipients_display.configure(state="disabled")

# Send email option
self.send_email_var = ctk.BooleanVar(value=True)
self.send_email_checkbox = ctk.CTkCheckBox(
    parent,
    text="Send emails to recipients",
    font=("Consolas", 12),
    checkbox_width=20,
    checkbox_height=20,
    variable=self.send_email_var
)
self.send_email_checkbox.pack(anchor="w", padx=30, pady=10)

# Button frame
button_frame = ctk.CTkFrame(parent, fg_color=TECH_BG)
button_frame.pack(fill="x", pady=10)

# Add 'New Encode' button to reset form
new_encode_btn = TechButton(
    button_frame,
    text="New Encode",
    font=("Consolas", 12),
    width=120,
    height=30,
    fg_color=TECH_WARNING,
    command=self.clear_encode_form
)
new_encode_btn.pack(side="left", padx=30, pady=10)

# Encode button
self.encode_button = TechButton(
    button_frame,
    text="Encode and Send",
    font=("Consolas", 14, "bold"),
    width=200,
    height=40,
    fg_color=TECH_GREEN,
    hover_color=TECH_LIGHT_GREEN,
    command=self.encode_and_send
)
```

```
    self.encode_button.pack(side="right", padx=30, pady=20)

def clear_encode_form(self):
    """Clear all fields in the encode form"""
    self.image_entry.delete(0, ctk.END)
    self.message_text.delete("1.0", ctk.END)
    self.recipient_entry.delete(0, ctk.END)
    self.clear_recipients()
    self.max_chars_label.configure(text="Max characters: Not calculated")
    self.char_count_var.set("Characters: 0")

def setup_decode_tab(self, parent):
    """Set up the decode tab"""
    # Image selection
    image_frame = ctk.CTkFrame(parent, fg_color=TECH_FG)
    image_frame.pack(fill="x", padx=20, pady=20)

    image_label = ctk.CTkLabel(
        image_frame,
        text="Select Image:",
        font=("Consolas", 12),
        width=100,
        anchor="w"
    )
    image_label.grid(row=0, column=0, padx=10, pady=10)

    self.decode_image_entry = ctk.CTkEntry(
        image_frame,
        font=("Consolas", 12),
        width=300,
        height=30
    )
    self.decode_image_entry.grid(row=0, column=1, padx=10, pady=10)

    browse_btn = TechButton(
        image_frame,
        text="Browse",
        font=("Consolas", 12),
        width=80,
        command=self.browse_decode_image
    )
    browse_btn.grid(row=0, column=2, padx=10, pady=10)

    # Button frame
    button_frame = ctk.CTkFrame(parent, fg_color=TECH_BG)
    button_frame.pack(fill="x", pady=10)

    # New Decode button
    new_decode_btn = TechButton(
        button_frame,
        text="New Decode",
        font=("Consolas", 12),
        width=120,
        height=36,
```

```
            fg_color=TECH_WARNING,
            command=self.clear_decode_form
        )
        new_decode_btn.pack(side="left", padx=20)

        # Decode button
        self.decode_button = TechButton(
            button_frame,
            text="Decode Message",
            font=("Consolas", 14, "bold"),
            width=200,
            height=40,
            command=self.decode_image_handler
        )
        self.decode_button.pack(side="right", padx=20)

        # Message display area
        message_frame = ctk.CTkFrame(parent)
        message_frame.pack(fill="both", expand=True, padx=20, pady=10)

        message_label = ctk.CTkLabel(
            message_frame,
            text="Decoded Message:",
            font=("Consolas", 12, "bold"),
            anchor="w"
        )
        message_label.pack(anchor="w", padx=10, pady=(10, 5))

        self.decoded_text = ctk.CTkTextbox(
            message_frame,
            font=("Consolas", 14),
            corner_radius=6,
            border_width=1,
            border_color=TECH_FG,
            height=200
        )
        self.decoded_text.pack(fill="both", expand=True, padx=10, pady=10)
        self.decoded_text.configure(state="disabled")

        # Status label
        self.decode_status = ctk.CTkLabel(
            parent,
            text="No message decoded yet",
            font=("Consolas", 12),
            text_color=TECH_GRAY
        )
        self.decode_status.pack(pady=10)

    def clear_decode_form(self):
        """Clear decode form fields"""
        self.decode_image_entry.delete(0, ctk.END)
        self.decoded_text.configure(state="normal")
        self.decoded_text.delete("1.0", ctk.END)
        self.decoded_text.configure(state="disabled")
```

```
    self.decode_status.configure(text="No message decoded yet", text_color=TECH_GRAY)

def update_char_count(self, event=None):
    """Update character count and check against limit"""
    text = self.message_text.get("1.0", "end-1c")
    count = len(text)

    if hasattr(self, 'max_chars') and self.max_chars > 0:
        if count > self.max_chars:
            self.char_count_var.set(f"Characters: {count}/{self.max_chars} - TOO LONG!")
            self.char_count_label.configure(text_color=TECH_ERROR)
        else:
            self.char_count_var.set(f"Characters: {count}/{self.max_chars}")
            self.char_count_label.configure(text_color=TECH_GRAY)
    else:
        self.char_count_var.set(f"Characters: {count}")

def browse_image(self):
    """Open file dialog to select image for encoding"""
    filename = filedialog.askopenfilename(
        title="Select Image",
        filetypes=[("Image files", "*.png *.jpg *.jpeg *.bmp *.gif")]
    )
    if filename:
        self.image_entry.delete(0, ctk.END)
        self.image_entry.insert(0, filename)

        # Check file size
        try:
            file_size = os.path.getsize(filename) / (1024 * 1024)  # Size in MB
            if file_size > 10:  # If larger than 10MB
                messagebox.showwarning(
                    "Large Image",
                    f"The selected image is {file_size:.1f}MB in size.\n\n" +
                    "Processing very large images may take several minutes and could be resized " +
                    "to improve performance while maintaining quality."
                )
        except:
            pass

        try:
            # Create progress window for calculation
            progress_window = self.create_progress_window(
                "Calculating",
                "Calculating maximum message length...\n" +
                "This might take a moment for large images."
            )

            # Use a thread to prevent UI freezing
            def calculation_thread():
                try:
                    # Calculate max message length through server or estimation
                    self.max_chars = self.client.calculate_max_message_length(filename)
```

```
                # Update UI in main thread
                self.after(0, lambda: self.max_chars_label.configure(text=f"Max characters: {self.max_chars}"))
                self.after(0, lambda: self.update_char_count())
                self.after(0, lambda: progress_window.destroy())
            except Exception as e:
                self.after(0, lambda: progress_window.destroy())
                self.after(0, lambda: messagebox.showerror("Error",
                                                           f"Could not calculate message length: {str(e)}"))

        # Start calculation thread
        threading.Thread(target=calculation_thread, daemon=True).start()

    except Exception as e:
        messagebox.showerror("Error", f"Could not calculate message length: {str(e)}")

def browse_decode_image(self):
    """Open file dialog to select image for decoding"""
    filename = filedialog.askopenfilename(
        title="Select Image to Decode",
        filetypes=[("Image files", "*.png *.jpg *.jpeg *.bmp *.gif")]
    )
    if filename:
        self.decode_image_entry.delete(0, ctk.END)
        self.decode_image_entry.insert(0, filename)

def add_recipient(self):
    """Add a recipient to the list"""
    recipient = self.recipient_entry.get().strip()
    if not recipient:
        messagebox.showerror("Error", "Please enter a recipient username")
        return

    try:
        # Check if username exists through server
        if not self.client.check_username_exists(recipient):
            messagebox.showerror("Error", f"Username '{recipient}' does not exist")
            return

        # Check if already added
        if recipient in self.recipients:
            messagebox.showerror("Error", f"Username '{recipient}' already added")
            return

        # Add to list
        self.recipients.append(recipient)
        self.recipient_entry.delete(0, ctk.END)

        # Update display
        self.update_recipients_display()
    except Exception as e:
        messagebox.showerror("Connection Error", f"Could not check username: {str(e)}")

def clear_recipients(self):
```

```
        """Clear all recipients"""
        self.recipients = []
        self.update_recipients_display()

    def update_recipients_display(self):
        """Update the recipients display"""
        self.recipients_display.configure(state="normal")
        self.recipients_display.delete("1.0", ctk.END)

        if not self.recipients:
            self.recipients_display.insert(ctk.END, "No recipients added yet.")
        else:
            for i, recipient in enumerate(self.recipients, 1):
                self.recipients_display.insert(ctk.END, f"{i}. {recipient}\n")

        self.recipients_display.configure(state="disabled")

    def encode_and_send(self):
        """Encode a message into an image and send to recipients through server"""
        image_path = self.image_entry.get()
        message = self.message_text.get("1.0", "end-1c")
        should_send_email = self.send_email_var.get()

        # Validation
        if not image_path:
            messagebox.showerror("Error", "Please select an image")
            return

        if not message:
            messagebox.showerror("Error", "Please enter a message")
            return

        if not self.recipients:
            messagebox.showerror("Error", "Please add at least one recipient")
            return

        # Check message length
        if hasattr(self, 'max_chars') and len(message) > self.max_chars:
            messagebox.showerror("Error", f"Message too long. Maximum {self.max_chars}
characters for this image.")
            return

        # Show processing status
        self.encode_button.configure(text="Processing...", state="disabled")
        self.update()

        # Create progress window
        progress_window = self.create_progress_window(
            "Encoding Image",
            f"Encoding message for {len(self.recipients)} recipient(s)...\n\n" +
            "This process may take several minutes for large images."
        )

        # Use a thread to avoid freezing the UI
```

```
    def encoding_thread():
        try:
            # Send encode request to server
            success, output_path, image_id = self.client.encode_and_send(image_path, message,
self.recipients)

            # Reset button state in the main thread
            self.after(0, lambda: self.encode_button.configure(text="Encode and Send",
state="normal"))

            # Close progress window
            self.after(0, lambda: progress_window.destroy())

            if success:
                # Send emails if requested
                if should_send_email:
                    recipient_emails = self.client.get_users_emails(self.recipients)

                    if recipient_emails:
                        # Create a new progress window for sending emails
                        email_progress_window = self.create_progress_window(
                            "Sending Emails",
                            f"Sending emails to {len(recipient_emails)} recipients..."
                        )

                        # Send emails in a separate thread
                        def send_emails_thread():
                            try:
                                results = self.client.send_emails_to_multiple_recipients(
                                    recipient_emails,
                                    "StegnoLogic: New Encoded Message",
                                    "You have received an encoded message. Open with StegnoLogic to
decode.",
                                    output_path
                                )

                                # Close progress window
                                self.after(0, lambda: email_progress_window.destroy())

                                # Count successes and failures
                                success_count = sum(1 for success in results.values() if success)
                                failure_count = len(results) - success_count

                                if failure_count == 0:
                                    self.after(0, lambda: messagebox.showinfo(
                                        "Success",
                                        f"Message sent successfully to all {success_count} recipients"
                                    ))
                                else:
                                    self.after(0, lambda: messagebox.showinfo(
                                        "Partial Success",
                                        f"Message sent to {success_count} recipients. Failed to send to
{failure_count} recipients."
                                    ))
```

```
                except Exception as e:
                    self.after(0, lambda: email_progress_window.destroy())
                    self.after(0, lambda: messagebox.showerror("Error",
                                                               f"Failed to send emails: {str(e)}"))

                # Start the email thread
                threading.Thread(target=send_emails_thread, daemon=True).start()
            else:
                messagebox.showerror("Error", "Failed to get recipient emails")
        else:
            messagebox.showinfo("Success", "Image encoded successfully")

            self.show_images(image_path, output_path, message)
        else:
            messagebox.showerror("Error", "Failed to encode image")
    except Exception as e:
        # Close progress window and reset button in case of error
        self.after(0, lambda: progress_window.destroy())
        self.after(0, lambda: self.encode_button.configure(text="Encode and Send",
state="normal"))
        self.after(0, lambda: messagebox.showerror("Error", f"Failed to encode image:
{str(e)}"))

    # Start the encoding thread
    encoding_thread = threading.Thread(target=encoding_thread, daemon=True)
    encoding_thread.start()

def decode_image_handler(self):
    """Decode a message from an image through server"""
    image_path = self.decode_image_entry.get()
    if not image_path:
        messagebox.showerror("Error", "Please select an image")
        return

    # Show decoding status
    self.decode_button.configure(text="Decoding...", state="disabled")
    self.update()

    # Create progress window
    progress_window = self.create_progress_window(
        "Decoding Image",
        "Searching for hidden message...\n\n" +
        "This process may take several minutes for large images."
    )

    # Use a thread to avoid freezing the UI
    def decoding_thread():
        try:
            # Send decode request to server
            success, message = self.client.decode_image(image_path)

            # Reset button state in the main thread
            self.after(0, lambda: self.decode_button.configure(text="Decode Message",
state="normal"))
```

```
            # Close progress window
            self.after(0, lambda: progress_window.destroy())

            if success:
                # Show the image and message
                self.after(0, lambda: self.show_decoded_image(image_path, message))

                # Update status and text area
                self.after(0, lambda: self.decode_status.configure(
                    text="Message decoded successfully!",
                    text_color=TECH_GREEN
                ))

                self.after(0, lambda: self.decoded_text.configure(state="normal"))
                self.after(0, lambda: self.decoded_text.delete("1.0", ctk.END))
                self.after(0, lambda: self.decoded_text.insert("1.0", message))
                self.after(0, lambda: self.decoded_text.configure(state="disabled"))
            else:
                self.after(0, lambda: self.decode_status.configure(
                    text=message,
                    text_color=TECH_ERROR
                ))

                self.after(0, lambda: self.decoded_text.configure(state="normal"))
                self.after(0, lambda: self.decoded_text.delete("1.0", ctk.END))
                self.after(0, lambda: self.decoded_text.configure(state="disabled"))

                self.after(0, lambda: messagebox.showinfo("Decode Status", message))
        except Exception as e:
            # Close progress window and reset button in case of error
            self.after(0, lambda: progress_window.destroy())
            self.after(0, lambda: self.decode_button.configure(text="Decode Message",
state="normal"))
            self.after(0, lambda: self.decode_status.configure(
                text=f"Error: {e}",
                text_color=TECH_ERROR
            ))
            self.after(0, lambda: messagebox.showerror("Error", f"Failed to decode image:
{str(e)}"))

    # Start the decoding thread
    decoding_thread = threading.Thread(target=decoding_thread, daemon=True)
    decoding_thread.start()

  def show_decoded_image(self, image_path, message):
    """Show decoded image and message in a new window"""
    window = tk.Toplevel(self)
    window.title("Decoded Message")
    window.geometry("1000x700")
    window.configure(bg=TECH_BG)

    # Center the window
    screen_width = window.winfo_screenwidth()
```

```
screen_height = window.winfo_screenheight()
x = (screen_width - 1000) // 2
y = (screen_height - 700) // 2
window.geometry(f"1000x700+{x}+{y}")

# Title
title_label = tk.Label(
    window,
    text="Decoded Message",
    font=("Consolas", 20, "bold"),
    fg=TECH_GREEN,
    bg=TECH_BG
)
title_label.pack(pady=(30, 20))

# Image display
try:
    img = Image.open(image_path)
    img.thumbnail((500, 500))
    img_tk = ImageTk.PhotoImage(img)

    img_label = tk.Label(window, image=img_tk, bg=TECH_BG)
    img_label.image = img_tk  # Keep reference to prevent garbage collection
    img_label.pack(pady=20)
except Exception:
    error_label = tk.Label(
        window,
        text="Could not display image",
        font=("Consolas", 14),
        fg=TECH_ERROR,
        bg=TECH_BG
    )
    error_label.pack(pady=20)

# Message display
msg_label = tk.Label(
    window,
    text="Hidden Message:",
    font=("Consolas", 16, "bold"),
    fg=TECH_GREEN,
    bg=TECH_BG
)
msg_label.pack(pady=(20, 10))

text_frame = tk.Frame(window, bg=TECH_BG)
text_frame.pack(fill="both", expand=True, padx=40, pady=10)

msg_text = tk.Text(
    text_frame,
    font=("Consolas", 14),
    wrap="word",
    height=10,
    bg=TECH_FG,
    fg=TECH_TEXT,
```

```
        bd=1,
        relief="solid"
    )
    msg_text.pack(fill="both", expand=True)
    msg_text.insert("1.0", message)
    msg_text.config(state="disabled")

    # Close button with better visibility
    close_btn = tk.Button(
        window,
        text="Close",
        font=("Consolas", 16, "bold"),
        bg=TECH_ERROR,
        fg=TECH_TEXT,
        command=window.destroy,
        padx=30,
        pady=15
    )
    close_btn.pack(side="bottom", pady=30)

def show_images(self, original_path, encoded_path, message):
    """Show original and encoded images in a new window"""
    window = tk.Toplevel(self)
    window.title("Encoded Image")
    window.geometry("1000x800")
    window.configure(bg=TECH_BG)

    # Center the window
    screen_width = window.winfo_screenwidth()
    screen_height = window.winfo_screenheight()
    x = (screen_width - 1000) // 2
    y = (screen_height - 800) // 2
    window.geometry(f"1000x800+{x}+{y}")

    # Title
    title_label = tk.Label(
        window,
        text="Image Encoding Result",
        font=("Consolas", 20, "bold"),
        fg=TECH_GREEN,
        bg=TECH_BG
    )
    title_label.pack(pady=(30, 20))

    # Images frame
    images_frame = tk.Frame(window, bg=TECH_BG)
    images_frame.pack(fill="both", expand=True, padx=30, pady=10)

    # Original image
    orig_frame = tk.Frame(images_frame, bg=TECH_BG)
    orig_frame.grid(row=0, column=0, padx=20, pady=20)

    orig_label = tk.Label(
        orig_frame,
```

```
    text="Original Image",
    font=("Consolas", 16, "bold"),
    fg=TECH_TEXT,
    bg=TECH_BG
)
orig_label.pack(pady=(0, 15))

# Display original image
try:
    orig_img = Image.open(original_path)
    orig_img.thumbnail((450, 450))
    orig_img_tk = ImageTk.PhotoImage(orig_img)

    orig_img_label = tk.Label(orig_frame, image=orig_img_tk, bg=TECH_BG)
    orig_img_label.image = orig_img_tk
    orig_img_label.pack()
except Exception:
    error_label = tk.Label(
        orig_frame,
        text="Could not display image",
        font=("Consolas", 14),
        fg=TECH_ERROR,
        bg=TECH_BG
    )
    error_label.pack(pady=20)

# Encoded image
enc_frame = tk.Frame(images_frame, bg=TECH_BG)
enc_frame.grid(row=0, column=1, padx=20, pady=20)

enc_label = tk.Label(
    enc_frame,
    text="Encoded Image",
    font=("Consolas", 16, "bold"),
    fg=TECH_TEXT,
    bg=TECH_BG
)
enc_label.pack(pady=(0, 15))

# Display encoded image
try:
    enc_img = Image.open(encoded_path)
    enc_img.thumbnail((450, 450))
    enc_img_tk = ImageTk.PhotoImage(enc_img)

    enc_img_label = tk.Label(enc_frame, image=enc_img_tk, bg=TECH_BG)
    enc_img_label.image = enc_img_tk
    enc_img_label.pack()
except Exception:
    error_label = tk.Label(
        enc_frame,
        text="Could not display image",
        font=("Consolas", 14),
        fg=TECH_ERROR,
```

```
        bg=TECH_BG
    )
    error_label.pack(pady=20)

# Info section
info_frame = tk.Frame(window, bg=TECH_BG)
info_frame.pack(fill="x", padx=40, pady=20)

# Recipients
recipients_label = tk.Label(
    info_frame,
    text=f"Recipients: {', '.join(self.recipients)}",
    font=("Consolas", 14),
    fg=TECH_TEXT,
    bg=TECH_BG,
    anchor="w",
    justify="left"
)
recipients_label.pack(fill="x", pady=(10, 10))

# Message
message_label = tk.Label(
    info_frame,
    text="Hidden Message:",
    font=("Consolas", 14, "bold"),
    fg=TECH_GREEN,
    bg=TECH_BG,
    anchor="w",
    justify="left"
)
message_label.pack(fill="x", pady=(15, 5))

message_text = tk.Text(
    info_frame,
    font=("Consolas", 14),
    wrap="word",
    height=8,
    bg=TECH_FG,
    fg=TECH_TEXT,
    bd=1,
    relief="solid"
)
message_text.pack(fill="x", pady=10)
message_text.insert("1.0", message)
message_text.config(state="disabled")

# Close button with better visibility
close_btn = tk.Button(
    window,
    text="Close",
    font=("Consolas", 16, "bold"),
    bg=TECH_ERROR,
    fg=TECH_TEXT,
    command=window.destroy,
```

```
            padx=30,
            pady=15
        )
        close_btn.pack(side="bottom", pady=30)

    def log_off(self):
        """Log off and return to welcome screen"""
        response = messagebox.askyesno("Log Off", "Are you sure you want to log off?")
        if response:
            # Reset client user
            self.client.current_user = None

            # Reset local state
            self.verification_code = None
            self.verification_code_time = None
            self.recipients = []
            self.login_in_progress = False

            # Clear user info from header
            for widget in self.header.winfo_children():
                if isinstance(widget, ctk.CTkLabel) and "User: " in widget.cget("text"):
                    widget.destroy()
                elif isinstance(widget, TechButton) and widget.cget("text") == "Log Off":
                    widget.destroy()

            self.show_welcome_screen()
            messagebox.showinfo("Logged Out", "You have been logged out successfully.")
```

requirements

```
rsa==4.9
pycryptodome==3.17.0
python-dotenv==1.0.0
pillow==9.4.0
customtkinter==5.1.3
#pip install -r requirements.txt
```

.env

```
SERVER_HOST=0.0.0.0

SERVER_PORT=8080

EMAIL_ADDRESS=your_email@gmail.com

EMAIL_PASSWORD=your_app_password
```